LLMs in Modern Agentic Systems: A Systems Survey

Mehrdad Zakershahrak[1] Samira Ghodratnama[2] Mohammadali Karfarma[3]

[1,2]Honorary Research Fellow, School of Computing, Macquarie University

mehrdad.zakershahrak@mq.edu.au samira.ghodratnama@mq.edu.au makarfarma@gmail.com

LLMs in Modern Agentic Systems Project Page

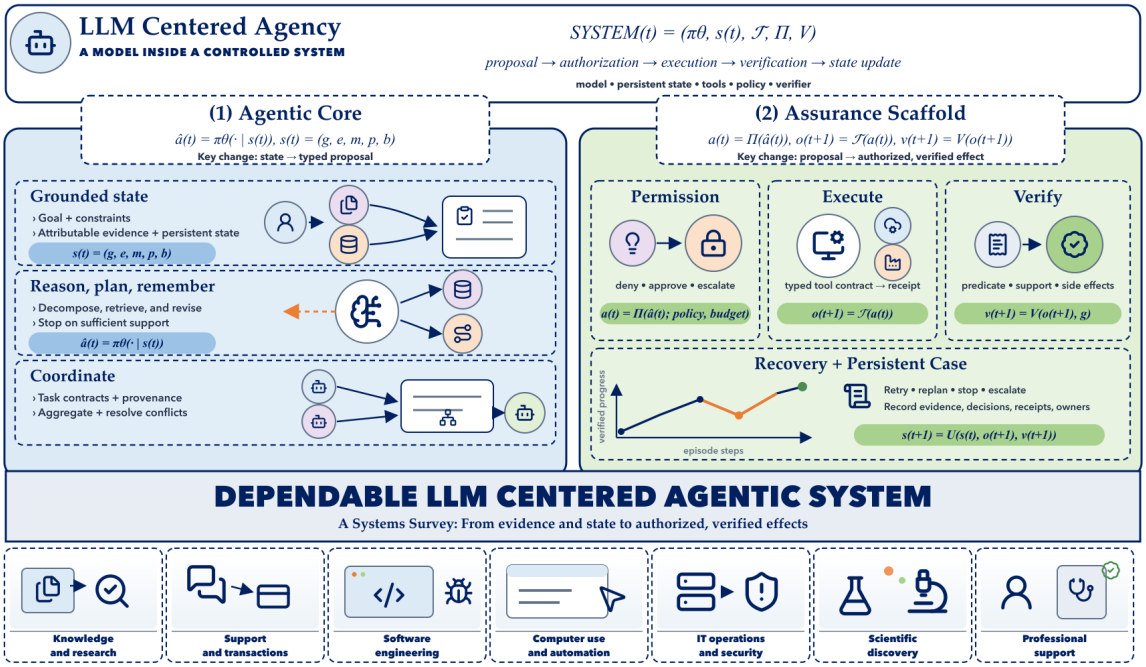


Figure 1. Systems overview used in this survey. An agentic core transforms grounded state into typed proposals through reasoning, memory, and coordination. An assurance scaffold then constrains permissions, executes typed tool contracts, verifies effects, and records recovery evidence in a persistent case. The same control structure specializes across application domains without treating greater autonomy as a maturity ladder.

Abstract

Large language models are increasingly embedded in systems that retrieve evidence, maintain state, invoke tools, coordinate specialized components, and act over extended horizons. We use *LLM-centered agentic systems* as the umbrella term for this systems family while distinguishing fixed language model applications and retrieval pipelines from workflows, bounded agents, and multi-agent systems by adaptive control, persistent state, and action authority. Our central argument is that dependable agency is a representation and transition control problem as much as a model capability problem: heterogeneous evidence must be transformed into bounded, reviewable, and verifiably completed effects through explicit, typed, persistent data structures. This comprehensive survey develops that argument through July 2026 across six interacting layers: grounding and perception; memory and state; reasoning and planning; action and tool interfaces; coordination and interoperability; and safety, governance, and observability. We connect classical agent foundations to work on agentic retrieval, memory, tool use learning, graph-based orchestration, self-improvement, multi-agent coordination, interoperability protocols, and long-horizon reinforcement learning. Comparative tables and evidence maps distinguish what component a method changes, what feedback it consumes, what authority it receives, and what its evaluation can actually establish. We synthesize benchmarks and deployment evidence across verified outcome quality, trajectory efficiency, recovery, memory-mediated action, cost, side effects, and human accountability. Across this literature, retrieved content is not automatically admissible evidence, model output is not authorization, a successful tool response is not proof of the intended effect, and a benchmark score is not a transferable guarantee about an unrecorded system. Reliable agency instead requires a persistent case that links evidence, state, proposals, permissions, execution receipts, verified effects, resources, and responsibility. We conclude with lifecycle invariants and a research agenda for grounded, governable, interoperable, and resource-bounded agency.

Keywords

LLM-centered agentic systems; agentic AI; retrieval-augmented generation; autonomous agents; memory; planning; tool use; multi-agent systems; evaluation; safety; interoperability

1 Introduction

Large language models (LLMs) have changed the interface to artificial intelligence by allowing users to specify tasks, constraints, and desired outputs in natural language. Their fluency and broad task coverage, however, should not be confused with dependable autonomy. A standalone LLM has no persistent state outside its current context, no guaranteed access to current or private information, and no intrinsic ability to execute actions in an external environment. It may also produce unsupported claims, follow irrelevant instructions embedded in retrieved content, or lose track of constraints across a long interaction. These limitations are especially consequential when a model is used not only to generate text but also to query databases, edit files, operate software, or trigger business processes [53].

Retrieval-Augmented Generation (RAG) addresses one part of this problem by conditioning generation on evidence retrieved from an external corpus [46, 49, 80]. RAG makes knowledge easier to update, enables source attribution, and supports domain adaptation without retraining the underlying model. Nevertheless, retrieval alone does not determine when additional evidence is needed, how a complex goal should be decomposed, whether a tool result is valid, or how the system should recover when an action fails. These requirements motivate a shift from *context augmentation* to *closed-loop control*: the system repeatedly observes its environment, updates state, selects an action, evaluates the result, and revises its plan.

The resulting design space is commonly described as *agentic AI*. The term is useful but incon-

Table 1. Operational taxonomy used in this survey. The boundary between categories is determined by control flow, state, and action authority rather than by marketing terminology.

System type	Control flow	State and tools	Typical example
LLM application	One model invocation or a fixed sequence of invocations	Context-local state; no external action or only application-controlled post-processing	Summarization, extraction, classification
RAG application	Fixed retrieve–construct context–generate pipeline	External knowledge access; retrieval policy is largely predetermined	Enterprise question answering with citations
LLM workflow	Developer-authored graph, state machine, or business process	May include tools, retries, and persistent state, but branching is mostly specified in advance	Document processing pipeline with approval gates
Bounded agent	The model selects among allowed actions and may revise the plan from observations	Explicit task state, tools, stopping criteria, budgets, and permission boundaries	Research, coding, or support agent operating in a sandbox
Multi-agent system	Multiple bounded agents exchange tasks, evidence, or control	Distributed state and role-specific tools; requires coordination and conflict resolution	Manager–worker, debate, or blackboard architecture

sistently applied. Some systems labeled as agents are deterministic workflows with an LLM embedded in one step; others allow the model to select tools but not alter the control flow; still others maintain long-term state, delegate to specialized agents, and adapt their strategy over many interactions. This ambiguity makes it difficult to compare methods, interpret benchmark results, and reason about deployment risk. We therefore use *LLM-centered agentic systems* as the umbrella term for the full systems family studied here. The umbrella includes RAG and workflows as enabling system forms, but the operational taxonomy below reserves *agent* for systems in which a model adaptively participates in choosing the next action.

1.1 From language model applications to agentic systems

We distinguish five system types by where control decisions are made and how much authority the model receives; the categories make no claim about human-like intelligence.

An *agent* is a system in which a model helps choose the next action from a bounded action space using a goal, current state, and observations. An *agentic* system is the surrounding sociotechnical system of agents, tools, policies, memory, evaluators, and human oversight. The broader *LLM-centered agentic system* may also contain fixed retrieval and workflow components and is analyzed by execution form, functional layer, adaptation locus, and evidence. Ordinary prompting remains a baseline rather than an agent; production agents should operate under explicit permissions, budgets, stopping criteria, and escalation rules.

In *agentic RAG*, retrieval becomes an action within the control loop: the system decides whether and where to retrieve, reformulates queries, inspects conflicts, and stops when support is sufficient. Agents do not replace RAG; retrieval becomes one capability among planning, memory, computation, communication, and execution.

The practical story is an escalation of responsibility. A model first assists with a response; retrieval adds current or attributable evidence; a workflow makes predictable processing explicit; and a bounded agent is warranted only when the next useful step cannot be fixed in advance. Each move expands both capability and failure surface. The survey follows that trade-off from

evidence through decision and action to evaluation and governance.

1.2 Historical progression and current systems view

Modern agents combine several research lines: classical beliefs, goals, planning, and coordination [133, 183]; transformer-based language-conditioned policies [9, 169]; non-parametric knowledge through RAG [80]; and tool-augmented reasoning and interaction [116, 144, 201]. Orchestration frameworks now expose state, retries, branches, and approval, while MCP and A2A seek to standardize tool and agent communication [1, 110].

These components are compositional rather than chronological. Reliable systems are usually hybrid. A bounded agent may use a deterministic workflow for high-risk steps, a symbolic planner for constrained subproblems, RAG for evidence, and a human approval gate before committing an irreversible action. The appropriate architecture depends on task uncertainty, the cost of errors, environmental dynamics, and the availability of objective verification.

1.3 Positioning relative to related surveys

Existing surveys offer complementary views: broad reviews organize agent capabilities and applications [131, 177, 188], while focused reviews cover RAG [46, 152], memory [213], tool learning [149], evaluation [203], or self-improvement [136]. Their root terms are not interchangeable: self-improving agents update model parameters or scaffolds from feedback, whereas many RAG, workflow, and bounded-agent systems remain fixed between episodes. Table 2 makes the distinction explicit.

The overview in Figure 2 turns this broad scope into a navigable map. It assigns 98 representative sources to one primary locus in the survey taxonomy, while the text and comparative tables interpret the larger literature and its cross-cutting contributions.

1.4 Scope and literature selection

This systems-oriented narrative survey covers LLM-centered agentic systems through July 2026across the execution forms and functional layers in Figure 1; it does not claim to enumerate every implementation. We synthesize peer-reviewed research, public benchmarks, influential preprints where mature coverage is unavailable, and official specifications for evolving protocols. Included work contributes materially to grounding, architecture, state, planning, tools, coordination, adaptation, evaluation, safety, interoperability, or deployment. Ordinary chat and fixed pipelines appear only when they clarify a boundary, and vendor deployments are treated as case studies rather than independent evidence of general effectiveness.

Because products and leaderboards change quickly, we emphasize durable abstractions and benchmark methodology: frameworks are compared by control and operational guarantees, and results expose capability gaps rather than declare permanent rankings.

1.5 Contributions and organization

The survey makes the following contributions:

- **An umbrella term and operational definitions.** We use LLM-centered agentic systems for the broader systems family, then distinguish LLM applications, RAG, deterministic workflows, bounded agents, and multi-agent systems by control, state, and action authority.
- **A multidimensional systems taxonomy.** We compare execution form, six functional layers, locus of adaptation, and evidence rather than presenting RAG, workflows, and agents

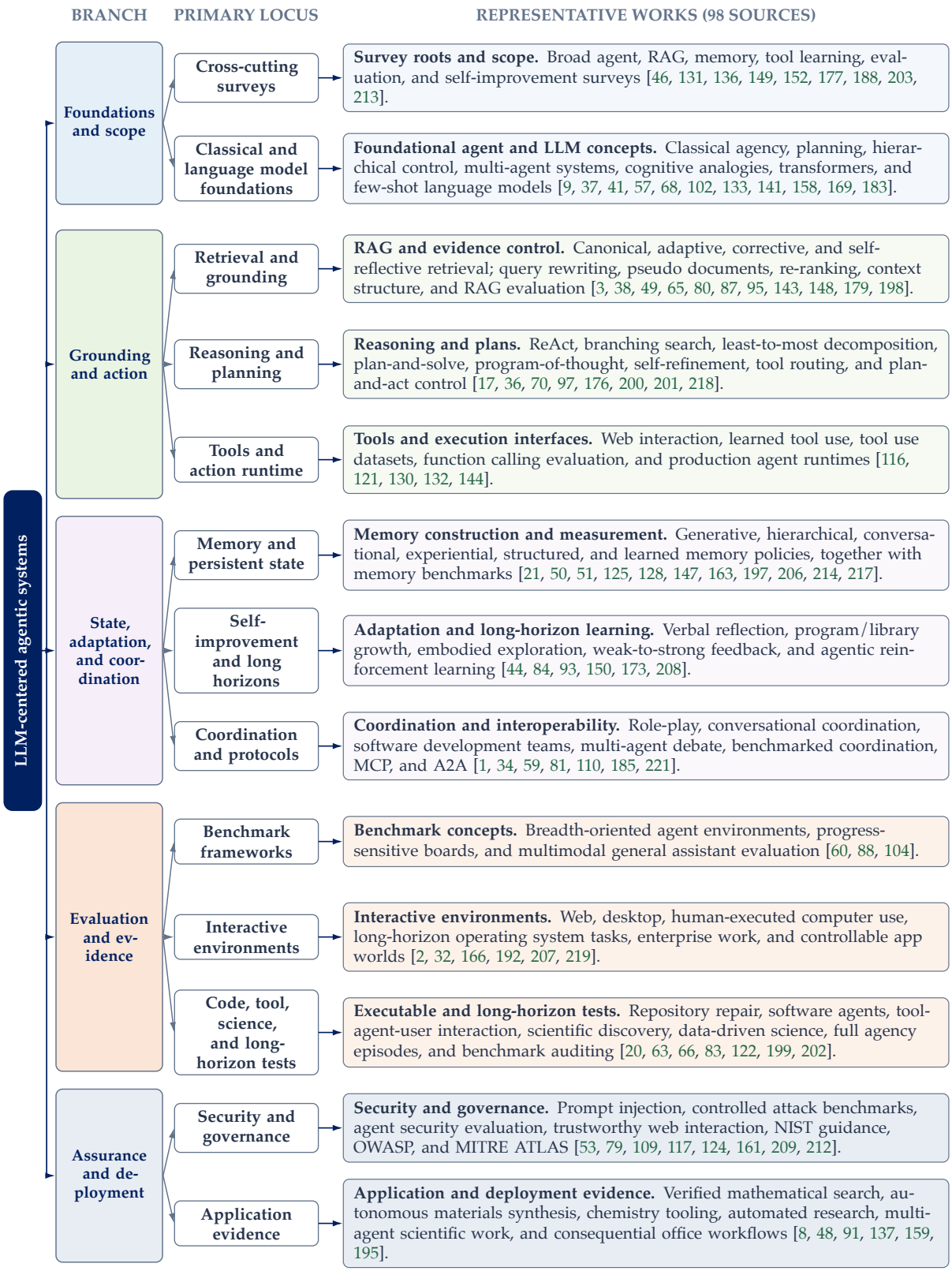


Figure 2. Comprehensive literature taxonomy for LLM-centered agentic systems. Inspired by the umbrella-first, work-level mapping in the self-improvement survey of Ren et al. [136], this atlas assigns 98 representative sources to one primary locus. The placement expresses the source's principal contribution, not an exclusive claim: many methods span multiple layers, and the surrounding text and tables cover additional work.

Table 2. Positioning against representative surveys. Primary denotes the organizing lens rather than every topic mentioned by a paper.

Survey	Umbrella or root term	Primary organizing lens	Relationship to this survey
Gao et al. [46]	Retrieval-augmented generation	Naive, advanced, and modular retrieval pipelines	Deep grounding coverage; limited treatment of action authority, multi-agent coordination, and governance
Wang et al. [177]	LLM-based autonomous agents	Construction, applications, and evaluation	Broad agent overview; less emphasis on cross-layer reliability contracts and operational boundaries
Xi et al. [188]	LLM-based agents	Brain, perception, action, and application domains	Comprehensive capability view; RAG-to-workflow boundaries and governance are not the central lens
Zhang et al. [213]	LLM-based agent memory	Memory sources, forms, operations, and evaluation	Deep memory coverage used here as one layer within a larger systems architecture
Shi et al. [149]	Automatic tool agents	Tool discovery, learning, selection, and use	Deep tool learning coverage; does not by itself integrate memory, governance, or deployment evidence
Yehudai et al. [203]	LLM agent evaluation	Benchmarks, metrics, and evaluation protocols	Deep evaluation treatment; this survey connects evaluation to architecture and action authority
Ren et al. [136]	Self-improving agents	Foundation model versus scaffold improvement	Provides the umbrella-first taxonomic inspiration; its scope begins where feedback changes the model or scaffold
This survey	**LLM-centered agentic systems**	**Execution form, six functional layers, adaptation locus, and evidence**	**Integrates RAG, workflows, agents, evaluation, and governance through bounded system authority**

as a single maturity ladder.

- **Integrated foundations and current mechanisms.** We connect classical agency and planning to modern RAG, memory policies, long-horizon learning, MCP, and A2A.
- **Comparative evidence and deployment guidance.** We map mechanisms, benchmarks, safety sources, and application studies to the claims they support, then connect outcome, trajectory, cost, recovery, and side effect metrics to typed contracts, least privilege, approval, verification, tracing, and replay.

The review follows one systems narrative: RAG moves from grounding a response to deciding when to seek evidence; foundations explain state, planning, uncertainty, and bounded action; architecture turns them into memory, decision, tool, and coordination components; evaluation asks what can justify a capability claim; and governance and applications test whether that claim survives consequential deployment. The final sections identify open problems along the same evidence-to-action-to-accountability path.

2 Retrieval-Augmented Generation and the Transition to Agentic Retrieval

Retrieval-Augmented Generation (RAG) is the first point in this survey at which external information becomes explicit system state rather than remaining implicit in model parameters or transient prompting. In its canonical form, a retriever maps a query to passages from an external corpus and a generator conditions on the query and those passages to produce an answer [80]. The systems consequence is more important than the pipeline shorthand: each retrieved passage can be represented as an evidence object carrying content, source identity and version, retrieval time, access scope, score, and provenance, then persisted with the claims it is used to support. This boundary makes frequently changing or access-controlled knowledge replaceable and auditable without retraining the model. It does not make a retrieved passage true, nor does it grant the model action authority; it makes the available support inspectable. The model is therefore a synthesis and decision component operating over a bounded evidence record, while the external collection remains the attributable source.

The familiar four-stage summary (indexing, retrieval, context construction, and generation) hides the state transformations that make RAG dependable. Indexing creates addressable, versioned evidence units; retrieval selects candidates under a query and access policy; context construction normalizes, reranks, deduplicates, and assigns evidence identifiers; generation consumes that bounded record; and post-generation verification maps claims back to supporting items. Production systems therefore add ingestion controls, access filtering, ranking, citation mapping, caching, monitoring, and verification. Because the retrieval layer may combine sparse lexical search, dense vectors, graph traversal, structured queries, or external search APIs, RAG is a family of architectures whose choices determine which evidence and metadata the system can represent [46]. Work on LLM-mediated information retrieval reaches the same systems conclusion: retrieval quality is coupled to model selection, orchestration, scaling cost, privacy, interpretability, and personalization [49]. A retrieval metric cannot therefore be interpreted in isolation; the pipeline must preserve provenance and access constraints and expose its transformations as inspectable state transitions.

2.1 Why RAG is useful and what it does not guarantee

RAG's four recurring advantages all follow from externalizing evidence. An evidence store can be refreshed without retraining the base model; retrieval can enforce scope and access policy before content enters model context; passage identifiers and document versions can support citation and audit; and domain-specific corpora can improve task performance without full model adaptation. None of these properties makes the resulting answer true. Failures arise at distinct layers: retrieval can return irrelevant, stale, duplicated, or unauthorized items; representation can omit provenance or access metadata; context use can fail when the model overlooks relevant passages, especially in long contexts [87]; and retrieved content can carry adversarial instructions across the trust boundary [53]. Even an intact citation establishes association, not entailment of the attached claim. RAGAS-style evaluation therefore separates context relevance, answer relevance, and faithfulness rather than treating retrieval success as a single score [38]. RAG reduces reliance on parametric memory and makes support auditable, but it does not provide a factuality guarantee; downstream control still needs explicit support, conflict, abstention, and stopping states.

Evaluating RAG as an evidence-to-claim transformation requires three questions and three corresponding records:

1. **Evidence discovery**: Does the retrieval record contain sufficient relevant information from permissible sources, together with the query, source version, rank, and retrieval

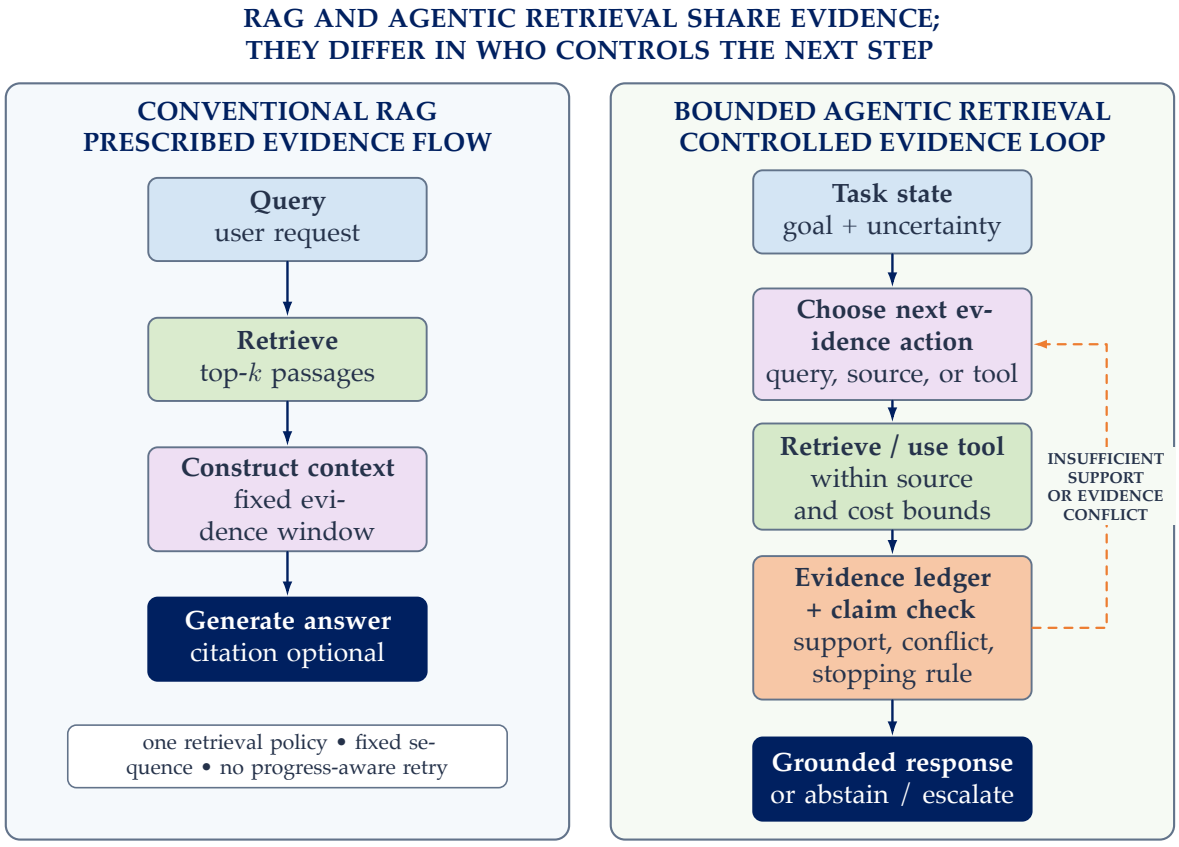


Figure 3. Conventional RAG and bounded agentic retrieval share an evidence pipeline but not a control model. Conventional RAG executes a prescribed retrieval sequence. Bounded agentic retrieval preserves task and evidence state, chooses retrieval actions inside an explicit authority boundary, and uses support or conflict to re-plan, abstain, or escalate. The additional loop is justified only when those decisions are observable and governed.

metadata needed to explain the selection?

2. **Evidence use**: Does the generation trace show that the model correctly interpreted the retrieved information, resolved or exposed conflicts, and carried relevant uncertainty into the answer?

3. **Claim support**: Can each externally verifiable claim be linked to identifiable evidence that entails it, or be explicitly marked unsupported and handled through abstention or further search?

The three levels are causally ordered but not interchangeable. A correct answer may come from parametric memory despite failed retrieval; a high-quality context may be ignored or misinterpreted; and correctly used evidence may still fail to entail the final wording of a claim. Evaluation should therefore report the levels separately and retain their artifacts for attribution and replay, so improvement at one layer cannot conceal regression at another.

2.2 Evolution of RAG architectures

The familiar progression from naive to advanced, modular, and agentic RAG is useful only when read as a relocation of control rather than a maturity ladder. Naive RAG fixes the retrieve–construct–generate sequence; advanced RAG improves one or more transformations inside that sequence; modular RAG exposes retrieval, ranking, synthesis, and verification as composable components; and agentic RAG allows a controller to select the next evidence action from explicit

task and evidence state [46, 152]. The categories overlap, and production systems are often hybrid. We therefore compare them along three operational axes: who selects the next retrieval transition, which evidence and intermediate state persist, and which stopping and verification boundaries constrain the result.

Naive RAG. Naive RAG fixes the evidence policy: encode one query, retrieve top-k passages, construct context, and generate [80]. This path is inexpensive and auditable for narrow questions over stable corpora, but it cannot revise the source, query, or stopping decision when evidence is weak. Query wording, chunking, and context position therefore remain consequential failure points [87]. Naive RAG is a fixed workflow, not a primitive agent.

Advanced RAG. Advanced RAG keeps a developer-authored flow while improving query, candidate, or context transformations. Rewriting and pseudo document expansion change the represented information need [95, 179]; REPLUG adapts retrieval around a black-box model [148]; and FLARE and Self-RAG condition retrieval on uncertainty or learned critique [3, 65]. These mechanisms make evidence acquisition responsive, but the enclosing system remains an adaptive workflow when its branches and authority are fixed in advance.

Modular RAG. Modular RAG exposes routing, retrieval, ranking, context construction, synthesis, and verification as components with typed artifacts. RAPTOR adds multi-resolution retrieval [143], while Corrective RAG represents relevance and invokes fallback search when local evidence is weak [198]. Modularity localizes evaluation and supports heterogeneous sources, but expands the contract surface for schemas, freshness, access, and provenance. A developer-authored routing graph is still a workflow; its importance is that it creates inspectable state surfaces on which bounded control can operate.

Agentic RAG. Agentic RAG moves selection of the next evidence transition to a controller that can rewrite, change source, inspect conflict, request clarification, or abstain from explicit task and evidence state. ReAct illustrates how observations can revise subsequent action [201], and Agentic RAG generalizes that pattern to iterative evidence acquisition [152]. The benefit is adaptive multi-hop search; the cost is looping, noisy evidence overfitting, boundary crossing, and indirect prompt injection [53]. Agentic retrieval is therefore bounded evidence action under permissions, budgets, stopping rules, and traceable state.

2.3 Data, indexing, and freshness

The quality of a RAG system is bounded before the first query is issued. Ingestion converts a source document into persistent, typed evidence records; each record should retain document identity and version, timestamps, ownership, access policy, lineage, and the offsets needed to reconstruct the source passage. Chunking then determines the semantic unit to which retrieval scores, citations, and later claim checks refer. Units must be small enough for discriminative retrieval but complete enough to preserve definitions, tables, code structure, and cross-references. Fixed-length chunks are operationally convenient yet can sever precisely the context needed for entailment. Structure-aware and hierarchical schemes preserve section and parent–child relations and allow local passages to remain connected to broader summaries, as illustrated by RAG architecture surveys and RAPTOR [46, 143]. Bounded action therefore begins with the data model: a later controller cannot reliably enforce freshness, authorization, or provenance that ingestion discarded.

Table 3. Comparison of representative retrieval-augmented architectures. The rows describe dominant tendencies rather than strict implementation requirements.

Architecture	Retrieval policy	Control flow	Primary strength	Characteristic risk
Naive RAG	One top-k query over a fixed index	Fixed retrieve–generate pipeline	Simplicity and low latency	Query mismatch and unsupported synthesis
Advanced RAG	Rewriting, hybrid search, re-ranking, or iterative retrieval	Mostly developer-defined	Better evidence recall and precision	Pipeline complexity and evaluation leakage between stages
Modular RAG	Source routing and task-specific retrieval modules	Explicit graph or state machine	Inspectability and heterogeneous evidence access	Configuration burden and inconsistent component contracts
Agentic RAG	Model- or policy-selected retrieval actions	Adaptive observe–retrieve–reason–verify loop	Multi-hop search and uncertainty-driven evidence collection	Runaway search, adversarial content, and non-reproducible trajectories

Freshness is not merely an indexing schedule; it is a consistency property of the evidence state. Vector and lexical stores can accept incremental insertions, deletions, and replacements, but every derived object (chunk, embedding, summary, graph edge, and cache entry) must inherit the source's identity and version so that a change can invalidate its descendants. This dependency graph allows the system to distinguish current, stale, and revoked evidence and to record which version supported a later claim or action. For stable corpora, scheduled incremental maintenance may suffice; for volatile facts, evidence should instead be obtained at decision time from an authoritative database, API, or search index. FreshLLMs demonstrates why this distinction matters: models struggle on fast-changing questions, whereas retrieving current search evidence improves factuality [170]; CRAG similarly invokes web search as a corrective source when local retrieval is unreliable [198]. In the survey's terms, index maintenance is therefore not back-office plumbing: it is the temporal contract between evidence state and bounded action.

Authorization is part of evidence validity, not a filter applied to prose after generation. A passage is eligible evidence only when the requesting principal and tenant are permitted to use its source at retrieval time. Row-, document-, and tenant-level policy labels must therefore travel with the evidence record and be enforced in the retrieval query; cache keys, shared summaries, and reranking stages must preserve the same boundary. Post-generation redaction is strictly weaker because the model has already observed the material, and a retrieved instruction may redirect later tool use. Greshake et al. show that indirect prompts embedded in retrieved data can induce data theft and unauthorized API behavior [53], while OWASP recommends least privilege access to data sources and authorization in downstream systems rather than delegation to the model [124]. The retrieval trace should consequently record the principal, policy version, authorization decision, and evidence objects released. This turns access control into an auditable constraint on the evidence-to-action transition.

2.4 Context construction and evidence compression

Retrieval produces candidates; context construction selects the evidence the model will actually see. The objective is sufficient, complementary support within a bounded context, not maxi-

Table 4. Representative RAG mechanisms and the evidence each isolates. Reported improvements are not directly comparable because tasks, corpora, generators, and retrieval budgets differ.

Method	Mechanism changed	Evaluation focus	Systems implication and limitation
Query2doc [179]	Generates a pseudo document to expand the query	Retrieval effectiveness across standard search datasets	Improves query representation, but added generated text can introduce unsupported terms
Query rewriting [95]	Rewrites the user request before retrieval	Downstream QA and retrieval quality	Separates intent interpretation from retrieval, but relies on rewrite faithfulness
REPLUG [148]	Trains or weights a retriever around a frozen black-box LM	Language model and knowledge-intensive tasks	Supports model-agnostic retrieval; does not supply an agent control loop
FLARE [65]	Triggers retrieval during generation from anticipated uncertainty	Long-form knowledge-intensive generation	Demonstrates adaptive retrieval, but retrieval policy and generation quality remain coupled
RAPTOR [143]	Retrieves over a tree of recursively summarized clusters	Multi-step question answering over long corpora	Adds multi-resolution evidence access; summaries create additional lineage and fidelity obligations
Self-RAG [3]	Learns retrieve, relevance, support, and utility reflection tokens	Factuality and task performance across QA and generation	Integrates retrieval decisions with generation; requires trained reflection behavior
Corrective RAG [198]	Scores retrieved evidence and invokes a corrective search path	Robustness to low-quality initial retrieval	Makes recovery explicit; external search broadens trust and provenance boundaries
RAGAS [38]	Decomposes evaluation into faithfulness and context-related metrics	Reference-light RAG evaluation	Enables component diagnosis, but judge-based metrics require calibration against humans

mum passage count. Position effects make indiscriminate length unreliable [87]; RECOMP shows that selective compression can sharply reduce context with little task loss and can omit unhelpful augmentation [194]. Compression is therefore an evidence transformation whose source identity, version, offsets, and lineage must remain recoverable.

Long context and retrieval solve different problems. A context window holds transient input; it does not select evidence, enforce freshness, or preserve state. Retrieval remains useful even at longer context lengths and can match substantially larger fine-tuned contexts with less computation [196]. Long context is therefore workspace, retrieval locates and refreshes evidence, and structured memory persists state. The design question is which information remains transient, fetched on demand, or durably retained.

2.5 When retrieval should become agentic

Retrieval should become agentic only when consequential evidence choices are difficult to prespecify yet observable enough to verify. Fixed retrieval remains preferable when sources, rank-

ing, and stopping are known. When evidence is insufficient or conflicting, an escalation ladder can permit a bounded rewrite, second source, clarification, or abstention before granting open-ended planning. FLARE and CRAG show how adaptive retrieval can sit behind explicit triggers [65, 198]. Each added degree of freedom should carry source, step, token, and time limits plus a stopping rule, so autonomy increases only with observability and control.

2.6 From evidence access to closed-loop agency

RAG solves evidence access; closed-loop agency begins when evidence can revise state and the next bounded action. ReAct and Reflexion show how observations and evaluated outcomes can alter later behavior [150, 201], but reliability requires the goal, evidence, proposal, authorization, result, and remaining uncertainty to remain explicit. Retrieval then becomes one perception mechanism within a bounded state machine that chooses among typed alternatives and verifies their postconditions. Evidence becomes operational only through that controlled transition into action.

3 Foundations of Agentic Systems

The term *agent* predates large language models by decades. Classical artificial intelligence defines agency through a control loop: an entity perceives an environment, maintains information relevant to an objective, and selects actions that alter what happens next [141]; multi-agent systems additionally make autonomy, interaction, and coordination first-class concerns [183]. The LLM era changes the representational substrate, not this contract. Natural language can encode goals, observations, plans, tool requests, and messages, but those strings become reliable system state only when mapped into explicit, typed, and persistent records. An LLM is therefore best understood as a policy and representation transformer within an agent architecture, not as the architecture itself. A modern agentic system must preserve what was observed, which evidence was accepted, which goal is active, what action was proposed, why it was permitted, and what result followed. That data model is the bridge from evidence to bounded action. This section uses reactive control, deliberative planning, decision-making under uncertainty, hierarchical organization, and multi-agent coordination to explain which component should own each transition and under what constraints.

3.1 An operational model of agency

We model a bounded agent as a tuple

$$\mathcal{A} = \langle \mathcal{O}, \mathcal{S}, \mathcal{G}, \mathcal{U}, \pi, \mathcal{V}, \mathcal{B} \rangle, \tag{1}$$

The tuple separates representations from control responsibilities. $\mathcal{O}$ specifies the admissible observation and evidence types; $\mathcal{S}$ is the maintained, persistent state space; $\mathcal{G}$ contains task specifications, including success and termination criteria; and $\mathcal{U}$ defines the schemas for actions or tool calls the architecture can issue. The policy π proposes an action from the current state and task, but it does not by itself authorize execution. $\mathcal{B}$ supplies externally enforceable bounds such as permissions, time, cost, source access, and maximum steps, while $\mathcal{V}$ evaluates the transition and determines whether to accept, repair, escalate, or terminate. For $g \in \mathcal{G}$, π may be an LLM, a symbolic controller, a learned policy, or a composition of them. This prevents the generative component from defining evidence, state, action, authority, and outcome at once. Observations, goals, proposals, policy decisions, and outcomes must remain distinct typed records so the loop can be inspected or resumed.

At step t, an observation updates state, the policy proposes an action, the bounds gate authorizes it, and the environment returns the next observation:

$$s_t = f(s_{t-1}, o_t), \tag{2}$$
$$\tilde{u}_t \sim \pi(\cdot \mid s_t, g, \mathcal{U}), \tag{3}$$
$$u_t = \Gamma_{\mathcal{B}}(\tilde{u}_t, s_t), \tag{4}$$
$$o_{t+1} \sim P_{\mathcal{E}}(\cdot \mid s_t, u_t), \tag{5}$$
$$d_t = \mathcal{V}(s_t, \tilde{u}_t, u_t, o_{t+1}), \tag{6}$$

The equations separate proposal, authority, and outcome. The update function f first commits the new observation to maintained state. The policy π then emits a candidate $\tilde{u}_t$; it does not execute it. The gate $\Gamma_{\mathcal{B}}$ evaluates that proposal against the current state and operational bounds, returning either an authorized $u_t \in \mathcal{U}$ or $\perp$. The latter is a typed result rather than an exception: it records denial, abortion, a controlled no-op, or escalation. Only the authorized action reaches the environment kernel $P_{\mathcal{E}}$, whose stochastic response captures changing APIs, tools, users, and physical or digital environments. Execution success is therefore not equivalent to task success. The validator $\mathcal{V}$ compares the prior state, proposed and authorized actions, and resulting observation to produce d_t, such as accept, repair, escalate, or terminate. That decision and its supporting evidence should be persisted with the transition. The formulation imposes two useful invariants: no proposed action bypasses the bounds gate, and no observed outcome bypasses validation. An LLM may implement part of π or $\mathcal{V}$, but it cannot supply these guarantees alone; agency emerges from the complete, inspectable loop.

3.2 Reactive, deliberative, and hybrid control

Reactive and deliberative control differ principally in how much structured state stands between observation and action. A reactive controller maps the current observation and a compact local state directly to an action. This minimizes latency and can be robust when preconditions are observable and local rules are sufficient, but it provides little machinery for aggregating evidence across time, representing delayed consequences, or comparing alternative futures. A deliberative controller instead persists a goal, an action model, and candidate future states before selecting what to do. STRIPS and PDDL make this representational requirement explicit by encoding actions as transition operators with typed preconditions and effects [41, 102]; BDI architectures separate beliefs, desires, and intentions, distinguishing informational state, objectives, and committed courses of action [133]. These are not merely historical labels. Reactive control deliberately narrows the representation and decision horizon, whereas deliberation expands both the state that must be maintained and the transitions that must be validated. The appropriate choice depends on the evidence and consequence structure of the task, not on whether an LLM is present.

Most practical LLM agents are hybrid because proposal and execution have different assurance requirements. A language model planner can interpret an underspecified goal, decompose it, and propose or revise a candidate plan; a reactive execution layer must translate each step into a typed command, check its schema, preconditions, permissions, idempotency, and budget, and commit the result to state. Toolformer demonstrates that a model can learn which API to call, when to call it, and which arguments to supply [144]; ToolLLM extends tool learning evidence to single- and multi-tool solution paths over a large API collection [132]. These results establish proposal competence, not execution authority or postcondition assurance. A plan expressed in language remains a hypothesis about the environment: API state, user permissions, external data, and earlier side effects may invalidate it before the next step. The executor should therefore reread current evidence before every action, bind and validate the concrete parameters, then persist the returned value, error, side effect, and provenance before replanning. Hybrid

control allows deliberation to remain flexible while making operational transitions deterministic where possible and auditable where uncertainty remains.

ReAct made the hybrid loop explicit by interleaving reasoning traces, environment actions, and observations [201]. Its central architectural contribution is that new environmental evidence can revise the next decision. A textual scratchpad, however, is not a progress record: it can rationalize an error, omit a side effect, repeat an equivalent action, or claim completion without representing the required postconditions. Interactive benchmarks expose this gap. AgentBench identifies long-horizon reasoning, decision-making, and instruction following as recurring obstacles [88], while the original WebArena study reported only 14.41% end-to-end success for its best GPT-4-based agent, compared with 78.24% for humans [219]. Each loop iteration should therefore update a typed progress ledger containing the intended subgoal, action identifier, checked preconditions, observation, state delta, postcondition verdict, retry count, and remaining budget. Stopping and escalation logic should read that ledger rather than trust the model's assertion that it is finished. A reason–act loop becomes a reliability mechanism only when environment evidence is converted into measurable progress, externally validated transitions, and bounded termination decisions.

3.3 Decision-making under partial observability

Partial observability is the normal operating condition of an agent, not an edge case. The system acts on evidence about the environment rather than on the environment's complete state: a user may omit constraints, a web interface may expose only the current view, a tool may return partial or stale data, and another agent may hold private information. Partially observable Markov decision processes (POMDPs) formalize this separation by maintaining a belief over possible states and selecting actions according to both expected task value and the value of additional information [68]. An LLM-centered system need not encode that belief as a calibrated probability distribution in every application, but it must preserve an equivalent representational distinction. Observed facts, inferred claims, competing hypotheses, confidence, source, and timestamp should remain separately addressable, and a new observation should revise rather than silently overwrite the prior evidence state. Querying, retrieving, diagnosing, or requesting clarification can then be chosen as information gathering actions. The reliability failure is not that the agent lacks complete state; it is that the architecture lets incomplete evidence masquerade as complete knowledge before action.

The systems consequence is an evidence ledger that keeps observations, inferences, assumptions, and conflicts separately addressable. A scratchpad may assist reasoning, but it cannot be authoritative state because it lacks stable identity, provenance, and update semantics. Information gathering is justified when it reduces decision-critical uncertainty enough to change whether the system proceeds, asks, retrieves, abstains, or escalates. The precise ledger schema belongs to the architecture section; the foundational point is that partial observability becomes governable only when uncertainty persists as state rather than disappearing into fluent prose.

3.4 Hierarchical control and temporal abstraction

Hierarchy is a compression of the decision horizon: a controller reasons over subgoals and reusable skills instead of reconsidering every primitive action at every step. Hierarchical task networks formalize this idea by recursively refining an abstract task until its leaves are executable operators [37]; the options framework represents a temporally extended policy through its initiation set, internal policy, and termination condition [158]. Both traditions show that a reusable unit must be represented as more than a natural language label. It requires conditions under which it may start, the state and evidence it consumes, the behavior it controls, and a criterion for recognizing its completion or failure. Modern agentic systems instantiate the same ab-

straction through planner–executor separation, manager–worker workflows, task graphs, and skill libraries. Voyager, for example, stores increasingly complex behaviors as executable code and improves them using environment feedback, execution errors, and self-verification [173]. A collection of agents does not automatically create hierarchy: hierarchy exists only when parent and child task states, authority, and result admission are explicitly related. The resulting task tree is a persistent control structure that connects high-level intent to bounded executable actions.

Hierarchy is valuable only when abstraction reduces decision load without erasing control evidence. A delegated result should return as a candidate state transition and pass an admission gate before it changes the parent plan. This boundary can contain local failure, but it also adds routing, synchronization, duplicated context, and recovery costs. MultiAgentBench confirms that topology and planning strategy materially affect milestone completion [221]. Hierarchy is therefore a contract between parent and child state (goal, evidence, authority, budget, expected artifact, and completion status), not conversational delegation. Section 4 develops the corresponding task and admission records.

3.5 Language models as policies over structured actions

A pretrained LLM produces a probability distribution over text; an executable system requires a controlled choice among typed actions. The tool interface is the boundary that performs this conversion. A model output is interpreted as a proposal naming an operation and supplying arguments, after which a runtime can parse, validate, authorize, execute, and record it. MRKL systems established the routing pattern by selecting external modules for a query [70]; Toolformer showed that a model can learn when to invoke APIs and which arguments to provide [144]; ReAct connected action selection to an iterative stream of environmental observations [201]. Together, these approaches expand language modeling into policy proposal, but they do not make generated actions correct or safe by construction. Typed function calling narrows the output space and makes malformed calls rejectable; a schema-valid call can still be unsupported by current evidence, inconsistent with state, unauthorized, redundant, or irreversible. The system should therefore preserve both the model's original proposal and its normalized action object so that interpretation errors remain inspectable. Autoregressive generation becomes part of an agent policy only after its output crosses this explicit type boundary and remains subject to runtime control.

The language policy should therefore sit behind a versioned action contract rather than a prose-only tool description. The model proposes an operation and arguments; the runtime validates their schema, evidence, authority, budget, and expected effects before execution and records a typed receipt afterward. BFCL shows that correct function selection and argument construction are distinct capabilities that require executable evaluation [130]. Yet even a schema-valid call may be unsafe or unsupported. Dependability is joint: model competence matters, but the interface and enforcing runtime determine which errors can reach the world.

3.6 Memory, external cognition, and persistent state

Agent memory is an architecture of typed state classes, not one store or the current prompt. Working state, episodes, semantic evidence, profiles, and procedural skills differ in authority, retention, retrieval, and validation [213]. MemGPT illustrates the systems value of moving information between limited context and external tiers [125]; AgeMem makes add, update, delete, retrieve, summarize, and filter explicit policy actions [206]. These mechanisms also expose the central risk: a generated note or retrieved episode can influence later decisions without becoming verified evidence. Memory writes and revisions therefore require provenance, scope, versioning, and an admission decision, while retrieval must preserve authorization and freshness.

The architecture section develops these lifecycle operations; here memory matters because it extends the agent's control loop across time and turns persistence itself into a bounded action surface.

3.7 Multi-agent foundations

A multi-agent system externalizes decision-making across entities with different observations, objectives, capabilities, or authority [183]. This distribution is useful when no single controller should see every datum or hold every permission, when subtasks require genuinely different tools or expertise, or when independent exploration and critique produce evidence that can be compared. Contemporary LLM systems encode that structure in different ways: AutoGen treats coordination as configurable conversations among agents [185]; CAMEL studies role-conditioned cooperation through agent dialogue [81]; and MetaGPT organizes software work through specialized roles and workflow artifacts [59]. These designs differ in topology and control, but each makes communication part of the architecture rather than merely additional prose. An agent should have an explicit identity, capability and permission profile, observation scope, memory view, active commitments, and allowed message types. A message should similarly identify sender, recipient, task and correlation identifiers, claim or requested action, evidence references, timestamp, and confidence or commitment status. Receiving a message does not make its content true; an admission rule determines whether it updates shared task state. In this sense, a multi-agent system is a distributed state machine whose transitions happen through typed messages. Splitting one model prompt into several personas without distinct evidence, authority, or state boundaries creates dialogue, not substantive distribution.

The distribution introduced above is useful only when it creates complementary information, capability, or authority. Cloning one model behind several role labels can preserve shared blind spots: agents may repeat the same unsupported premise, while agreement amplifies confidence without adding independent evidence. The empirical record is correspondingly mixed. Iterative multi-agent debate improved factuality and reasoning on the controlled tasks studied by Du et al. [34]; by contrast, Smit et al. found that current debate strategies did not reliably outperform self-consistency or ensembles of independent reasoning paths without careful tuning [154]. MultiAgentBench further shows that milestone completion changes with coordination topology and planning strategy, while exposing repetition and inefficient collaboration as observable failure modes [221]. The design question is therefore not how many agents participate, but what independence the architecture actually creates. A coordination contract should specify the communication topology, admissible message types, ownership of shared state, conflict resolution and aggregation rules, termination criteria, and escalation paths. Roles should be functionally diverse, using different evidence sources, tools, permissions, or verification duties, and their outputs should be merged by an evidence-aware rule that preserves provenance and dissent. Collective judgment must therefore rest on independently grounded contributions and explicit reconciliation, not conversational agreement.

3.8 A bounded interpretation of autonomy

Autonomy is not a property of model output; it is an allocation of decision rights across transitions. Human–automation research treats automation as stage-specific [127], and the NIST AI Risk Management Framework requires explicit oversight roles [162]. An LLM-centered system should therefore represent autonomy as an enforceable profile rather than a label. Capability and authority remain distinct: knowing how to initiate an action does not grant permission to execute it. Relevant dimensions include:

- **decision autonomy**: which branches or actions the model may choose;

- **execution autonomy**: whether proposed actions are executed automatically;
- **temporal autonomy**: how long the system may operate before review;
- **resource autonomy**: what compute, money, tools, and data it may consume;
- **social autonomy**: whether it may communicate or delegate to people and agents;
- **adaptation autonomy**: whether it may alter memory, prompts, policies, or code.

These dimensions form a profile, not a ladder. A research agent may search and synthesize independently while remaining unable to send messages, modify records, or spend money without approval. High automation can coexist with high human control [151]; what matters is the separation among capability, delegated authority, and observed behavior. Governance and evaluation should compare those quantities directly, because labels such as "copilot" and "fully autonomous" conceal the transitions that actually require control.

3.9 Conceptual analogies and their limits

Neuroscientific and cognitive theories are useful as sources of architectural hypotheses, not as evidence that an LLM-centered system implements the inspiring mechanism. Marr's computational, algorithmic, and implementation levels provide a useful test [100]: a metaphor needs a precise mapping, an architectural claim needs an executable mechanism and ablation, and a biological claim needs causal evidence. The Thousand Brains Theory may motivate specialized evidence views and reconciliation [57], but specialized modules are not thereby cortical columns. We retain only the operational lesson that multiple evidence-bearing representations can be reconciled before bounded action.

The foundations converge on one control structure: partial evidence becomes typed state; state supports a proposal; goals, permissions, and budgets constrain authorization; and observed effects update the next decision. Memory carries the loop across time, hierarchy across temporal scales, and multi-agent messaging across responsibility boundaries. The common requirement is a representation contract for observations, claims, plans, authority, effects, and failure. The next section asks where those records live, who owns them, and which component enforces their transitions.

4 Architectures for LLM-Centered Agents

An agent architecture is the operational realization of the representation contract developed in the previous section: it determines where goals, evidence, task state, plans, permissions, actions, outcomes, and evaluations live, and which transitions may change them. CoALA provides a useful conceptual decomposition into modular memory, a structured action space, and a generalized decision-making process [157]. For deployed systems, that decomposition must be extended with explicit ownership and enforcement. The central engineering question is therefore not whether an LLM is present, but which records it may read or propose, which component holds authoritative state, which rules authorize a transition, and which validator decides that an effect occurred. An LLM may interpret evidence, construct a plan, or propose a tool call without becoming the authority for provenance, permissions, or outcome truth. Production architectures consequently place probabilistic model proposals inside independently checkable control surfaces: schemas type inputs and outputs, state machines or graphs constrain transitions, policy gates enforce permissions and budgets, and validators determine whether execution may continue, retry, escalate, or commit an external effect. Two systems using the same model and prompts can therefore have materially different agentic behavior

because their state ownership, transition semantics, and authority boundaries differ. Figure 4 summarizes this separation between proposal and authorized commitment.

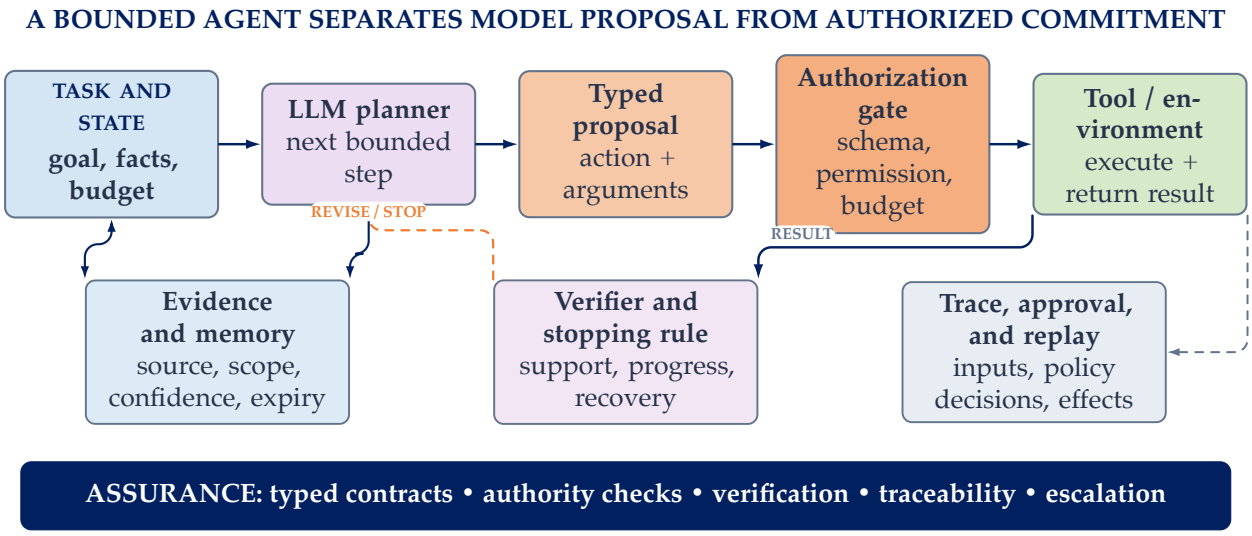


Figure 4. Reference architecture for bounded LLM-centered agents. The model plans and proposes a typed action, but an external gate authorizes the action against schemas, permissions, and budgets before any environment change. Evidence and memory inform the next decision; results are checked for support and progress; and approval plus trace paths preserve accountability.

4.1 Control topologies

Control topology is not merely the shape of an orchestration diagram; it determines which component selects the next transition, which state enables that transition, and how parallelism, recovery, and termination are represented. Workflow pattern research established that orchestration systems should be compared through observable control semantics, including sequence, choice, concurrency, synchronization, cancellation, and exception handling, rather than through vendor-specific syntax [168]. LLM-centered systems add three concerns to that classical lens: a proposed transition may be probabilistic, its supporting evidence may be incomplete or contested, and its execution may require separate authorization. We therefore compare each topology by five questions: where task state persists; who proposes the next step; how evidence, preconditions, permissions, and budgets are checked; how results and errors update subsequent decisions; and how the episode terminates or escalates. This lens separates architectural substance from labels: the same visible sequence can be implemented with very different state and authority boundaries, while a system with several named agents can still have one centralized controller.

Prompt loop. A prompt loop appends each observation to the transcript, asks the LLM for the next step, and repeats. ReAct demonstrates the value of this lightweight feedback structure [201], but one transcript then serves as evidence, memory, plan, and program counter. AgentBench and WebArena show that model capability alone does not remove long-horizon control failures [88, 219]. A production loop should externalize typed step, authorization, outcome, progress, and stopping records; otherwise replay reconstructs prose rather than state transitions. Prompt loops remain useful baselines, but incomplete production architectures.

Planner–executor. A planner–executor topology separates task decomposition from environment-specific action. Plan-and-Solve reduces missing-step errors in reasoning [176], while Plan-and-Act extends the split to long-horizon web tasks [36]. The architectural value lies in the interface:

the plan is versioned, prerequisites and permissions are checked at execution time, and typed outcomes can retain, repair, or invalidate later steps. This works when intent is stable but observations are volatile; it collapses into a prompt loop when the plan is only prose or the executor hides state and errors.

Graph or state machine orchestration. A graph or state machine represents process state and permitted transitions outside the model. StateFlow reports higher success and lower inference cost than ReAct on its InterCode SQL and ALFWorld evaluations [187]. Nodes need typed inputs, effects, and success predicates; edges need guards and failure destinations; the runtime, not the LLM, commits transitions. This makes branching, intervention, replay, and recovery inspectable, but creates configuration cost and false assurance when guards still depend on stale prose or unchecked model judgment.

Event-driven architecture. An event-driven topology distributes control through messages. The actor model motivates modular message-driven behavior [58], while Lamport's partial ordering explains why causal identity and versioning matter [74]. Events need typed identity, provenance, authorization context, and deduplication; delivery itself is not authority. Consumers must recheck state and permissions, tolerate duplicate delivery, and correlate results and retries. The topology supports asynchronous work and recovery, but introduces stale or reordered evidence, duplicate effects, races, and confused-deputy risk.

Hierarchical and multi-agent orchestration. A hierarchical or multi-agent topology delegates bounded work to managers, workers, or peers. MetaGPT demonstrates role and artifact coordination [59], and MultiAgentBench shows that topology and planning strategy affect milestone completion [221]. Delegation becomes architectural only through a task contract that limits evidence, tools, authority, budget, output, and success criteria. Workers return candidate results; a parent or arbiter admits them through provenance and conflict checks. This supports separable parallel work, but becomes costly when state is tightly coupled or the manager, shared context, or delegated authority becomes an unverified bottleneck.

Table 5. Common control topologies and their operational trade-offs.

Topology	Best suited to	Main advantage	Primary risk
Prompt loop	Low-risk prototypes and short tool tasks	Minimal implementation overhead	Implicit state, weak stopping, poor replay
Planner–executor	Multi-step tasks with well-defined tools	Separation of deliberation and actuation	Plans become stale; executor errors may be hidden
Graph/state machine	Auditable workflows with conditional branches	Explicit transitions, checkpoints, and recovery	Graph complexity and state version management
Event-driven	Long-running or asynchronous processes	Durability and scalable decoupling	Duplicate events, races, and partial failures
Hierarchical / multi-agent	Tasks benefiting from specialization or parallelism	Modular roles and permission isolation	Coordination overhead and correlated errors

4.2 State and memory architectures

In an agentic system, state and memory answer different operational questions. State is the authoritative, typed record required to determine and safely commit the next transition: the

current step, evidence used, pending approvals, observed tool effects, budgets, and version. Memory is retained information that may be retrieved to inform later reasoning. The distinction appears from complementary directions in prior systems: StateFlow makes execution progress explicit through states and transitions [187], CoALA separates working, episodic, semantic, and procedural stores within an agent architecture [157], and MemGPT manages movement between in-context and external memory [125]. Once an agent can act on the world, however, this distinction becomes an integrity boundary. Retrieved memory should enter the current computation as a provenance-bearing candidate rather than silently overwrite authoritative task state; conversely, facts needed for recovery or authorization cannot survive only in a transcript, summary, or embedding. State therefore needs a schema, owner, version, invariants, and controlled updates, whereas memory needs relevance, provenance, freshness, retention, and access policies. Blurring the two produces systems that remember more language while preserving less operational truth.

4.2.1 Functional memory layers

A practical architecture separates at least five layers, not as progressively larger versions of the same store, but as representations that answer different control questions. The sequence below moves from information that must remain authoritative within the active run to information whose value emerges across tasks:

1. **Execution state: What may occur next?** Authoritative progress, approvals, budgets, tool outcomes, and committed effects for continuation and recovery.
2. **Working memory: What is active now?** Task-local hypotheses, constraints, candidates, and intermediate artifacts that remain provisional.
3. **Episodic memory: What happened before?** Provenance-bearing trajectories and outcomes, as in Generative Agents [128].
4. **Semantic memory: What is believed across episodes?** Versioned facts, preferences, concepts, and organizational knowledge with validity and source.
5. **Procedural memory: How should the system act?** Reusable, revocable skills, plans, prompts, code, and lessons such as those extracted by ExpeL [214].

The layers therefore differ not only in update frequency and retention, but in authority. A retrieved episode or semantic record may become a candidate in working memory; it does not become an execution fact merely because it was retrieved. Conversely, a successful trajectory should not silently rewrite a shared skill. Information should cross these boundaries through explicit operations (retrieve, validate, admit, commit, consolidate, or discard) that preserve provenance and version history. This staged flow prevents a fluent summary or superficially similar memory from changing what the system is permitted to do.

4.2.2 Representations and stores

After deciding what role information plays, an architecture must choose how that information is represented. This is an operational decision: the representation determines which claims can be addressed exactly, which updates can be made atomically, whether provenance survives compaction, and whether recalled material can be distinguished from current authorization. No single form (raw transcript, summary, key–value record, embedding, relational table, knowledge graph, or executable artifact) supports all of these operations. Representative systems expose different parts of the trade-off. Generative Agents makes observation streams and synthesized reflections retrievable, supporting continuity in simulated social behavior but not

supplying an authoritative transaction record [128]. MemGPT manages movement between in-context and external memory, showing that location and access are themselves control decisions [125]. MemoryBank makes retention, updating, and forgetting explicit, emphasizing the lifecycle of long-term conversational records [217]. Taken together, these designs argue for composite representation: typed records protect operational truth, provenance-bearing artifacts preserve evidence, and associative indices support recall without granting retrieved content authority by default.

- **Transcripts** preserve chronology but mix observations, instructions, assertions, and tool returns; they are archives rather than canonical state.
- **Summaries** reduce context cost but can erase constraints and provenance, so they must link to source spans and transformation metadata.
- **Vector indexes** support semantic recall, but similarity does not establish truth, recency, permission, or applicability.
- **Structured records** support exact filters, invariants, atomic updates, and access control, making them the strongest fit for authoritative state.
- **Knowledge graphs** expose relations, time, and provenance, while leaving entity resolution and edge validity as explicit obligations.
- **Executable memory** stores code, plans, or skills, as in Voyager [173]; because retrieval can change behavior, it requires sandboxing, evaluation, versioning, and revocation.

The practical conclusion is a hybrid record, not a winning store. A task may keep authoritative status, budgets, and approvals in a typed transactional record; immutable source documents and tool receipts in an artifact store; embeddings or graph links as discovery indices; a derived summary for model context; and separately versioned procedures for reuse. Stable identifiers connect these views so that a retrieved passage can be traced to evidence and an action can be traced to the exact state and procedure that authorized it. The representation stack thus implements the paper's central boundary: associative memory proposes relevant material, while structured state determines what the system may commit.

4.2.3 Memory operations as policy decisions

A stored object becomes operational memory only through a lifecycle policy that decides when to create, link, retrieve, update, consolidate, or discard it, and which later decisions may consume it. Contemporary systems make different portions of that policy explicit. A-MEM generates attributes and links among records, so accumulated experience becomes a navigable relational structure [197]; Mem0 selectively extracts and maintains long-term information from interactions [21]; and ExpeL transforms completed trajectories into reusable experiential lessons [214]. These are not interchangeable variants of one retrieval algorithm. They implement different typed transformations (event to linked record, interaction to retained fact, or trajectory to procedure) and grant the resulting object different authority. Reported gains on each system's native tasks therefore answer different questions. A meaningful comparison should identify the source object, transformation, retained provenance, intended consumer, and decision enabled, rather than collapse all results into a single memory leaderboard.

Every memory operation should be a typed, versioned event. Writes preserve source, scope, confidence, validity, and retention; reads return provenance, freshness, and conflicts; updates supersede rather than erase. AgeMem learns when to store, retrieve, update, and discard [206], replacing fixed schedules with task-sensitive control. The same flexibility relocates risk into

the learned policy, so schemas, authorization, versioning, and invalidation must remain independently enforced.

Personalized summarization provides an earlier example: feedback becomes an explicit preference model that changes later selection and synthesis [50, 51]. In an acting system, that representation can alter tool choice, so an inferred preference may propose an update but cannot silently become durable authorization.

Memory benchmarks make the distinction empirical. MemBench tests factual and reflective recall [163], whereas Mem2ActBench tests whether remembered preferences and task state improve tool selection and argument grounding [147]. Current systems are weaker at active use than retrieval-only results suggest. Evaluation must therefore follow write, retrieval, admission, decision, and effect; question answering establishes recall, not safe improvement to action.

Table 6. Representative memory designs. Results across rows are not directly comparable because the systems target different tasks and memory horizons.

System	Memory object	Write or update policy	Evidence setting	Primary limitation
Generative Agents [128]	Natural language observations, reflections, and plans	Stores observations; periodically synthesizes reflections	Interactive simulated social environment and human judgments	Simulation behavior does not establish factual or transactional reliability
MemGPT [125]	In-context working memory plus external archival memory	Model-mediated movement between memory tiers	Long conversations and document analysis	Model-controlled paging can omit or retrieve the wrong state
MemoryBank [217]	Long-term conversational events and summaries	Updating and forgetting inspired by memory decay	Long-term dialogue and personality consistency	Conversation metrics do not test downstream tool action
ExpeL [214]	Reusable natural language lessons from trajectories	Reflects over successes and failures across episodes	Reasoning and interactive agent tasks	Extracted lessons can preserve evaluator errors
A-MEM [197]	Linked notes with generated contextual attributes	Dynamically constructs and updates relationships	Knowledge and long-horizon agent tasks	Generated links and attributes require provenance validation
AgeMem [206]	Short- and long-term memory under a learned policy	Reinforcement-learned store, retrieve, update, and discard actions	Long-horizon agent memory benchmarks	Policy optimization broadens the attack and regression surface

4.2.4 Memory failure modes

Memory failures are not a miscellaneous collection of model mistakes; each violates a field or transition in the representation contract:

- **Staleness (valid-time failure):** a once-correct item remains eligible after the world changes. Store valid-from and valid-to times, freshness requirements, and supersession links, and revalidate high-impact facts at use time.
- **Contamination (source type failure):** model-generated speculation is persisted as obser-

vation or evidence. Preserve typed source roles and require explicit promotion before an inference can enter semantic memory.

- **Scope leakage (access control failure):** information from one user, tenant, or task is retrieved in another. Experiments on Memory EXTRaction Attacks demonstrate that stored user–agent interactions create a practical privacy surface [172]; every record and query therefore needs subject, tenant, purpose, and permission labels.
- **Memory poisoning (integrity failure):** untrusted content crosses a write boundary and later steers decisions. A systematic study identifies multiple write channels and structural vulnerabilities and finds that more aggressive writing and retrieval can increase exploitability [24]. Separate trust domains, validate writes, and screen retrieved content before admission.
- **Over-retrieval (selection failure):** irrelevant or weakly matched memories crowd out current evidence. Bound retrieval budgets, filter by time and scope, diversify candidates, and require an admission decision rather than inserting all matches into context.
- **Under-retention (retention failure):** commitments, constraints, approvals, or unresolved obligations are discarded. Promote these objects to typed execution or semantic records with explicit retention rules instead of relying on lossy summaries.
- **Self-reinforcement (feedback failure):** repeated retrieval of an erroneous note increases its apparent credibility. Empirical work on experience-following reports both error propagation and misleading experience replay [193]; retrieval frequency must not substitute for independent evidence, and negative outcomes should reduce or revoke an item's eligibility.

Mitigation belongs at the failed boundary: provenance and type checks at write time; version, time, scope, and conflict filters at retrieval; authorization at admission; and reconciliation or revocation after truth changes. Cleanup alone is too late once a stale or poisoned item has influenced action. Every retained object needs an origin, every use an admissibility decision, and every invalidation a path to dependent state.

4.3 Reasoning and planning

Reasoning and planning are distinct transformations over system representations. Reasoning organizes evidence, constraints, hypotheses, and tests into intermediate claims; planning maps a goal and the accepted current state into an executable, versioned action structure. Classical planning makes the required semantics explicit through action preconditions and effects, as in STRIPS [41], and through policies conditioned on uncertain observations, as in partially observable planning [68]. LLM scaffolds can propose these structures in natural language, but fluency does not remove the distinction: a rationale may omit evidence provenance, and a list of steps may be plan-shaped text without valid dependencies, permissions, or effects. A reliable architecture therefore separates a proposed rationale, a validated plan, the committed next action, and the observed result. Each transition should preserve typed inputs, ownership, success and failure conditions, and version history so that execution can reject, repair, or replan without treating the model's explanation as authority.

4.3.1 Decomposition and explicit plans

Decomposition becomes operational when subgoals are plan nodes rather than prose bullets. Least-to-most and Plan-and-Solve show benefits from staged reasoning [176, 218]; hierarchical task networks provide the stronger systems analogy of refinement under explicit

constraints [37]. A plan therefore needs versioned goals, dependencies, evidence, authority, expected effects, completion tests, and failure routes that a runtime can validate and schedule.

A plan is a versioned hypothesis, not a fixed commitment. Plan-and-Act separates construction from adaptive execution [36]; executor outcomes should therefore complete, block, fail, or invalidate nodes against observed effects. Contradictory evidence or unavailable authority creates a linked successor plan, while completed and irreversible effects remain part of history. Repair preserves valid work instead of restarting from a fresh prompt.

4.3.2 Interleaved reasoning and acting

Interleaving matters because pre-execution plans rest on untested predictions. ReAct shows the value of alternating decisions with observations [201], but a free-form transcript blurs evidence, inference, authority, and effect. A production loop should persist the unresolved subgoal, evidence, assumptions, proposal, authorization, receipt, and state difference. The rationale may help the model; typed runtime fields make progress inspectable and resumable.

A useful loop is:

1. select the next eligible unresolved subgoal from the current plan and state;
2. assemble a scoped evidence bundle and record assumptions that remain unverified;
3. propose an allowed action with typed arguments, expected effect, and success criterion;
4. validate permissions, preconditions, schema, and budget, then execute with an idempotency or transaction identifier where possible;
5. classify the observation, preserve the raw receipt, and derive a typed result and state difference;
6. commit accepted updates, evaluate progress and termination conditions, and continue, repair, retry, escalate, or stop.

Progress closes the loop by comparing accepted before and after state with completion predicates, not by counting tokens or steps. Fewer unresolved requirements, satisfied tests, accepted evidence, and confirmed effects establish movement. No-progress detection and resource limits bound repeated exploration, while termination records completion, partial completion, blockage, failure, or cancellation together with remaining obligations.

4.3.3 Search and self-correction

Search and self-correction improve a trajectory by creating and evaluating alternative representations, but the alternatives differ across methods. Tree-of-Thoughts materializes multiple intermediate states and uses an evaluator to select or prune branches [200]; Reflexion converts feedback from completed attempts into verbal episodic records that condition later trajectories [150]; and Self-Refine repeatedly transforms the current artifact through model-generated critique and revision [97]. These are therefore not interchangeable instances of a single *reflection* mechanism: one searches a state tree, one writes experience for a later episode, and one iterates over the same output. Their reported gains establish that additional structure and feedback can outperform a single greedy pass on their evaluated tasks, but not that self-generated evaluation is independent evidence. When the generator, critic, and selector share a blind spot, the loop may strengthen a coherent error rather than correct it.

A search controller should preserve candidate lineage, evidence, evaluator, result, rejection reason, and cost. Branching is justified by uncertainty and expected decision value; pruning

should rely on executable tests, constraints, independent evidence, or human judgment. Termination depends on accepted solution criteria and the marginal value of another branch. Search earns its cost only when alternatives create better evidence for a bounded decision.

4.3.4 Programmatic and symbolic planning

Some reasoning and planning problems are better represented as programs, constraints, or symbolic operators than as free-form language. Program-Aided Language Models and Program-of-Thought prompting externalize arithmetic or symbolic computation into executable code [17, 44]; STRIPS and PDDL make planning operators, preconditions, effects, objects, and goals explicit [41, 102]. These approaches strengthen different boundaries: code supplies deterministic execution for a specified computation, while a planning model supplies a search space with machine-checkable transition semantics. A hybrid agent can use an LLM to propose the formal representation and a parser, type checker, solver, or planner to reject malformed or unsatisfiable candidates. The translation itself remains a critical representation step, however: every variable, constraint, operator, and goal condition should be traceable to user intent, accepted evidence, or policy rather than introduced as an unsupported model assumption.

This pattern is strongest when generation is difficult but validity is cheap to check. A model may propose SQL, code, a schedule, or a symbolic plan; the environment parses, constrains, tests, or simulates it and returns a counterexample. Passing a validator does not prove that the specification captured user intent, so the artifact, specification, validator, result, and repair must remain linked. Executable checks strengthen evidence without eliminating provenance or high-impact review.

4.4 Action selection and execution

Action selection is the boundary where a model proposal may become an external effect. Toolformer establishes proposal competence [144]; τ-bench shows why policy compliance and final task state matter beyond call selection [202]. The runtime must keep proposal, authorization, execution receipt, and independently observed effect distinct. A valid call is not necessarily permitted, and a success response is not proof of the intended state change. Schema-first interfaces, staged commitment, and explicit execution semantics constrain this transition.

4.4.1 Schema-first interfaces

A schema-first interface is an executable boundary, not a formatting hint. BFCL separates call conformance from long-horizon reliability and finds stronger performance on isolated calls than on memory and dynamic decision making [130]. A tool contract should version the capability, inputs, outputs, scopes, side effects, preconditions, cost, idempotency, retry, and compensation. Descriptions help the model propose a call; the runtime binds and enforces the machine-readable contract.

Validation protects both directions. Before execution, the runtime canonicalizes arguments and checks resource identity, permission, state, preconditions, and budget. Afterward, it preserves the receipt, validates provenance and structure, and checks the claimed effect. InjecAgent's 1,054 cases show why returned content must remain untrusted evidence: its ReAct-prompted GPT-4 configuration followed attacks in 24% of cases [209]. Schemas reject malformed structure; they do not establish permission, truth, or goal satisfaction.

4.4.2 Read, propose, commit

Read–propose–commit is a staged state transition protocol: it separates the evidence used to understand the environment, the candidate operation constructed from that evidence, and the

authority to create an external effect. It is not equivalent to placing a human in every loop. Parasuraman, Sheridan, and Wickens distinguish automation of information acquisition, analysis, decision, and action, and argue that the level of automation should be chosen for each function [127]; Shneiderman likewise argues that high computer automation and high human control can coexist [151]. For an agentic runtime, the architectural consequence is that automation and oversight can differ by stage, but every handoff must be represented explicitly:

1. **read**: invoke only the observation capabilities authorized for the task and return a provenance-bearing record containing the source, query or resource, timestamp, state version, trust label, and raw receipt. A read does not change the target environment, although the runtime may append the observation to its audit and evidence stores;

2. **propose**: materialize an immutable candidate containing the tool and schema version, canonical arguments, exact target resources, evidence references, expected state difference, uncertainty, cost, side effect and reversibility class, required permissions, and completion test. Dry runs, simulation, static checks, or diff generation attach validation evidence to this proposal rather than silently modifying it;

3. **commit**: have a policy gate or authorized person evaluate that exact proposal and persist the decision with principal, policy version, granted scopes, constraints, expiry, and proposal identifier. The executor rejects a stale or altered proposal, applies an idempotency or transaction identifier, and returns both an execution receipt and a separately validated effect record.

Approval is a versioned capability grant, not conversational assent. A change to arguments, target state, schema, evidence, or predicted effect invalidates the grant. Review should expose the concrete diff, target, uncertainty, and reversibility; graduated authority can automate inspection while reserving consequential commits. Reads still carry privacy and cost, and approval does not prove correctness, so least privilege and post-effect validation remain necessary.

4.4.3 Execution semantics

Authorization does not remove execution ambiguity: after a timeout or broken connection, the effect may have failed, succeeded, or partially applied. The runtime must distinguish the logical operation, each attempt, and observed world state. Its lifecycle separates authorization, dispatch, acknowledgement, and effect verification, together with denial, ambiguity, partial application, and compensation. A plan node is satisfied by verified state change, not a response string.

Recovery follows the result class. Invalid or denied proposals wait for changed input or authority; transient failures permit bounded backoff; unknown effects require status reconciliation before a side-effecting retry. Attempts remain linked to one logical operation. For non-atomic workflows, sagas provide explicit compensation [47]; compensation is itself an authorized, fallible action rather than erasure of history.

Tool returns should be normalized into typed success, partial, denied, unavailable, ambiguous, or terminal outcomes while preserving the receipt. AppWorld's state-based tests across 750 tasks and 457 APIs show why effect evidence matters; its strongest reported GPT-4o configuration reached 48.8% on normal and 30.2% on challenge tasks [166]. Invocation and fluent explanation are weak proxies for success, so continuation or recovery must follow the validated state difference.

4.4.4 Tool selection at scale

At catalog scale, tool selection is constrained retrieval over a changing capability space rather than classification over every description in the prompt. ToolLLM makes the retrieval problem explicit: ToolBench collected 16,464 real-world APIs across 49 categories, and ToolLLaMA uses a neural API retriever to recommend a tractable candidate set [132]. BFCL exposes a complementary limit: models that perform strongly on single-turn calls still face open challenges in stateful, multi-step decision making, and the benchmark explicitly evaluates abstention [130]. Together, these results separate three questions that are often collapsed: whether the catalog contains a useful capability, whether retrieval surfaces it, and whether the current task state and authority permit its use. A semantically relevant tool may still be unavailable, obsolete, too costly, insufficiently trusted, or outside the actor's scope.

The capability registry should be authoritative and versioned rather than appended to a prompt. Entries expose schema, authority, side effects, health, cost, version, and validation status. Deterministic filters first remove unauthorized or incompatible tools; retrieval ranks the admissible remainder; the model selects or abstains within that bounded set. Permissions, health, and schemas are revalidated at dispatch.

Hierarchical routing reduces context cost but can hide the only valid tool, so candidate recall must be measured separately from call accuracy. Fallbacks require compatible semantics, permissions, side effects, and outputs. Evaluation should distinguish catalog coverage, retrieval, selection, authorization, execution, effect correctness, collateral change, latency, and cost.

4.5 Learning and adaptation

Learning and adaptation is the boundary at which runtime evidence becomes a persistent change to future behavior. A completed trajectory contains observations, decisions, actions, receipts, effects, feedback, and cost, but a successful outcome does not by itself identify which element caused the success or whether the behavior should generalize. The relevant literature changes different substrates: ExpeL extracts reusable natural language lessons from experience [214]; Voyager adds executable skills to a growing library [173]; and SOAR optimizes an agentic tool use policy from observed behavior [84]. The self-improvement taxonomy further distinguishes updates to foundation model parameters from updates to the surrounding scaffold, including prompts, memory, tools, and control logic [136]. These mechanisms are not interchangeable. A textual lesson, executable procedure, routing statistic, memory item, and weight update have different scope, authority, failure modes, evaluation costs, and rollback semantics. The architectural question is therefore not simply whether an agent learns, but which representation changes, from what accepted evidence, for whose future tasks, and under whose authority.

A production learning pipeline links immutable trajectories to candidate lessons, skills, prompts, memory policies, routers, or model updates. Each candidate needs scope, source evidence, expected benefit, risks, evaluation, approval, and rollback. Temporary within-episode adaptation may remain working state; shared changes require comparative testing against a frozen baseline and monitored promotion. This staging limits feedback contamination, reward hacking, and durable regressions.

4.5.1 Learning from trajectories

A trajectory becomes supervision only when its state transitions, evidence, proposals, authority, receipts, effects, costs, and evaluator remain attributable. Behavioral cloning inherits demonstration bias; preference optimization depends on shown comparisons and judges; reinforcement learning depends on delayed and potentially misspecified reward. Successful, failed,

denied, corrected, and compensated episodes therefore provide different evidence and should not be collapsed into one undifferentiated transcript.

Recent methods expand the learned object beyond the final action. SOAR explicitly treats environment observations as supervision rather than assigning learning signal only from outcome rewards; across three domains and 14 benchmarks, it reports gains of up to 7.0% on general reasoning and 16.9% on deep research tasks, together with fewer erroneous or inefficient tool uses [84]. SUPO jointly optimizes tool behavior and the summaries used to compress long interaction histories, making context management part of the policy rather than a fixed preprocessing rule [93]. AgeMem exposes store, retrieve, update, summarize, and discard operations as actions and trains unified short- and long-term memory management across five long-horizon benchmarks [206]. These methods target different representations (action consequences, compressed context, and memory operations), so their gains should not be summarized as generic evidence that additional trajectory learning is uniformly better. They instead show that the placement and granularity of credit determine which component learns from the same episode.

A trajectory-to-training pipeline must preserve provenance, evaluator reliability, environment and tool versions, sampling, objective, data splits, and the produced artifact. Held-out evaluation should compare with a frozen baseline on outcome quality, violations, collateral effects, calibration, efficiency, and regressions. Only an artifact that passes explicit promotion criteria should alter deployed policy. Learning is a controlled transformation of execution evidence, not replay of whatever received a favorable score.

4.5.2 Online adaptation and self-improvement

Online adaptation shortens the interval between feedback and behavioral change, but the literature uses the label for materially different persistence mechanisms. Reflexion converts task feedback into verbal records retained in an episodic memory buffer for later trials [150]; ExpeL aggregates experiences from training tasks into natural language insights and retrieves both insights and prior episodes at inference [214]; Voyager refines executable programs from environment feedback and commits successful programs to a reusable skill library [173]. In its Minecraft evaluation, Voyager reports 3.3 times more unique items, 2.3 times longer travel, and technology tree milestones reached up to 15.3 times faster than prior systems, while also reusing its skill library in a new world. These results support explicit scaffold adaptation in that environment, not a general claim that self-written memories or code are safe to share or deploy. The broader self-improvement taxonomy is therefore important: foundation model updates, prompt changes, memory writes, tool or skill updates, and control logic revisions modify different representations and require different evidence [136].

Adaptation should be classified by blast radius: episode-local state, subject- or task-scoped memory, shared scaffolds, and model parameters. Promotion thresholds grow with persistence, privilege, and audience. A temporary hint may need local checks; a shared executable skill needs provenance, sandbox and held-out tests, approval, staged rollout, monitoring, and rollback.

Generation, evaluation, and promotion authority must remain separate. Feedback can be noisy, manipulated, or privacy-sensitive, and persistent stores enlarge integrity and disclosure risks [24, 172]. Candidate changes should be quarantined and tested against independent effects, invariants, held-out tasks, or accountable review. Bounded self-improvement requires a precise answer to what changed, why, where it applies, and how it can be revoked.

4.6 Hierarchical agency

Hierarchical agency is a representation of nested responsibility: it determines which controller owns the root objective and authoritative state, which subgoals and capabilities may be delegated, and which evidence permits a child result to change the parent plan. Classical abstractions make these obligations concrete. An option is a temporally extended action defined by an initiation set, an internal policy, and a termination condition [158]; hierarchical task network planning refines abstract tasks into constrained networks of subtasks [37]. Plan-and-Act instantiates the same separation in an LLM agent through a structured high-level planner and an environment-specific executor, reporting a 54% success rate on WebArena-Lite [36]. These results support hierarchy as a way to divide long-horizon control, but they do not justify multiplying conversational roles. Whether implemented with one model at several control levels or with several agents, the hierarchy resides in state ownership, admissible transitions, and completion authority.

The task tree is therefore a persistent control structure rather than a transcript outline. A parent retains the root task identifier, objective, global plan version, authoritative state, policy and budget envelope, and final completion authority. A child receives a versioned snapshot for a bounded subtask and owns only the local state needed to execute it. Manager–worker delegation, planner–executor separation, reusable skill libraries, and task graphs are useful only when their boundaries align with different context, evidence, tools, permissions, or verification duties. Privilege separation follows naturally: research and planning children may retrieve evidence and propose actions, while a narrowly scoped executor alone holds commit capabilities. A delegation should expire or be revoked when its parent plan, input evidence, permission, or target state version changes.

A delegation contract should specify:

- **identity and lineage:** stable task, parent, plan version, dependency, and assignee identifiers;
- **objective and entry conditions:** the bounded subgoal, constraints, preconditions, and assumptions;
- **input state and evidence:** referenced artifacts, provenance, freshness, and the state version from which the child starts;
- **capability envelope:** allowed tools, data, permissions, action classes, time, tokens, and monetary budget;
- **return schema:** required artifacts, claims with evidence references, expected effects, and proposed state delta;
- **exit conditions:** machine-checkable success tests, retry limits, failure classes, and escalation route; and
- **delegation policy:** whether further delegation is allowed, its depth and authority limits, and expiry or cancellation conditions.

Return is a second typed boundary. A child should report a status, produced artifacts and claims, supporting evidence, action receipts and verified effects, its proposed state delta, unresolved assumptions, consumed budget, and a recommended parent transition. The parent then checks the contract, evidence, permissions, completion predicate, and referenced state versions before admitting the result. It may accept, reject, request repair, escalate, or invalidate dependent nodes; the child should not overwrite authoritative root state directly. This admission

step prevents a fluent completion message from being mistaken for completion and localizes rework when a branch fails.

The architectural trade-off is between global consistency and local autonomy. Centralized parents simplify prioritization, conflict resolution, and budget control but can become decision and context bottlenecks. More autonomous children permit specialization and parallel progress, but increase stale state, duplicated work, reconciliation, and authority risks. Hierarchy is therefore justified when subproblems can be separated along state, data, tool, or permission boundaries and their outputs can be tested before merge. It should be evaluated through milestone completion, accepted handoffs, rework and conflict rates, escalation frequency, latency and cost, and privilege violations, not by the number of agents, roles, or exchanged messages.

4.7 Multi-agent coordination

Multi-agent coordination distributes work, evidence, and control authority. AutoGen emphasizes programmable conversations [185], CAMEL role-conditioned interaction [81], and MetaGPT specialized roles and artifacts [59]. The architectural question is what information and authority are genuinely separated and how contributions are reconciled. Membership, state ownership, messages, permissions, conflict resolution, termination, and accountability, not the number of model calls, define the system.

4.7.1 Coordination topologies

Topology determines which participants can observe state, assign work, communicate evidence, resolve conflicts, and authorize a shared transition. Four patterns recur:

- **centralized**: a manager owns the root plan and authoritative ledger, issues versioned task contracts, and admits validated results;
- **peer-to-peer**: participants own local state and exchange typed proposals, evidence, acknowledgments, and conflict notices without a permanent central integrator;
- **blackboard**: participants coordinate through a shared structured workspace whose records expose partial solutions and unresolved needs [119];
- **market or contract**: a task is announced with constraints and an evaluation rule, candidates bid their capability, cost, or expected utility, and assignment becomes an explicit contract [155].

These patterns change the failure surface rather than defining a universal ranking. Centralization simplifies global prioritization, least privilege enforcement, and final accountability, but concentrates context, queueing, and recovery in one manager. Peer-to-peer structures remove that chokepoint and can support local adaptation, yet require explicit discovery, synchronization, duplicate work detection, and distributed conflict resolution. A blackboard decouples producers from consumers and preserves visible intermediate state, but only if writes are atomic, versioned, attributable, access-controlled, and invalidated when upstream evidence changes. Contract-based allocation makes selection criteria observable and can allocate heterogeneous capabilities dynamically, while adding bidding latency and the risk that self-reported capability or cost is poorly calibrated.

Contemporary evidence reinforces this conditional view. MultiAgentBench compares star, chain, tree, and graph protocols across interactive collaboration and competition scenarios; it reports that the graph protocol performed best in its research scenario and that cognitive planning increased milestone achievement by three percentage points [221]. This is evidence that

topology and planning policy affect progress, not that graph communication is globally superior. A sound design should choose the narrowest topology that matches the dependency structure: centralized control when state is tightly coupled or authority must remain singular; peer or graph communication when dependencies are local and dynamic; a blackboard when several specialists must contribute asynchronously to a common artifact; and contracting when task allocation itself is uncertain. Hybrids are often appropriate: for example, a central authority may control commits while workers exchange evidence through a shared blackboard and receive work through explicit contracts.

The topology should be persisted as configuration rather than implied by prompts. It defines membership, capability, permission, state ownership, communication, allocation, merge, escalation, budget, termination, and reconfiguration authority. Runtime measures of stale reads, duplicate work, conflicts, latency, privilege violations, and accepted progress distinguish useful distribution from multiplying the same reasoning path.

4.7.2 Communication and aggregation

Communication is where local observations and proposals become candidates for shared state. FIPA-style performatives distinguish requests, reports, proposals, agreements, and refusals from their payloads [42]; LLM systems need the same separation even when content remains natural language. A typed message envelope binds sender, recipient, task and state versions, evidence, requested transition, authority, uncertainty, causality, expiry, and acknowledgment so operational meaning is not inferred from style.

Delivery is not acceptance. The recipient should validate the sender's identity and delegated authority, parse the declared schema, resolve evidence references, compare state and plan versions, and check freshness, policy, and budget before the message changes a task or triggers an action. It should persist whether the message was received, rejected, admitted, superseded, or awaiting repair, together with the validator and reason. This protocol state supports replay, deduplication, timeout handling, cancellation, and accountability; it also prevents quoted instructions, stale plans, and unsupported claims inside one agent's output from silently controlling another. Natural language explanation can remain attached for human and model interpretation, but the executable consequence must be selected from an explicit transition set.

Aggregation is a separate admission problem: several contributions must be transformed into one decision without erasing their provenance or dissent. Objective validation should dominate when answers or effects can be tested. Evidence-aware selection can score source quality, coverage, recency, and contradiction; majority vote is defensible only when candidate errors are sufficiently independent; confidence weighting requires calibration on the relevant task distribution; and debate followed by a judge adds value only if interaction exposes information that independent sampling would miss. Du et al. report factuality and reasoning gains from iterative multi-agent debate in their studied settings [34]. Smit et al., however, find that debate protocols do not reliably outperform self-consistency or ensembles of independent reasoning paths without careful tuning, while showing that agreement intensity can materially change results [154]. Explanation-mediated weak-to-strong supervision provides another structured exchange of claims, critiques, and supervisory signals [208]. Together, these results argue for treating communication and aggregation as evaluated algorithms, not assuming that discussion produces verification.

The admitted decision should preserve candidates, evidence, eligibility, verifier results, aggregation rule, conflicts, selected result or abstention, and accountable authority. Agreement is weak evidence when agents share models, prompts, corpora, or context. Meaningful diversity must come from different evidence, tools, permissions, models, or verification functions; correlated failure should remain explicit.

Evaluation should compare the protocol with one agent, independent sampling, and deterministic verification. Message validity, duplication, disagreement, judge calibration, communication cost, and accepted progress expose coordination value. MultiAgentBench confirms that topology and planning alter milestone progress [221]; additional agents are justified only when evidence, specialization, or verification pays for reconciliation.

4.7.3 Multi-agent failure modes

Multi-agent failures are usually failures of represented coordination before they are failures of model count. Cemri et al. construct the Multi-Agent System Failure Taxonomy from 150 expert-annotated execution traces and scale the accompanying dataset to more than 1,600 traces across seven frameworks; its 14 failure modes fall into system design, inter-agent-misalignment, and task verification categories [12]. This evidence is important because it localizes failure in the protocol that connects agents, rather than treating an unsuccessful final answer as an undifferentiated model error. The three categories correspond to missing or violated representation contracts:

- **specification and system design:** task or role constraints are not encoded and checked; steps repeat because completion is not persisted; relevant history disappears; or no controller owns a termination condition;
- **inter-agent misalignment:** agents fail to request clarification, drift from the assigned task, withhold or ignore relevant evidence, reset the conversation, or describe reasoning that does not match the action taken; and
- **verification and termination:** the system stops before postconditions hold, performs no or incomplete verification, or admits an incorrect check as proof of completion.

The categories interact: an underspecified contract creates divergent interpretations, untyped messages hide them, shared state admits incompatible claims, and weak completion tests convert agreement into success. Duplicate work, stale state, and premature consensus are propagation paths. MultiAgentBench exposes repetition and inefficient collaboration [221]; containment therefore requires lineage from the first violated contract to dependent claims, plans, actions, and effects.

A typed incident should identify the failed contract, task and plan versions, evidence, affected artifacts and effects, retryability, containment owner, and disposition. Detection moves work through blocked, quarantined, repairing, compensating, escalated, or failed states and invalidates dependent state. Specification, alignment, and verification failures require different controls; none is repaired by another unsupported completion claim.

Recovery is part of coordination. Retries preserve operation and failure identity and avoid duplicate effects; partial success may require compensation or a successor plan. Failed workers return usable artifacts and unresolved obligations so only unfinished work is reassigned. Failed managers require externally persisted root state and takeover rules. Human escalation should carry the incident and a precise decision request, not the entire transcript.

Multi-agent value must be measured against coordination burden. Compare failure propagation, duplicate execution, stale handoffs, conflicts, recovery, privilege violations, messages, latency, and cost with a single agent and independent samples. If distribution does not produce measurable gains from parallelism, complementary evidence, specialization, or independent verification, one agent with explicit modules is usually more reliable.

4.8 Interoperability: tools, context, and agent-to-agent communication

Interoperability externalizes the representation contract across process, vendor, and organizational trust boundaries. It determines how a remote capability is discovered, how inputs and outputs are serialized, how long-running work is tracked, and how one system relates a returned artifact to its own task state. Syntactic compatibility is necessary but insufficient: a valid protocol message does not establish that the sender is trustworthy, the capability is appropriate, the request is authorized, the response is true, or an intended external effect occurred. Two protocol families address complementary parts of this problem.

Model Context Protocol (MCP). MCP standardizes how an AI host connects through clients to servers that expose tools, resources, and prompts. The specification defines a stateful, capability-negotiated protocol over JSON-RPC and standard transports [110]. Its durable architectural contribution is a common description and invocation surface: a tool can declare an input schema and optional output schema; a resource has a URI and metadata; requests, responses, notifications, progress, errors, and cancellation can be correlated. The declaration is still a capability claim made by a server. A host should persist the server identity and protocol version, negotiated capability snapshot, tool or resource identifier and schema version, validated arguments, authorization context, request and result identifiers, returned content and structured output, errors, timing, and cost. Returned text or data becomes an observation with provenance, not an instruction or verified effect.

MCP authorization mechanisms can authenticate parties, discover authorization servers, bind tokens to intended resources, and constrain scopes, but authentication and token validity do not answer whether a particular agent should perform a particular action in the current task. The application must still map the authenticated principal and granted scopes to a local capability policy, obtain any required user approval, recheck current state and budget, and validate the observed effect after invocation. This preserves the read–propose–authorize–execute–verify sequence even when the connector is standardized.

Agent2Agent (A2A). A2A standardizes interaction between independent agents. An Agent Card advertises identity, endpoint, skills, interfaces, and security requirements; exchanges use messages for immediate communication and tasks for stateful work that can produce status updates and artifacts [1]. This makes remote delegation more explicit than forwarding a chat prompt, but an advertised skill is not evidence that the agent is competent, available, authorized, or safe for the requested data. The delegating system should therefore translate its local subtask contract into an A2A task linked to the parent task, evidence snapshot, permission and data use envelope, expected artifact schema, budget, deadline, and completion test. It should persist the remote Agent Card and version used, endpoint and authenticated principal, task and context identifiers, messages, task state transitions, artifacts, cancellation or input requests, and terminal status.

The parent retains admission and completion authority. A remote *completed* state means that the remote protocol lifecycle ended successfully; it does not prove that local postconditions hold. Likewise, an authentication or authorization-required state does not itself grant permission for an operation. Returned artifacts must pass provenance, schema, policy, freshness, and task-specific verification before they update the parent plan or trigger a local action. This distinction permits agents operated by different organizations to collaborate without silently merging their trust domains.

Table 7. Complementary representation boundaries of MCP and A2A. Both require application-owned authorization, verification, and trace continuity.

	MCP	**A2A**
Primary boundary	Host application to tools and context providers	Delegating agent to an independent remote agent
Advertised surface	Tools, resources, prompts, schemas, capabilities	Agent Card, skills, interfaces, security schemes
Persistent unit	Session plus correlated request, result, progress, or error	Message or stateful task with status and context
Returned record	Content, structured output, resource link, or error	Message, task status, artifact, or terminal outcome
Typical use	Connect an agent to files, databases, APIs, or services	Delegate bounded work to a specialized agent
Not guaranteed	Local authorization, output truth, or verified effect	Competence, shared policy, or local task completion

The protocols can be composed without conflating them. A parent may create an A2A task for a remote specialist, while that specialist uses MCP clients to obtain evidence or invoke tools. The end-to-end trace should preserve one lineage from the root request and evidence snapshot, through the delegation contract and remote task, to every MCP invocation and receipt, produced artifact, independently observed effect, and parent admission decision. Correlation identifiers alone are insufficient if each hop discards the state version, authority, or evidence that justified it; the application needs a cross-protocol trace record that binds those fields explicitly.

Protocol adoption also expands the supply chain. A registry, server, adapter, or remote agent may be compromised, stale, misconfigured, or simply unreliable. Clients should authenticate endpoints, negotiate and pin compatible protocol and schema versions, record the capability snapshot used for each decision, minimize scopes and disclosed data, enforce time and cost bounds, validate structured inputs and outputs, make side-effecting retries idempotent, support cancellation and revocation, and quarantine untrusted returned content. Capability discovery is not authorization, authentication is not trust, delivery is not acceptance, and protocol completion is not effect verification.

Because these specifications define interfaces rather than empirical reliability guarantees, evaluation should test both conformance and application behavior. A compatibility matrix should exercise supported transports, protocol versions, capabilities, schemas, authentication modes, streaming, cancellation, and failure responses across independent implementations. End-to-end measures should include discovery-to-invocation success, stale capability and version negotiation failures, authorization denials and scope escalation, schema-valid output, trace completeness across hops, cancellation and retry correctness, task and effect success, latency, and cost. Security tests should include malicious capability descriptions, prompt injection in resources or artifacts, forged or confused identities, token audience errors, oversized responses, and unavailable peers. Comparisons with a native connector or local agent should report whether standardization reduces integration work without weakening the bounded transition controls required by the application.

4.9 Framework landscape

An agent framework is not the architecture itself; it is a runtime and set of defaults for representing some parts of that architecture. It may externalize graph state while leaving authorization to application code, make handoffs explicit while keeping task completion conversational,

or provide tracing without durable recovery. Feature checklists age quickly as APIs, product names, and maturity levels change. A more durable comparison asks which evidence, state, control, and authority boundaries a framework makes inspectable and enforceable, and which remain implicit or must be supplied by the application.

Table 8. Representative framework categories classified by their externalized representations. Entries are illustrative; documented primitives are not reliability guarantees.

Category	Representative systems	Externalized representation	Architectural evidence to verify
Graph/workflow orchestration	LangGraph; Semantic Kernel	Nodes, typed state, guarded edges, checkpoints, interrupts	Persistence and resume semantics; approval scope; idempotent side effects; failure edges
Agent conversation	AutoGen; CrewAI	Agents, roles, messages, tasks, handoffs	Ownership and message schemas; shared state merge; authority; termination and admission
Data/RAG-centered agents	LlamaIndex; Haystack	Documents, indexes, retrievers, typed pipelines, tools	Provenance, access and refresh policy, retrieval evaluation, action state integration
Native agent runtimes	OpenAI Agents SDK; Google ADK	Agents, tools, handoffs, sessions, guardrails, traces	Model portability; durable recovery; policy boundary; trace and evaluation export
Interoperability protocols	MCP; A2A	Capability schemas, Agent Cards, tasks, messages, artifacts	Authentication, trust and admission boundaries, version negotiation, conformance
Research prototypes	AutoGPT; BabyAGI and related loops	Open-ended plan, task list, memory, and iteration loop	Historical influence; state, authorization, verification, and recovery that must be added

Primary documentation illustrates the variation in what is made explicit. LangGraph describes checkpointed graph state and resumable interrupts [75]; Semantic Kernel exposes multiple agent orchestration patterns while marking important orchestration features as experimental [105]; and Haystack defines schema-based shared agent state with typed component inputs and outputs [27]. AutoGen emphasizes programmable agent conversation [185]; the OpenAI Agents SDK exposes agents, tools, handoffs, guardrails, sessions, and tracing [121]; and Google ADK documents agents, tools, sessions, state, callbacks, evaluation, and deployment abstractions [52]. These sources establish interface capabilities, not comparative production reliability. MAST's observation of recurring coordination failures across seven multi-agent frameworks reinforces that adopting a framework does not remove the need for explicit contracts, verification, and recovery [12].

Framework selection should therefore begin with a representation matrix, not a catalog of integrations. For every required record (the evidence bundle, task and plan node, memory item, tool schema, action proposal, approval, execution receipt, verified effect, incident, and trace), the team should state whether the framework owns it, merely transports it, or leaves it to application code. The same matrix should locate who may create, mutate, approve, invalidate, and inspect each record. A versioned framework selection record can then capture candidate and dependency versions, requirements, gaps, adapters, security and data boundaries, operating limits, the evaluation baseline, and migration and rollback conditions. This turns framework

choice from a one-time preference into an auditable architectural decision.

Portability depends on preserving the application's semantic state outside framework-specific transcripts and identifiers. Adapters should map canonical application records into framework nodes, sessions, handoffs, or events, and map framework results back into typed outcomes. Prompts, tool schemas, policies, evaluators, and traces should be versioned independently where practical. An upgrade is safe only when it cannot silently change the meaning of stored state, completion, authorization, or recovery.

Finally, a proof-of-fit should execute one representative long-horizon workflow with fixed models, tools, evidence, and budgets. It should test restart from checkpoint, approval edits and invalidation, duplicate delivery, malformed tool output, stale evidence, timeout, cancellation, partial side effects, worker failure, and trace export. Report task success and verified effect success separately, together with recovery and replay success, policy violations, trace completeness, latency and cost, required custom code, and behavior under injected faults. The preferred framework is the one that provides the smallest trustworthy extension surface for the system's actual obligations, not the one with the longest feature list.

4.10 Reference design principles

The preceding architectures can be synthesized as a chain-of-custody problem. Evidence is not yet operational state; state is not authorization; a successful API response is not a verified effect; and a plausible final answer is not proof that the intervening process was acceptable. Modern agentic systems become governable when each conversion is represented by a typed, persistent record and admitted through an explicit transition. The following principles are therefore coupled design obligations rather than an independent feature checklist.

1. **Minimize adaptive control.** A model should decide only where the system genuinely needs semantic interpretation, uncertain judgment, or open-ended generation. Routing rules, eligibility constraints, budgets, authorization checks, and terminal conditions should remain deterministic whenever they can be specified. StateFlow's state-driven workflows illustrate how explicit states can organize model calls [187], while long-horizon benchmarks such as τ-bench and AppWorld show that fluent tool use does not by itself produce consistent rule following or correct environment state [166, 202]. The model should therefore emit a proposal with a reason code and supporting evidence; a deterministic controller should decide whether that proposal is an admissible transition.

2. **Make state explicit.** Every consequential decision should read from and write to a declared state model rather than depend on facts recoverable only from a transcript. At minimum, the architecture should externalize evidence bundles, task and plan nodes, memory items, action proposals, approvals, execution receipts, verified effects, incidents, budgets, and termination status. Each record needs an identifier, type, schema version, provenance, owner, timestamps, and lifecycle state. Cognitive agent architectures motivate the separation of working, episodic, semantic, and procedural state [157]; LangGraph's checkpoints and Haystack's schema-based agent state provide current implementation examples [27, 75]. These primitives are useful only when the application also defines merge, invalidation, retention, and authority semantics.

3. **Treat tools as privileged interfaces.** A tool joins language interpretation to an external capability and is therefore both an execution boundary and an attack surface. Capability discovery, argument construction, authorization, execution, and effect verification should be distinct stages. The action proposal should be immutable once approved, and authorization should evaluate the authenticated actor, current state, requested scope, intended effect, and applicable policy at execution time. τ-bench exposes failures in policy-

constrained tool interaction [202], while InjecAgent demonstrates that untrusted tool content can redirect an agent toward harmful actions or data exfiltration [209]. Consequently, tool outputs must return as untrusted observations, and a successful call must yield a receipt rather than automatically mark the task complete.

4. **Verify at natural boundaries.** Verification should occur whenever the meaning or authority of a representation changes: evidence before synthesis, a plan before delegation, arguments before a tool call, receipts before effect claims, and artifacts before acceptance. Schema-valid function calls measure only one boundary [130]. AppWorld instead evaluates final application state and checks for collateral changes [166], while ScienceAgentBench evaluates executable scientific artifacts rather than relying only on textual plausibility [20]. A strong verifier should therefore produce its own typed record containing the tested claim, method, evidence, result, confidence or tolerance, and disposition. Failed verification should lead to a defined repair, rollback, escalation, or terminal state rather than another unconstrained model turn.

5. **Bound every loop.** Interleaved reasoning, search, reflection, and revision can improve difficult trajectories [150, 200, 201], but each additional cycle also expands cost, latency, and the opportunity for non-progress or policy drift. Step, time, token, monetary, retrieval, tool call, and retry budgets should be explicit state fields decremented by the controller. Progress should be defined over observable state changes, for example, a newly satisfied dependency, reduced uncertainty, or verified partial effect, rather than the production of more text. When no progress occurs within a declared window, the system should terminate, re-plan under a bounded alternative, or escalate with the last valid checkpoint and unresolved obligations.

6. **Design for failure.** Failure is not a single exception but a family of state transitions. An error record should distinguish malformed output, denied authorization, stale evidence, unavailable dependency, timeout, duplicate delivery, inconsistent state, partial side effect, failed verification, and exhausted budget; it should also mark the condition as retryable, compensatable, terminal, or requiring human judgment. Durable checkpoints support controlled resume [75], while saga-style compensation provides a foundation for reversing distributed partial work when atomic rollback is unavailable [47]. The recurring coordination failures identified by MAST show why recovery cannot be delegated to conversation alone [12]. A retry is safe only when its input state is still valid and its external effect is absent, idempotent, or compensatable.

7. **Trace decisions and effects.** A trace should be a causal graph of records, not merely a chronological transcript. Evidence identifiers, task and plan identifiers, model and prompt versions, policy decisions, tool arguments, approvals, receipts, effect checks, retries, costs, and terminal reasons should be connected by stable trace and parent identifiers. Causal ordering matters when events cross workers or agents [74]; tracing interfaces such as those exposed by the OpenAI Agents SDK make collection easier but do not determine the application's semantics [121]. The trace is complete only when a reviewer can move from a claimed outcome back to the evidence and authority that justified every consequential transition.

8. **Version adaptive components.** Prompts, memory policies, skills, retrieval configurations, evaluators, tool schemas, protocol adapters, and model dependencies change the meaning of state and must therefore be deployable, migratable, and reversible artifacts. The execution record should identify the versions actually used, not merely the versions currently configured. Evidence staleness [170] and memory poisoning [24] demonstrate that retained representations can become invalid or adversarial even when application code is

unchanged. An upgrade should specify compatibility rules, migration and re-evaluation requirements, dependent approvals or caches to invalidate, rollback criteria, and the state version to restore.

Together, these principles impose two architectural invariants: no external side effect should occur without a current, authorized action proposal, and no task should be declared successful without a verified effect or an explicit unresolved status. The practical implementation sequence follows the same narrative: define canonical schemas and lifecycle states, persist an append-only transition history, enforce policy and authorization gates, make execution idempotent or compensatable, and attach effect verifiers and evaluation hooks. This turns the chapter's taxonomy into an engineering argument: agentic capability becomes dependable only when evidence, state, authority, action, and verification remain linked throughout the trajectory.

5 Evaluation Methodologies and Benchmarks

Evaluation closes the architectural argument developed in the preceding sections. If a modern agentic system is intended to transform evidence into bounded action through explicit representations, an evaluation must determine whether those representations remained valid and whether each transition preserved its contract. A final answer alone collapses critical distinctions: an agent can produce the right text from unsupported evidence, reach the target state through a prohibited action, receive a successful API response without producing the intended effect, or fail safely after correctly denying an inadmissible request. AppWorld's state-based tests expose both goal completion and collateral changes [166], while τ-bench evaluates final database state, policy following, and consistency across repeated trials [202]. These designs demonstrate why evaluation must cover evidence, state, authority, process, effects, resources, recovery, and risk.

The object being evaluated is therefore the configured system, not the base model in isolation. Its evaluation manifest should identify the model and decoding settings, prompts, retriever and evidence snapshot, memory policy, tool schemas, permissions, orchestration logic, budgets, verifiers, environment version, and termination rules. Changing any of these elements changes the policy that generates the trajectory. A score without this system boundary cannot support a controlled comparison or explain whether an observed gain came from the model, the scaffold, additional resources, a weaker evaluator, or privileged information.

A recent survey organizes agent evaluation across core capabilities, application-specific benchmarks, generalist agents, benchmark dimensions, and developer-facing evaluation frameworks, while identifying persistent gaps in cost, safety, robustness, and fine-grained diagnosis [203]. We complement that landscape with a systems-oriented question: at which representation or transition did the evidence-to-action chain succeed, degrade, or fail?

5.1 Unit of evaluation: from outputs to episodes

The natural unit for an agent is an *episode*: the evaluation contract, initial state and goal, sequence of proposed and authorized transitions, resulting observations and receipts, terminal state, and all resources consumed. To align with the proposal–gate–validate model introduced earlier, let $z_t = (s_t, o_t, \tilde{u}_t, u_t, \rho_t, d_t, s_{t+1})$ denote a transition record containing the prior state and observation, proposed action, authorized action (or $\perp$), execution receipt, validation disposition, and committed next state. The complete episode is $\tau = (g, \kappa, s_0, z_0, \ldots, z_{T-1}, s_T)$, where g is the goal and κ is the versioned system and evaluation manifest. This record distinguishes what the model proposed, what the system permitted, what the environment acknowledged, and what the verifier accepted.

Evaluation should preserve these distinctions in a vector rather than collapse them immediately into a scalar:

$$\mathbf{M}(\tau) = \langle Q(\tau), C(\tau), R(\tau), S(\tau), V(\tau) \rangle, \tag{7}$$

where $Q(\tau)$ measures verified task and artifact quality, $C(\tau)$ resource use and latency, $R(\tau)$ run-to-run reliability and recovery, $S(\tau)$ safety and policy compliance, and $V(\tau)$ lineage and record completeness. The dimensions are related but not interchangeable. For example, retries may improve eventual quality while increasing cost and exposure, and a system may reach the goal state while violating policy. Hard constraints such as an unauthorized commit, missing receipt, or unverified effect should therefore be reported as gates or disqualifying events rather than averaged away by strong performance elsewhere. Aggregate scores can aid ranking, but Pareto comparisons and the underlying dimension values are more informative.

The same episode should be examined at multiple, linked levels:

- **component level**: test the records produced by retrieval, memory, planning, tool selection, authorization, and verification for schema validity, provenance, freshness, calibration, and local correctness;
- **trajectory level**: test whether transitions were admissible and useful, including measurable progress, loops, redundant or contradictory actions, budget consumption, recovery, and policy decisions;
- **outcome level**: test the claimed artifact or environment effect, unmet obligations, collateral state changes, and whether completion was supported by a valid receipt and effect check;
- **system level**: test throughput, tail latency, cost, reliability under load, shared state contention, privacy, and incident rates across episodes;
- **human–system level**: test workload, intervention timing, trust calibration, override behavior, and downstream consequences for the people who authorize, supervise, or receive the work.

The levels must remain joinable through stable episode, transition, evidence, and artifact identifiers. Otherwise a failed outcome cannot be attributed to stale retrieval, an invalid proposal, a gate error, an execution fault, or a weak verifier. Existing benchmarks provide complementary slices of this hierarchy: AgentBoard uses authored milestones to reveal progress when binary success is sparse [60]; AppWorld uses executable state checks to accept different valid trajectories while detecting collateral damage [166]; and τ-bench's pass^k exposes inconsistency that a single successful run would hide [202]. A complete evaluation should combine such evidence rather than mistake any one slice for the system as a whole.

5.2 Core metrics

The metric vector becomes operational only when each dimension is tied to an observable record, an acceptance rule, and a level of aggregation. Every reported metric should name its unit, numerator and denominator, episode population, budget, treatment of abstentions and blocked actions, and uncertainty across repeated runs. This prevents a component proxy, such as valid JSON or retrieval recall, from being presented as evidence of end-to-end agency. The metrics below follow the episode from requested effect to transition quality, resource use, recovery, memory-mediated action, and safety.

5.2.1 Task success and artifact quality

Task success compares a typed acceptance contract with a verified effect. AppWorld checks application state and collateral changes [166], SWE-bench executes repository tests [66], and ScienceAgentBench evaluates generated programs and cost [20]. Outcome records should separate effect success, artifact validity, constraints, collateral damage, and unresolved obligations. Partial credit aids diagnosis but is not completion when the missing step is authorization, submission, or verification.

5.2.2 Trajectory quality

Trajectory quality distinguishes proposed, authorized, attempted, committed, compensated, and verified actions and measures progress, repetition, correction, rejection, and recovery. AgentBoard adds intermediate milestones [60]; MAST exposes coordination and termination failures hidden by plausible conversation [12]. For multi-agent systems, delegation, duplication, disagreement, and orphaned work also matter. Shorter is not automatically better when verification creates necessary evidence.

5.2.3 Efficiency and cost

Efficiency should attribute tokens, calls, tools, waiting, retries, verification, human work, and latency to transitions. OSWorld-Human reports roughly 1.4–2.7 times excess steps for evaluated agents and finds planning and reflection calls dominate latency [2]. Longer trajectories also enlarge error and attack surfaces. Report quality–cost and quality–latency frontiers under fixed contracts rather than one ratio that can hide low absolute success.

5.2.4 Reliability and recovery

Reliability asks whether the same contract produces acceptable effects across sampling and environment variation. τ-bench's pass^k measures repeated success [202]; reports should include first attempt success, abstention, denial, catastrophic failure, and cost variance. Recovery is distinct: fault injection should measure restoration from the last valid checkpoint without duplicate effects. Eventual completion alone does not establish recovery.

5.2.5 Memory and state

Memory evaluation follows admission, update, retrieval, conflict, use, invalidation, and deletion. MemBench emphasizes recall, whereas Mem2ActBench tests tool use grounded in retained state [147, 163]. Report write and retrieval quality, provenance, staleness, scope leakage, deletion, and action improvement against stateless and long-context baselines. Poisoning tests should trace admission through realized action to localize the failed layer [24].

5.2.6 Safety and side effects

Safety evaluation distinguishes unsafe observations and proposals from policy decisions, attempted execution, and realized effects. Agent Security Bench spans prompts, tools, and memory [212]; InjecAgent isolates indirect tool content attacks [209]; and ST-WebAgentBench adds explicit policies to web tasks [79]. Safety must be reported with verified utility: dependable systems preserve authorization and effect invariants without blocking everything.

5.3 Evaluation protocols

An evaluation protocol determines which parts of an episode can be observed and which conclusions the evidence can support. No single protocol covers the full chain. Offline tests isolate

record contracts cheaply; interactive environments expose stateful consequences and recovery; human review addresses underspecified value judgments and downstream consequences; and model judges scale bounded semantic assessments. Every protocol should emit a versioned manifest, the evaluated inputs and outputs, the applicable oracle or rubric, and a judgment record that can be joined back to the episode trace.

5.3.1 Offline component tests

Offline component tests are fast, inexpensive, and well suited to continuous regression checks. A case should specify the component and version, initial typed state, input record, authorization context, expected output schema or invariant, oracle, and prohibited outcomes. Useful suites cover retrieval and claim support, memory admission and conflict resolution, plan validation, tool selection and arguments, policy decisions, transition guards, and evaluator calibration. RAGAS illustrates reference-free and reference-based component measures for retrieval-augmented generation [38], while BFCL uses exact and executable checks for function selection and argument construction [130].

Component success is necessary but local. A valid tool call may be unauthorized, a relevant document may be stale, and a calibrated verifier may still receive an incomplete evidence bundle. Mutation and property-based tests should therefore vary schemas, missing fields, scope, freshness, conflicting records, and boundary values. These tests identify the failing contract precisely, but they cannot establish long-horizon progress, external effects, concurrency behavior, or recovery in a changing environment.

5.3.2 Interactive environments

Interactive protocols execute an agent in a sandboxed website, operating system, repository, simulator, or tool environment. Their contract should specify the initial environment snapshot, goal and acceptance criteria, available observations and tools, identity and permissions, applicable policies, budgets, reset semantics, expected effects, and prohibited collateral changes. WebArena and OSWorld evaluate agents in stateful interfaces [192, 219]; AppWorld adds controllable application state and programmatic unit test evaluators [166]. These environments provide stronger evidence of action correctness than static datasets because the agent must observe, decide, act, and handle the resulting state.

The added validity comes with setup cost, nondeterminism, environment maintenance, evaluator blind spots, and contamination from public tasks or trajectories. Results should identify the environment image or snapshot, evaluator and tool versions, model and scaffold manifest, seed where applicable, prompts, permissions, budgets, reset procedure, and network policy. These records make runs interpretable and longitudinal comparisons defensible. When live services or web evidence are necessary, the protocol should preserve the observations, requests, responses, timestamps, and verified final effects instead of treating an unversioned external environment as fixed.

5.3.3 Human evaluation

Human evaluation remains necessary for underspecified goals, usability, social and organizational consequences, and judgments whose authority properly belongs to people. Human-centered AI work emphasizes combining high automation with meaningful human control rather than treating the two as opposites [151]; established levels-of-automation research likewise shows that the placement of human and machine responsibility changes system performance and failure [127]. A protocol should therefore state whether the person is a user, domain expert, approver, supervisor, or outcome reviewer and what evidence and authority that role receives.

Reviewers should inspect the goal and rubric, relevant evidence, proposed and authorized actions, receipts and effects, and the parts of the trajectory needed for attribution, not only the final answer. The judgment record should capture criterion-level ratings, rationale, confidence, requested clarification, override or escalation decisions, and unresolved disagreement. System identity should be blinded when it is not part of the question; presentation order should be randomized; and inter-annotator agreement, adjudication, reviewer expertise, and workload should be reported. Human review should also measure whether intervention occurred at the right boundary and whether system confidence supported calibrated reliance.

5.3.4 LLM-as-judge

Model judges can scale semantic evaluation of open-ended artifacts when exact or executable oracles are unavailable. G-Eval reports improved alignment with human judgments for summarization and dialogue while also identifying bias toward model-generated text [89]; MT-Bench and Chatbot Arena show that strong judges can approximate aggregate human preferences but exhibit position, verbosity, self-enhancement, and reasoning biases [215]. A model judge should therefore be treated as another fallible evaluator component, not as ground truth.

Each judgment should persist the judge model and version, decoding settings, prompt and rubric version, evidence supplied, candidate ordering, criterion-level scores, rationale, confidence, and refusal or parsing status. Judges should be calibrated against expert labels on the target domain, blinded to irrelevant system identity, tested for order and length sensitivity, and supplemented by deterministic or executable checks. Pairwise comparisons can be more stable than absolute scores but still require randomized order and disagreement reporting. For grounded tasks, the judge should receive the authoritative evidence bundle and assess claims individually; for tool or environment tasks, it must not override an execution-based effect result.

The strongest evaluation program layers these protocols in increasing scope: isolate contracts offline, exercise representative transitions interactively, use humans for legitimate value and authority judgments, and use calibrated model judges only for bounded semantic criteria. When the layers disagree, the report should preserve the disagreement and identify which evaluator has authority for the claim in question. This protocol hierarchy prevents a convenient proxy from silently replacing the effect, policy, or human consequence the system was meant to satisfy.

5.4 Representative benchmarks

A benchmark is a measurement instrument whose environment and evaluator determine which parts of an episode are visible. It can support an architectural claim only when it observes the corresponding representation or effect. Table 9 preserves the breadth and historical difficulty exposed by representative introducing reports; Table 10 and Table 11 then compare diagnostic coverage and blind spots. The reported numbers are historical reference points rather than current leaderboard claims, because model, scaffold, budget, and evaluator performance change rapidly.

For interpretation, each benchmark should be read as a contract over six elements: interaction substrate, initial and terminal state, records exposed to the evaluator, permitted actions and policies, temporal horizon, and acceptance oracle. A high score on function calling, retrieval, or milestone progress is evidence for that boundary; it is not automatically evidence of authorization correctness, recovery, collateral effect control, or end-to-end task success.

Table 9. Representative benchmark breadth and historical difficulty. "Original evidence" reports a result from the introducing work or official initial release to contextualize the challenge at publication time, not to rank current systems.

Benchmark	Interaction substrate	Scale	Primary stressor	Original evidence of difficulty
GAIA [104]	Web, files, multimodal tools	466 questions	General assistant reasoning, browsing, tool use	Human respondents: 92%; GPT-4 with plugins: 15%
WebArena [219]	Controlled websites	812 tasks	Long-horizon web navigation and action	Human: 78.24%; best GPT-4-based baseline: 14.41%
OSWorld [192]	Real desktop operating systems and applications	369 tasks	Multimodal computer use across applications	Human: 72.36%; best reported baseline: 12.24%
SWE-bench [66]	Executable GitHub repositories	2,294 issues	Repository-scale software repair	Best original model: 1.96%; Verified later curated 500 human-checked tasks
τ-bench [202]	User–agent–tool simulation	Multi-domain dialogues	Policy following, tool use, and user interaction	Large variance across policies and repeated trials; pass^k exposes inconsistency
Multi-agent benchmark [221]	Cooperative and competitive tasks	Multiple scenarios	Coordination, milestones, communication topology	Performance depends materially on coordination topology and planning
Memory-to-action [147]	Persistent assistant sessions and tools	400 tool use tasks from 2,029 sessions	Active use of long-term memory	Seven evaluated memory frameworks remained inadequate at parameter grounding
Agency benchmark [83]	Dockerized real-world work scenarios	138 tasks, 32 scenarios	Six capabilities over long, feedback-rich episodes	Average 90 tool calls and about 1M tokens; closed vs. open models: 48.4% vs. 32.1%
Agent Security Bench [212]	Tool-using agent attack scenarios	10 scenarios, 400+ tools	Attacks and defenses for agent systems	Highest average attack success reported across tested settings reached 84.3%

The tables are therefore a coverage map, not a league table. A benchmark portfolio should include complementary evaluators for the specific claims a system makes. For example, BFCL can isolate whether the agent constructed a valid call, AppWorld can test whether application state changed correctly, ST-WebAgentBench can check policy-constrained web behavior, and a separate recovery protocol is still required to determine whether the system handles partial failure. The same model can legitimately receive different conclusions because the instruments observe different boundaries.

5.4.1 Generalist and long-horizon evaluation

Generalist benchmarks trade diagnostic depth for breadth. GAIA spans browsing and multimodal tool use [104]; AgentBench aggregates heterogeneous environments [88]; and AgentBoard adds progress milestones [60]. AgencyBench and OSWorld 2.0 increase the horizon: the former averages roughly ninety tool calls and one million tokens, while the latter evaluates 108 workflows and reports a best configuration of 20.6% under its stated 500-step protocol [83, 207]. These results expose state divergence, missed constraints, and weak verification, but their aggregate scores remain configuration- and evaluator-dependent evidence rather than universal rankings.

5.4.2 Web and computer use agents

WebArena, OSWorld, WorkArena, AppWorld, and ST-WebAgentBench make interface or application state part of the evidence and differ in whether evaluators inspect terminal pages, database fields, policy decisions, or collateral effects [32, 79, 166, 192, 219]. Their execution-based scoring is stronger than static answers but sensitive to interface drift and incomplete

Table 10. Diagnostic coverage of representative general and application benchmarks. Scores across incompatible substrates should not be compared directly.

Benchmark	Interaction substrate	Primary evaluator	Strongest evidence	Key limitation
AgentBench [88]	Multiple text and interactive environments	Per-environment success	Breadth across distinct agent tasks under a common protocol	Aggregate scores can hide scaffold choices made for each environment
AgentBoard [60]	Multi-turn embodied, game, web, and tool tasks	Progress and success milestones	Fine-grained progress diagnosis when end-to-end success is sparse	Authored milestones are proxies for useful progress
WorkArena [32]	Enterprise knowledge work applications	Task-specific browser state	Realistic navigation and business object manipulation	Replica environments do not capture every production permission or integration
AppWorld [166]	Controllable apps, APIs, and simulated users	Executable state checks	Complex cross-application coding and tool interaction	API-mediated interaction differs from noisy live software and human use

oracles. Environment version, permissions, reset health, checked effects, and efficiency must therefore accompany completion; OSWorld-Human shows that step excess and planning or reflection calls can dominate latency [2].

5.4.3 Software engineering agents

SWE-bench evaluates real repository repairs with executable tests [66], while SWE-agent shows that the interaction interface materially affects performance [199]. Executable oracles are strong but bounded by coverage and environment. A 2026 audit reported material design or description issues in 59.4% of the 138 SWE-bench Verified problems it examined [122]; the result motivates continuous task, oracle, contamination, and version audits rather than rejection of execution-based evaluation.

5.4.4 Memory, multi-agent, and security benchmarks

Specialized benchmarks isolate otherwise hidden representations: MemBench measures recall, Mem2ActBench memory-mediated tool use, MultiAgentBench coordination, Agent Security Bench cross-surface attacks and defenses, and InjecAgent indirect prompt injection [147, 163, 209, 212, 221]. Each supports only the boundary its evaluator observes. Benchmark selection should therefore map every architectural claim to the episode record and transition that would prove it and list unobserved claims as evaluation gaps.

5.5 Benchmark validity and common pitfalls

A benchmark score is not an intrinsic property of a model. It is the end of an inference chain that begins with a task distribution and a complete agent configuration, passes through a par-

Table 11. Diagnostic coverage of specialized tool use, safety, and scientific agent benchmarks.

Benchmark	Interaction substrate	Primary evaluator	Strongest evidence	Key limitation
BFCL [130]	Function schemas and multi-turn tool calls	Exact and executable call checks	Tool selection, argument construction, and stateful function calling	Function correctness does not establish end-to-end goal completion
ST-WebAgent-Bench [79]	Stateful web tasks with safety constraints	Utility and safety checks	Joint measurement of task success and trustworthy behavior	Threat and policy coverage remains bounded by authored scenarios
DiscoveryWorld [63]	Interactive simulated scientific environments	Discovery objectives and environment state	Experiment planning, hypothesis testing, and evidence gathering	Simulated science is more observable and resettable than real laboratories
ScienceAgent-Bench [20]	Data-driven scientific research tasks	Artifact and task-specific checks	Scientific coding and analysis grounded in realistic research workflows	Coverage is concentrated in tasks that admit computable artifacts

ticular environment and set of episode records, and terminates in an evaluator. Modern agentic systems make this chain unusually fragile: the same language model can be placed inside different planners, memory policies, tool interfaces, retry loops, and permission boundaries, while the same trajectory can receive different judgments from different acceptance oracles. Benchmark validity therefore asks whether the recorded evidence supports the claim being made, not merely whether a scoring script returned a number. Six recurrent threats break that connection.

Scaffold confounding. An agent result reflects a compound configuration. SWE-agent shows that interface design can materially change performance [199], and Kapoor et al. warn that additional inference or scaffold choices can be misattributed to agent design [69]. Model comparisons should hold scaffold and resources fixed; system comparisons should publish the full manifest and ablate the source of gains.

Budget confounding. More plans, retries, agents, context, or stronger judges can buy accuracy. Cost-controlled evaluation and success–cost frontiers expose whether a result reflects better policy or simply more search [69]. Comparisons should report common caps and realized tokens, calls, time, retries, and workers; differing budgets support only utility claims at those resource levels.

Contamination and public trajectories. Public tasks and trajectories can enter training, retrieval, prompts, or templates, measuring exposure rather than transfer [69]. Time-split, private, and refreshed sets reduce but do not prove absence of contamination [15]. Reports should

preserve task provenance, dates, splits, consulted artifacts, and known exposure and stratify results by public, held-out, and newly authored subsets.

Evaluator weakness. Tests can miss obligations, state oracles can miss collateral effects, and LLM judges can reward fluency or agreement. The SWE-bench audit illustrates the need to revalidate even human-filtered executable tasks [122]; model judges require calibration for position, verbosity, and rubric sensitivity [89, 215]. Oracles should be tested for false acceptance and rejection and disclose exactly which fields they observe.

Environment drift. Interactive benchmarks drift with browsers, packages, APIs, accounts, fixtures, and time. Runs should identify environment and dependency versions, permissions, reset result, evaluator, and timestamp. Baseline health tasks distinguish agent regressions from broken infrastructure; intentional environment changes begin a new benchmark version.

Single-run reporting. One trajectory shows possibility, not dependability. τ-bench distinguishes single-trial success from pass^k repeated success [202]. Reports should separate pass@k from pass^k and state sampling, trial count, retries, and confidence intervals. When full repetition is costly, use a prespecified representative or risk-weighted subset and retain traces for failure cluster analysis.

These threats are not independent housekeeping items. An unrecorded scaffold can consume an unrecorded budget, exploit a public trajectory, pass a weak oracle, and appear stable only because the environment and random seed never changed. The remedy is correspondingly structural: a system manifest, resource ledger, exposure ledger, oracle validation record, environment snapshot, and repeated episode outcomes. Together they turn a leaderboard number into a scoped claim whose alternative explanations are visible and testable.

5.6 A practical evaluation pipeline

A publishable and deployment-oriented evaluation pipeline is best understood as a control loop, not a terminal checklist. It converts an architectural or capability claim into an evaluation contract, collects the records needed to test that contract, and uses the resulting evidence to bound a research conclusion or release decision. The loop remains active after deployment because tasks, environments, models, policies, and risks change. This lifecycle view is consistent with the NIST AI Risk Management Framework's progression from mapping context to measuring and managing risk [117, 161]. In practical terms, the pipeline should:

1. **Scope the claim.** Define the task distribution, intended users, operating context, required artifacts or effects, and unacceptable outcomes. State whether the claim concerns a model under a fixed scaffold, a complete agentic system, or deployment utility; those are different estimands.

2. **Encode the evaluation contract.** Bind every run to a system manifest, environment version, initial state, permitted actions, resource limits, temporal horizon, evaluator-visible fields, and acceptance oracle. Claims that the oracle cannot observe should be listed as evaluation gaps before experiments begin.

3. **Challenge components and transitions.** Test retrieval evidence, plan and task node constraints, tool proposal schemas, memory admission and invalidation, policy gates, effect verification, and the evaluators themselves. Component tests localize defects that an end-to-end score would merge into a generic failure.

4. **Run complete episodes.** Execute from a declared initial state to a terminal artifact or verified external effect, retaining the joined evidence, state transition, action, receipt, and judgment records. State-based tests such as AppWorld's can accept different valid trajectories while also detecting collateral changes [166].

5. **Estimate stability and robustness.** Repeat a prespecified task set under fixed resource policies, then introduce semantic perturbations, tool faults, delays, and recovery opportunities. Report both average success and repeated success measures such as pass^k so that occasional capability is not confused with dependable behavior [202].

6. **Diagnose through linked records.** Classify failures with a stable taxonomy, but preserve the causal path from the first violated evidence or state contract to downstream decisions and effects. Progress-sensitive instruments such as AgentBoard show why intermediate milestones can be more diagnostic than terminal success alone [60].

7. **Report a multi-objective result.** Present task and artifact quality together with verified effect success, constraint satisfaction, safety, abstention, recovery, latency, token and tool cost, and uncertainty. Include task-level or risk-stratified breakdowns and success–cost frontiers rather than compressing incompatible outcomes into one rank.

8. **Audit the instrument.** Measure oracle false acceptance and rejection, check environment health, review exposure and holdout risks, challenge boundary cases, and retire or revise invalid tasks. The continuing reassessment of SWE-bench demonstrates that benchmark maintenance is part of the evidence lifecycle [122].

9. **Gate change and monitor deployment.** Translate the evaluation contract into explicit release thresholds, rollback and escalation triggers, and post-deployment monitoring. Use the same episode and effect schema for regression tests, production incidents, user appeals, and change review so that new evidence can update the claim rather than begin an incomparable record.

No single benchmark covers this full chain. AgentBoard exposes authored progress milestones, AppWorld evaluates terminal state and collateral effects, τ-bench measures repeated behavioral consistency, and ST-WebAgentBench adds explicit safety and trustworthiness policies [60, 79, 166, 202]. Their results become comparable evidence only after each instrument is mapped to the claims, records, transitions, and blind spots it can actually observe.

The central principle is that agent evaluation should use the same explicit, typed, and persistent representations demanded of the systems it studies. A leaderboard score is then a view over an episode ledger, not the evidence itself. The ledger must preserve what was attempted, which evidence and constraints governed each transition, what effects occurred, how those effects were judged, and which residual risks remained. Evaluation thereby closes the survey's core loop: evidence supports a bounded action, the action produces new evidence, and that evidence controls whether the system may progress, recover, or stop.

6 Reliability, Security, and Governance

Safety in an agentic system is not primarily a property of its generated text; it is a property of the transition from evidence to external effect. A chatbot can produce harmful content, but an agent can additionally read private data, admit information into persistent memory, invoke tools, communicate with third parties, and modify authoritative state. A failure becomes consequential when untrusted or unsupported evidence is admitted as state, when data is interpreted as instruction, when a proposed action receives authority beyond the user's purpose, or when

an effect is accepted without verification. Risk therefore arises across the entire evidence–state–action path rather than at the model boundary alone.

This view turns safety from a late output filter into a set of enforceable representation and control boundaries. Retrieved content, user instructions, tool results, memories, plans, action proposals, authorizations, and execution receipts should remain distinguishable through typed records with provenance and scope. The runtime can then apply least privilege, policy checks, confirmation, and effect verification at the transition where each control matters. OWASP's excessive-agency category identifies excessive functionality, permissions, and autonomy as distinct causes of damaging action [124]; the architectural response is correspondingly to bound all three rather than ask the language model to compensate for an overpowered runtime. Safety evidence must therefore show both that hazardous transitions are blocked and that legitimate work remains possible.

6.1 Threat model

A threat model makes those boundaries explicit. It should identify protected assets, actors, trust zones, admissible information flows, granted authorities, permitted effects, and recovery obligations. Relevant assets include credentials, private documents, user identity, financial resources, source code, memory stores, tool registries, decision records, and the integrity of external systems. Actors may include malicious users, compromised documents or websites, untrusted tool servers, peer agents, insiders, and accidental misconfiguration; the model and planner should also be treated as fallible components rather than trusted policy authorities.

For each possible state transition, the threat model should ask: who introduced the evidence; how its origin and integrity are represented; which principal and task authorize its use; what data the transition may read or disclose; which external effects it may produce; how those effects are verified; and how the system contains or reverses failure. This converts a generic asset inventory into testable security contracts. For example, a retrieved email may be admissible as evidence about a user's request while remaining inadmissible as authorization to send data, change memory, or invoke a financial tool.

The empirical literature covers complementary parts of this contract rather than one universal notion of security. InjecAgent contains 1,054 indirect prompt injection cases spanning 17 user tools and 62 attacker tools; among its tested configurations, a ReAct-prompted GPT-4 agent executed the attack in 24% of cases [209]. Agent Security Bench broadens coverage to ten scenarios, more than 400 tools and tasks, and 27 attack or defense methods; it reports a highest average attack success rate above 84.3% across its evaluated configurations [212]. ST-WebAgentBench instead attaches 3,057 safety and trustworthiness policies to 375 web tasks and measures six dimensions, explicitly coupling constraint satisfaction to useful task completion [79]. These results are not directly rankable because their actors, actions, policies, and acceptance rules differ. Together they support a narrower systems conclusion: as connectivity, persistence, and authority expand, every newly reachable transition requires a corresponding evidence label, policy gate, and effect test. Threat knowledge bases such as MITRE ATLAS help enumerate adversarial tactics, but they must be instantiated in such task-level contracts rather than treated as performance evidence [109].

6.2 Normative guidance and empirical security evidence

Agent assurance requires an evidence stack. NIST frameworks organize governance and risk work [117, 161]; OWASP and MITRE ATLAS enumerate vulnerability classes and tactics [109, 124]; controlled benchmarks test concrete systems under authored threats. None substitutes for another: a framework does not demonstrate prompt injection resistance, a taxonomy does not estimate control effectiveness, and a benchmark pass says nothing about untested threats.

The layers become coherent as an assurance case linking a scoped claim to a threat, control, test contract, versioned evidence, verdict, owner, and residual uncertainty. Stable identifiers and lifecycle state make applicability queryable when models, tools, data, or permissions change. Table 12 therefore compares what each source supports and where its inference must stop.

Table 12. Evidence stack for agent assurance. The sources are complementary rather than substitutable; the final column states the supported use and its limit.

Source	Instrument	What it contributes	Supported use and limit
NIST AI RMF [161]	Normative framework	Govern, map, measure, and manage outcomes across the lifecycle	Assign ownership and structure risk work; does not validate a particular runtime or control
NIST GenAI Profile [117]	Cross-sector risk profile	Generative AI risks and suggested lifecycle actions	Extend the control portfolio to model, data, content, and human system risks; not an agent test
OWASP LLM Top 10 [124]	Vulnerability taxonomy	Application risks including prompt injection, improper output handling, and excessive agency	Build coverage and design checklists; does not measure prevalence or mitigation effectiveness
MITRE ATLAS [109]	Living adversary catalog	Tactics, techniques, mitigations, and case study vocabulary for AI-enabled systems	Map threats, red-team cases, detection, and incidents; coverage evolves and is not a system score
InjecAgent [209]	Controlled benchmark	Indirect prompt injection through tool-integrated content	Estimate attack and defense behavior for covered tools and attacks; not general agent security
Agent Security Bench [212]	Controlled benchmark	Multiple attack and defense classes across tool-rich settings	Compare resilience under its configurations; claims remain bound to its scenarios, agents, and metrics
ST-WebAgent-Bench [79]	Policy-and-utility benchmark	Safety constraints and useful completion in stateful web tasks	Test policy compliance without rewarding blanket refusal; scope remains the authored web workflows

The practical implication is a claim-to-evidence chain. NIST can assign responsibility, OWASP and ATLAS map threat coverage, and benchmarks exercise specified transitions. The result supports only the observed controls and effects; residual threats, untested permissions, and evaluator blind spots remain open claims.

6.3 Prompt injection and instruction hierarchy

Prompt injection is representation collapse: source, type, and trust are flattened into text, allowing data to be interpreted as control. Direct attacks supply competing instructions; indirect attacks place them in later retrieved content [53]. The consequential failure is not textual disobedience but lower-trust evidence altering an objective, invoking authority, or redirecting an effect.

InjecAgent's 1,054 cases and AgentDojo's 629 security tests expose complementary tool-mediated attacks [25, 209]. Their outcomes depend on prompts, tools, permissions, task state, and evaluator policy rather than the model alone. Adaptive attacks exceeded 50% success against all eight studied defenses [210], showing that a fixed prompt suite is a bounded test contract, not a durable security boundary.

Instruction hierarchy and StruQ restore separation between higher-trust instructions and lower-trust data and report gains with limited utility loss [16, 171]. Their scope is model interpretation. A trust label matters only when created outside attacker control, and ranked instructions do not authenticate a principal or authorize an effect.

A reliable architecture must preserve the trust distinction across the complete evidence-to-action path rather than ask the model to recover it from wording. The resulting controls form a linked chain:

- **Ingest as evidence.** Store external content in typed records carrying origin, principal, channel, retrieval scope, integrity state, and permitted use; sanitization may remove active content but must not be treated as proof of trust.
- **Assemble context by policy.** Keep instructions and evidence in separate fields, compute precedence from authenticated channels rather than self-declared text, and minimize secrets or irrelevant instructions exposed to the model.
- **Constrain plan formation.** Propagate provenance from evidence into claims and proposed arguments; content may support a task but may not silently expand its objective, permissions, recipients, or action space.
- **Authorize outside the model.** Validate every tool request against the authenticated principal, declared purpose, resource scope, current state, and action class. The model proposes an invocation; a deterministic monitor decides whether it is admissible.
- **Separate proposal from effect.** Require explicit confirmation for disclosure, communication, financial commitment, or irreversible change, then bind the approval to the exact arguments and retain an authoritative execution receipt.
- **Evaluate the chain.** Measure benign utility, targeted and adaptive attack success, policy-violating effects, false refusals, and recovery across adversarial documents, compromised tool outputs, and multi-step exfiltration attempts.

The controls are complementary. Typed context without external authorization can still turn model error into action; authorization without provenance can approve poisoned arguments; and confirmation without an exact proposal is ambiguous. Prompt injection resistance is therefore the preservation of authority despite malicious strings, with the model treated as interpreter and proposer rather than final security boundary.

6.4 Tool misuse and confused-deputy failures

Prompt injection explains how low-trust evidence can influence a decision; the confused-deputy problem explains why that influence becomes a security violation. A deputy is confused when it exercises authority that it legitimately holds, but for a requester or purpose that was never entitled to use it [56]. In an agent, the requester may be a user, a retrieved document, a remote agent, or a tool response. If a document can cause an executor holding the user's cloud credential to upload a private file, the failure is not simply incorrect tool selection. Authority from one principal has been redirected by evidence from another.

Preventing redirection requires separating tool description, principal delegation, concrete action proposal, authorization decision, and execution receipt. Delegation defines purpose and scope; the proposal preserves argument provenance; the decision binds one version and current state; the receipt records the effect. A JSON schema validates shape but cannot replace these authority-bearing records.

Benchmarks reveal complementary parts of the problem. ToolEmu uses an LM-emulated sandbox to exercise 144 cases across nine risk types and 36 toolkits, making hazardous trajectories observable without executing them against real systems [140]. Agent Security Bench broadens coverage to ten scenarios, ten agents, more than 400 tools, 27 attack or defense methods, and seven evaluation metrics [212]. These suites identify risky behaviors and comparative weaknesses, but neither a simulated consequence nor a benchmark verdict grants deployment authority. Conversely, step-level monitoring can improve the transition immediately before execution: ToolSafe reports a 65% average reduction in harmful tool invocations and roughly a 10% improvement in benign completion under prompt injection [113]. Such learned guards are useful evidence and defense layers, yet their predictions remain inputs to authorization rather than authorization itself.

Least privilege means narrow executable authority, not fewer tool names. Separate read from write, scope credentials to task and resource, mediate calls after arguments are known, and separate high-impact proposal from commit. Values derived from untrusted content need provenance-aware flow controls, and policy must be rechecked against current state at dispatch.

CaMeL illustrates the architectural alternative to relying on model restraint. It extracts control and data flows from a trusted query, prevents untrusted data from changing program flow, and uses capabilities to block unauthorized exfiltration; within its stated threat model, it completes 67% of AgentDojo tasks with provable security [26]. The result is important precisely because it separates two questions that ordinary agent loops conflate: whether the model can interpret evidence and whether the resulting information flow is authorized. Its remaining utility gap also makes the trade-off visible rather than hiding it behind a single success score.

Tool output remains untrusted evidence after success and must be schema-, size-, provenance-, and content-checked. The governing invariant is *authority non-amplification*: evidence may refine an authorized action but cannot mint authority, change its principal, or broaden its purpose. The final effect must remain a subset of explicit delegation.

6.5 Memory poisoning and privacy

Persistent memory amplifies trust across later tasks, tools, and users. The failure occurs when storage silently promotes an untrusted message to fact, inference to preference, or trajectory to procedure. MemoryBank makes long-term updating and forgetting explicit [217], but persistence still requires distinct types and writers for observations, facts, preferences, summaries, procedures, policies, and authorization artifacts.

Memory records therefore separate payload from immutable identity, type, subject, provenance, validity, sensitivity, purpose, scope, retention, and supersession metadata. Quoted text cannot declare its own authority; summaries retain lineage, corrections supersede, and revocation propagates without preserving unnecessary sensitive payload.

Capability benchmarks and attack studies expose different inference boundaries. MemBench evaluates factual and reflective memory across participation and observation settings, emphasizing effectiveness, efficiency, and capacity [163]; those measurements do not test whether a remembered item is authorized or adversarial. AgentPoison demonstrates the integrity gap directly: poisoning less than 0.1% of a long-term memory or RAG knowledge base produced average attack success rates of at least 80% across three agent settings while changing benign performance by at most 1% [19]. MINJA removes the assumption of direct storage access and injects malicious records through ordinary query interactions and observed outputs [31]. Together, these results show that the write path and the retrieval policy form a joint attack surface; protecting only the database interface leaves model-mediated memory creation exposed.

A secure lifecycle uses separate write and read gates. Admission authenticates the writer, classifies type, checks conflict and sensitivity, and assigns scope and expiry. Retrieval applies access, purpose, validity, and revocation filters before similarity. Returned memory enters planning as typed evidence, not privileged instruction; current task policy decides its influence.

Derived representations inherit privacy obligations. Embedding inversion recovered 92% of tested 32-token inputs and reconstructed personal information from clinical notes [112], so vectors, summaries, entities, caches, and traces are not anonymized substitutes. Access, minimization, retention, and deletion must propagate across every derivative.

Evaluation should test adversarial writes, stale and conflicting records, poisoned summaries, scope leakage, inversion, trace disclosure, and deletion. Utility, integrity, privacy, update correctness, and action safety remain separate outcomes. The invariant is *scoped persistence*: memory cannot outlive its validity, escape its audience or purpose, or grant new authority.

6.6 Protocol and supply chain security

Interoperability is not only a connectivity problem; it is an authority composition problem. MCP standardizes the discovery and invocation of resources, prompts, and tools, while A2A uses Agent Cards to advertise identities, endpoints, skills, and authentication requirements [1, 110]. These records make heterogeneous components legible to one another, but they remain claims supplied by another administrative domain. Indeed, the MCP specification explicitly treats tool descriptions and annotations as untrusted unless they come from a trusted server [110]. A syntactically valid schema or signed Agent Card can establish what an endpoint says about itself; neither establishes that the advertised behavior is safe for the present user, task, or data. If discovery metadata is allowed to confer authority directly, adding a connector silently enlarges the system's privileges and transitive trust.

The control plane therefore maintains a connector manifest binding reviewed identity, version, capabilities, scopes, data flows, side effects, reach, approval, validity, revocation, and tests. Its lifecycle is *discover–review–pin–activate–invoke–monitor–revoke*. Material changes to endpoint, schema, scope, or side effect trigger disablement or reapproval and preserve an incident inventory.

Authentication protects the boundary only with delegation. MCP requires audience-bound tokens and rejects token pass-through [110]; A2A advertises authentication separately from credentials [1]. Remote agents should receive minimal task context and narrow, audience-bound authority, never ambient identity or memory. Returned artifacts remain untrusted evidence.

This separation is necessary because protocol-level attacks target the gap between description and behavior. MCP Security Bench operationalizes attacks including name collisions, malicious instructions in tool descriptions, out-of-scope parameter requests, user-impersonating responses, retrieval injection, and transfers between malicious tools; across more than 400 real tools, the study finds that stronger tool use does not by itself imply stronger resistance [211]. Static allow lists are therefore insufficient: evaluation should exercise malicious servers, tool shadowing, metadata and schema drift after approval, credential audience or scope substitution, compromised updates, exfiltration through tool results, cross-agent privilege escalation, and revocation during an active task. The outcome of interest is not merely whether the protocol exchange succeeds, but whether the composed workflow remains inside the authority recorded in the manifest.

in-toto and SLSA can link a deployed connector to reviewed production provenance [153, 165], but do not prove benign behavior or task authority. The invariant is *bounded composition*: protocol metadata describes capabilities; only local policy against pinned identity, version, and delegation may grant them.

6.7 Side effects and transactional safety

Authorization does not make an action operationally correct. Once a request crosses a tool boundary, the response channel and the effect channel can diverge: a timeout may arrive after a payment was accepted, a successful response may precede eventual visibility, a concurrent actor may invalidate a checked balance or file version, and a multi-system workflow may stop after only some writes. Blind retry can then duplicate an effect, while trusting the response can produce a false claim of completion. Transactional safety therefore asks a second question after "may this action run?": what exact state transition occurred, under which preconditions, and what recovery obligation remains?

The first control is to classify actions by impact and reversibility because different effects require different commit protocols. A useful taxonomy is:

- **read-only**: inspect data without modifying external state;
- **reversible write**: create a draft, branch, or staged change that can be undone;
- **externally visible communication**: send a message, publish content, or contact a third party;
- **financial or legal commitment**: transfer value, accept terms, or submit an official form;
- **destructive or safety-critical action**: delete data, change access, operate equipment, or alter production systems.

This classification is not merely a label for an approval screen. Read-only calls can often be repeated, although their freshness and hidden service-side effects still matter. Reversible writes can execute into a draft, branch, escrow, or staging area before promotion. Communication, financial commitment, and destructive change instead require a narrow authorization bound to the final target and arguments, stronger preconditions, and an explicit commit point; a later apology, refund, or restoration is not equivalent to preventing the original effect. The required authorization and verification should therefore increase with impact, irreversibility, uncertainty, and the number of external systems involved.

Each intended effect should be an *action transaction* linking the exact proposal, principal, target, arguments, preconditions, effect class, idempotency, approval, attempts, postconditions, compensation, and receipts. Its lifecycle separates proposal, authorization, dispatch, commit, verification, abortion, compensation, and reconciliation. Only the runtime advances the record from authenticated policy and external evidence; the goal is durable orchestration, not fictional atomicity across unrelated APIs.

Tool adapters make this record executable by declaring preconditions, postconditions, effect class, retry semantics, and available compensation. ToolGate illustrates the local contract boundary: it maintains typed symbolic state and uses Hoare-style preconditions to gate invocation and postconditions to decide whether a result may update trusted state [90]. GoEX emphasizes a complementary boundary, using undo and damage confinement to support post-facto validation of actions that have already executed [129]. The former prevents an invalid transition from being committed to the agent's state; the latter limits or reverses damage after execution. Neither alone establishes atomicity across independent services, so contract validation, containment, and workflow recovery should be treated as complementary controls rather than competing definitions of reliability.

Before dispatch, the executor revalidates proposal, authority, arguments, and resource version. Retryable mutations use a stable idempotency key enforced by the authoritative service; HTTP warns against automatic retry of non-idempotent requests without additional knowledge [40].

After ambiguity, reconcile state before retrying. In controlled simulations, verify-before-retry raised success from about 58% to 80% and reduced duplicates from about 42% to 20% [99], illustrating why recovery mechanisms require separate measurement.

When no common atomic commit exists, sagas decompose work into local transactions and explicit compensation [47]. Compensation is semantic repair, not time reversal. Atomix delays bufferable effects and tracks compensation; under its stated 0.30 fault setting, the full configuration achieved 57% clean success versus 0–7% for several baselines, while checkpoint replay was statistically tied at 53% [111]. The comparison supports structured recovery without making one runtime universal.

Evaluation should inject failures at every effect boundary: after dispatch but before response, during delayed visibility, on duplicate delivery, under stale preconditions and concurrent writes, after approval revocation, between steps of a multi-system workflow, and while compensation itself fails. Useful outcomes include unauthorized commit rate, duplicate effect rate, false completion rate, residual state divergence, recovery latency, uncompensated harm, and benign completion; collapsing them into task success hides the reason a workflow appears reliable. The governing invariant is *verified commit*: an action may be reported as complete only when the exact authorized proposal was executed against valid preconditions and an authoritative postcondition or receipt confirms the resulting state. Ambiguity remains a reconciliation obligation, not a success state.

6.8 Human oversight

Human oversight is not the presence of a person somewhere in the workflow; it is a transfer of decision authority at a specific state transition. A reviewer must have the appropriate role, receive the exact proposal and relevant evidence, understand why intervention is requested, have enough time to act, and possess an enforceable means to approve, modify, reject, pause, or take over. A notification after an irreversible effect, or an approval button whose decision is not bound to the executed version, provides visibility without control. The allocation of functions between people and automation is therefore an architectural choice rather than a generic requirement to keep a human "in the loop" [127].

Oversight should be placed where human judgment changes the risk of the next transition. Useful modes include co-planning before a plan becomes executable, checkpoint review before a long-running workflow crosses phases, action approval immediately before a high-impact commit, co-tasking or takeover when the system lacks information or control, and post-effect verification when an effect is reversible and damage is bounded. Magentic-UI instantiates these interaction modes through editable plans, co-tasking, action guards, answer verification, multitasking, and saved workflows [115]. Its autonomous, simulated user, qualitative, and targeted safety evaluations make the design space concrete, but they do not establish that every mechanism improves every task. Placement should instead depend on consequence, reversibility, evidence quality, verification strength, time sensitivity, and reviewer capacity. Low-risk, objectively verified transitions may use exception-based review; external communication, financial commitment, permission change, or destructive action should retain a gate at the final commitment boundary.

A typed *review request* binds one proposal and policy trigger to affected resources, evidence, uncertainty, alternatives, constraints, expected effects, reviewer role, deadline, safe default, and recovery. Interfaces may summarize, but should expose material conflict and the exact change under review. This turns "ask the user" into a routable control object rather than another chat turn.

More review is not automatically safer. Imperfect aids produce omission, commission, and complacency errors [126], while passive automation can reduce situation awareness and slow

takeover [35]. Interfaces should use selective gates, clear consequence and provenance displays, easy pause and takeover, and workload limits rather than approval fatigue. Risk and reversibility matter more than model confidence alone.

Escalation is a routing policy. Learning-to-defer models machine and expert jointly [114]; practical triggers include missing authority, conflict, novelty, weak verification, non-progress, or effects beyond delegation. The system must route to relevant authority and distinguish requests for information, judgment, approval, or execution. Field evidence finds different outcomes for technical and emotional escalations and an effect of timing [180], so type and timing belong in evaluation.

The decision record binds reviewer, role, rationale, conditions, expiry, and one proposal version. Modification creates a new proposal that must be rechecked; approval cannot float to a changed action. The receipt shows what occurred. Human correction remains episode-scoped unless separately reviewed for promotion into memory, procedure, or policy.

Evaluation must cover the combined human–agent system rather than report model accuracy plus the existence of a gate. Comparisons should include no-review, blanket review, and risk-triggered-review conditions and separately measure hazardous approvals, correct interventions, unnecessary escalations, reviewer workload, decision latency, time to takeover, recovery quality, residual harm, and benign completion. Adversarial cases should present stale proposals, misleading summaries, omitted conflicts, time pressure, sequences of harmless requests before a dangerous one, and edits made after approval. The governing invariant is *meaningful intervention*: a human decision counts as a control only when an authorized reviewer receives the correct version and material evidence with enough time and agency to alter the outcome, and the resulting effect remains traceable to that decision.

6.9 Observability, audit, and incident response

Observability is not a larger transcript or a dashboard of token counts. It is the preservation of enough typed relationships to reconstruct why a state transition occurred, which authority it used, what evidence it consumed, and what effect it produced. This distinction matters because an agentic run crosses model calls, retrieval services, memory, policy engines, tools, human gates, and external systems. Each component may emit a locally correct log while the end-to-end account remains ambiguous. In delegated or multi-agent execution, the same low-level actions can even be compatible with different assignments of authority unless delegation context is bound to the events when they occur [108]. Observability therefore continues the paper's central argument: evidence becomes governable action only when the system preserves the representations that connect proposal, authorization, execution, and result.

The basic representation is an append-only *trace event* with causal identity, actor and delegation, component version, artifact links, decision and receipt references, status, error, cost, integrity, and retention. Parent and correlation edges connect decomposition, evidence use, handoffs, retries, compensation, and asynchronous work. The result is a causal graph rather than interleaved text.

At minimum, the linked spans and records should cover:

- the task, principal, purpose, constraints, authorization scope, and delegation chain;
- model, prompt, policy, planner, evaluator, memory, and tool versions;
- retrieved evidence and memory items with identifiers, versions, provenance, and applicability;
- plans, state transitions, routing decisions, and the alternatives rejected at consequential

branches;

- tool proposals, validated arguments, precondition checks, requests, responses, retries, and errors;
- guardrail and policy decisions, including the rule, result, and policy version;
- human review requests, approvals, modifications, overrides, expiries, and the proposal version reviewed;
- external side effects, compensation attempts, execution receipts, and final authoritative state; and
- token, latency, compute, monetary, reviewer, and recovery costs.

The AgentOps taxonomy similarly treats observability as lifecycle coverage across agent artifacts and their associated data rather than as model call logging alone [30]. OpenTelemetry's emerging GenAI conventions provide a useful interoperability layer for model, agent, and tool spans [123]; however, a transport schema does not by itself capture the survey's semantic objects: evidence records, memory provenance, proposal versions, authority transfers, and verified effects. Those relationships should remain explicit even when exported through a common telemetry substrate.

Instrumentation must also separate what the agent *represented* from what the environment *did*. Application-level spans can expose goals, plan revisions, retrieval, routing, policy checks, and structured decision summaries. Boundary and system-level telemetry can independently observe network requests, file and process activity, resource use, and external commits. AgentSight illustrates the value of correlating these layers: its boundary tracing design joins semantic intent to system effects, reports less than 3% performance overhead, and detects prompt injection behavior, wasteful reasoning loops, and multi-agent coordination bottlenecks in its evaluated settings [216]. The comparison is architectural rather than universal: application instrumentation is semantically rich but framework-dependent, whereas boundary tracing is harder for the agent to bypass but cannot recover evidence or authority relationships that were never encoded. High-risk systems need both views and should treat the external receipt as authoritative when the two disagree.

TraceElephant reports up to 76.5% better failure attribution from full than partial traces [14], supporting context around decisive steps rather than permanent capture of every prompt. Structural metadata and sensitive content need separate access and retention; secrets should become scoped references. Nominal runs may be sampled, but policy exceptions, overrides, failures, and effects require complete structural records.

Replay is useful only when its semantics are explicit. A *trace replay* renders the recorded execution for inspection; a *decision replay* reruns selected model or policy steps against frozen inputs; and an *effect replay* invokes tools and is therefore unsafe unless performed with mocks, recorded responses, or an isolated test environment. The trace should record which observations were captured, which dependencies were substituted, and where a replay diverged. Because remote data, model behavior, credentials, and time-dependent preconditions change, exact reenactment is often impossible. The practical goal is narrower: localize the first material divergence, test whether a proposed policy or component change would alter it, and preserve the result as a regression case without repeating the original side effect.

Incident response should consume traces through a state machine aligned with preparation, detection, response, and recovery [118]. A typed incident links detection and severity to affected principals, components, credentials, resources, evidence, containment owner, remediation, and

recovery checks. Operators need scoped controls to pause, revoke, quarantine, disable, enumerate, and reconcile; global shutdown is a last resort.

Detection, attribution, and recovery are distinct. AgentDebugX operationalizes Detect–Attribute–Recover–Rerun; in its reported setting, DeepDebug reached 28.8% exact attribution versus 21.7% for a single-pass baseline and repaired 13 of 73 GAIA failures versus 4–6 for three alternatives [222]. The result is configuration-specific but shows that exposing a trace is not the same as diagnosing or repairing failure.

Evaluation should compare final output inspection, flat logs, typed causal traces, and typed traces augmented with boundary telemetry. Useful measures include trace coverage at consequential transitions, causal link accuracy, failure attribution accuracy and time, false attribution, affected episode recall, containment latency, unauthorized post-containment effects, evidence preservation failures, recovery correctness, residual state divergence, and recurrence on prior incidents. The governing invariant is *forensic closure*: an incident is not closed until the responsible authority can identify the affected executions and resources, reconstruct the material evidence–decision–action path, contain further authority, reconcile authoritative state, and bind corrective changes to regression evidence. Observability then becomes the evidence infrastructure for accountability, not a decorative view of agent activity.

6.10 Governance across the lifecycle

Governance connects technical controls to an accountable authority that can permit, constrain, change, suspend, and retire the system. The object being governed is therefore not an isolated model. It is a versioned deployment configuration comprising the model and prompts, planners and evaluators, retrieval and memory policies, tools and credentials, human review routes, runtime policies, user population, operating environment, and allowed effect classes. The NIST AI RMF's govern–map–measure–manage structure provides a useful lifecycle scaffold, while its generative AI profile adds model-specific risk considerations [117, 161]. For agentic systems, however, those functions must be bound to the concrete configuration that can act. A research assistant with read-only access and an operational agent that can modify customer accounts may use the same model while requiring different owners, evidence, controls, and release authority.

The persistent representation for this work is a *system governance record*. It should identify the system and configuration version; accountable owner, risk owner, release authority, and operators; intended and prohibited uses; principals, user populations, and affected parties; autonomy and action boundaries; models, prompts, data flows, memory stores, tools, credentials, policies, and dependencies; a risk register with hazards, controls, residual risk, and acceptance criteria; evaluation and incident evidence; monitoring obligations and thresholds; open exceptions and their expiry; release state and decision; and rollback and decommissioning plans. Links to the evidence, review, trace, incident, and receipt objects defined earlier keep the record compact without severing provenance. A governance claim such as "financial actions remain bounded" should point to the policy configuration, adversarial tests, approval design, and operational monitors that support it, not merely to a policy document.

This claim–evidence structure is an *assurance case*: a reasoned argument that identified properties hold for a defined system in a defined context. Recent work on AI safety cases distinguishes assertion-, constraint-, and capability-based claims; demonstrative, comparative, causal, risk-based, and normative arguments; and multiple evidence families, while emphasizing that prompt, scaffold, and deployment changes can invalidate an earlier case [77]. The useful unit is therefore a versioned graph whose claims declare scope, assumptions, evidence, owner, review status, and expiry. Contradictory or missing evidence remains visible, and a change impact edge identifies which claims must be reopened when a component, permission,

population, or environment changes.

Documentation is necessary but does not govern by itself. A review of 37 documentation frameworks and 22 empirical studies found plausible governance benefits, including risk communication, collaboration, and ethical deliberation, but also weak incentives, limited resources, organizational barriers, and poor integration with existing workflows [182]. A separate analysis of 32,111 Hugging Face model documents found uneven informativeness, with limitations and evaluation among the least completed sections [86]. These studies concern broader AI documentation rather than agent releases, but their comparison exposes a general failure mode: a static card can be complete enough to publish and still be stale, disconnected from an executable configuration, or unable to stop a release. The governance record should therefore be generated from and checked against version-controlled manifests, test results, policy state, and deployment telemetry, with human judgment preserved for risk acceptance and contested trade-offs.

Release should be a state machine rather than a one-time approval. A practical progression is *design–sandbox–shadow–read-only–limited action–bounded production*, with alternative *paused*, *rolled back*, and *decommissioned* states. Each promotion record should bind the exact configuration hash to an owner, deployment population, permitted actions and resources, evaluation results, unresolved limitations, monitoring thresholds, time window, rollback trigger, and signed decision. Evidence requirements should rise with impact, irreversibility, scale, uncertainty, and exposure to adversarial inputs. Runtime enforcement must then make the recorded boundary real. In Aegis's reported sandbox evaluation, prompt policy conditioning produced 79 risky comparator path leakage rows across 6,300 rows, whereas 2,100 governed rows produced zero governed mock tool applications and zero governed risky side effect completions; all attempted governed actions retained server-resolved provenance [101]. The authors explicitly limit the claim to that corpus, but the comparison illustrates why a release policy that exists only in prompts is weaker than a policy bound to the dispatch boundary.

Post-deployment monitoring tests whether the evidence supporting release remains valid. It should derive signals from named hazards and claims rather than accumulate generic dashboards: task and verification failures test capability claims; unauthorized or blocked action attempts test authority boundaries; escalations and reviewer workload test the human control design; retrieval conflicts and stale memory use test evidence quality; near misses, incidents, and complaints test the risk model; and drift in users, tools, data, or environment tests the deployment scope. NIST AI 800-4 organizes monitoring challenges across functionality, operations, human factors, security, content, and impacts, drawing on practitioner workshops and a literature review; it also reports fragmented logging, immature methods and information sharing, and difficulty scaling human monitoring [134]. These findings support broad coverage while cautioning that current monitoring practice is not yet a validated recipe. Every alert should therefore identify the governance claim it threatens, the affected configuration and population, the responsible owner, and the transition it may trigger.

Organizational design must match this continuous technical loop. Product and system owners define purpose and operation; tool, data, and security owners control dependencies and access; a risk owner maintains the hazard and residual risk view; an independent assessor challenges material claims; a release authority accepts or rejects the remaining risk; and operations and incident roles can pause or roll back the deployment. One person may hold several roles in a small organization, but the decision record should make conflicts and authority visible. Mixed-methods research on public sector agent oversight, combining a literature review with interviews of civil servants in AI-related roles, identifies continuous oversight, integration of governance with operations, and interdepartmental coordination as intensified challenges [145]. The evidence is sector-specific, yet it reinforces the architectural requirement that governance decisions, operational controls, and incident evidence share identifiers and escalation paths rather

than live in isolated teams.

Change management closes the lifecycle loop. A change to a model, prompt, retrieval corpus, memory policy, evaluator, tool schema, credential, human review rule, or monitoring threshold should produce a configuration diff and reopen the claims that depend on it. The resulting release packet combines targeted tests, normal and adversarial regressions, prior incidents, operational canaries, and an updated residual risk decision. Progressive exposure limits the consequence of uncertainty, but promotion should depend on evidence thresholds rather than elapsed time or absence of complaints. Decommissioning is also a governed transition: stop schedules and queues, revoke credentials and delegations, disable routes, resolve or transfer pending actions, archive or delete memory and traces under retention policy, notify affected users where required, and verify that no orphan authority remains.

Evaluation should compare paper-only approval, static documentation, versioned governance records, and records coupled to release and runtime enforcement. Useful measures include undocumented configuration drift, stale claim rate, control-to-hazard coverage, unauthorized promotion, policy/configuration mismatch, time to identify the responsible owner, rollback latency and correctness, affected population recall, orphan credentials after retirement, recurrence of prior incidents, residual risk exceptions past expiry, and benign completion under the approved boundary. The governing invariant is *authorized evolution*: autonomy may expand only through a recorded promotion of an identified configuration supported by current evidence and accepted residual risk, and it must contract when that evidence, context, or authority no longer holds. Governance then becomes the state machine that keeps evidence and action aligned throughout the system's life.

6.11 Defense in depth

Defense in depth is not the accumulation of filters. It is the deliberate allocation of prevention, detection, containment, and recovery responsibilities across distinct trust boundaries, so that one plausible failure cannot create, authorize, execute, and conceal a hazardous effect. This interpretation extends the classic principles of fail-safe defaults, complete mediation, separation of privilege, and least privilege from computer security [142] to the evidence–decision–action path of an agent. It also follows system security and system safety engineering in treating assurance as a property of the whole sociotechnical system and its interactions, not a sum of individually reliable components [78, 138]. A prompt instruction, a classifier, a human approval, and a log entry are therefore not interchangeable controls: they occupy different points in the causal path and fail in different ways.

The representation that makes this architecture inspectable is a *defense map*. For each threat or hazard, it records the protected asset or state transition; preventive, detective, containment, and recovery controls; the layer and authority that enforce each control; inputs and trust dependencies; evidence and test identifiers; owner, version, health, and expiry; residual risk and failure assumptions; and escalation and rollback routes. Dependency edges should expose common-mode risk. Two nominally separate checks are not independent if they rely on the same model judgment, prompt-visible labels, tool response, credentials, telemetry channel, or overloaded operator. The defense map thus connects the threat register to the typed evidence, authority, receipt, trace, incident, and governance objects developed throughout this survey.

A defensible agent stack applies that map along the full path:

1. at the *evidence boundary*, typed source, trust, provenance, and freshness fields keep untrusted data distinct from instructions and policy;
2. at the *state boundary*, scoped admission, retrieval, update, retention, and deletion rules prevent memory from silently converting observations into durable authority;

3. at the *planning boundary*, constrained plan and argument objects bind proposed steps to evidence, assumptions, permissions, budgets, and verification conditions;
4. at the *authority boundary*, bounded delegation and least privilege credentials limit principals, resources, actions, duration, and downstream redelegation, with complete mediation on every consequential use;
5. at the *dispatch boundary*, typed schemas, deterministic policy, preconditions, and fail-closed validation decide whether a proposal may become a tool call;
6. at the *effect boundary*, propose–commit separation, version-bound approval, idempotency keys, receipts, and compensation constrain irreversible or externally visible changes;
7. at the *isolation boundary*, sandbox, network, filesystem, secret, time, and resource limits contain compromised code, tools, and dependencies;
8. at the *human boundary*, risk-triggered review, faithful review packets, workload limits, and tested takeover routes reserve judgment for decisions whose residual risk requires it;
9. at the *detection and recovery boundary*, independent telemetry, objective postconditions, non-progress checks, trace replay, incident response, and rollback expose and contain failures that prevention misses; and
10. at the *lifecycle boundary*, adaptive attacks, fault injection, regression tests, staged release, monitoring, and decommissioning determine whether the preceding controls remain effective as the system changes.

The ordering matters because later layers should not merely repeat an earlier model judgment. The proposer should not be its sole evaluator; a self-declared "trusted" label should not establish provenance; the tool response that claims an effect occurred should not be the only postcondition; and a human gate is not independent if the agent can omit material evidence from the review packet or saturate the reviewer. Logging alone is detection evidence, not prevention or containment. Conversely, a strict preventive gate without receipts, monitoring, or recovery can fail silently when a dependency changes. Defense in depth therefore requires both *coverage* across the causal path and *independence* among the controls protecting the same high-impact transition.

Comparative evidence supports this architectural interpretation while delimiting what any one layer proves. In AgentDojo, an adaptive attacker exceeded 50% attack success against every one of eight prompt injection defenses studied, showing that fixed prompt-level defenses should not be treated as a complete boundary [210]. CaMeL instead separates control flow from untrusted data and reports 67% task completion with provable security under its stated threat model, but its utility gap and assumptions leave roles for other layers [26]. ToolSafe reports an average 65% reduction in harmful tool invocations and roughly 10% improvement in benign task completion under prompt injection, yet its learned judgment remains a prediction rather than an execution authority [113]. Aegis's sandbox comparison illustrates the complementary role of an external dispatch gate: prompt policy conditioning leaked 79 risky comparator paths across 6,300 rows, whereas 2,100 governed rows produced no governed mock tool applications or risky effects in that corpus [101]. These results are not directly interchangeable benchmarks, but together they show why instruction robustness, architectural information flow control, learned risk assessment, and deterministic action enforcement should be represented as different defenses rather than collapsed into a single safety score.

Prevention must likewise be connected to diagnosis and recovery. Under a 0.30 injected fault rate, Atomix reports 57% clean task success versus 0–7% for several baselines, while checkpoint replay reached a statistically tied 53%, indicating that structured recovery can preserve

progress after faults that a front-end filter cannot prevent [111]. TraceElephant reports up to 76.5% better failure attribution from full traces than from partial traces, supporting complete causal evidence for incident analysis without implying that traces themselves block harm [14]. Human review adds a distinct form of judgment, but automation complacency and out-of-the-loop performance problems mean that it should be evaluated as an operational control with measurable information quality, workload, and takeover performance rather than assumed to be a universal backstop [35, 126].

Evaluation should compare a single control, staged combinations, and the full mapped stack under the same consequential tasks. Tests should include adaptive prompt injection, poisoned or stale state, compromised tools and monitors, invalid or revoked approval, misleading review packets, telemetry loss, partial execution, recovery failure, and changes that invalidate earlier evidence. Useful measures include hazardous effect escape rate, common-mode bypass rate, hazard-to-control coverage, control independence violations, false refusal, benign completion, reviewer load, containment latency, residual harm, attribution accuracy, recovery correctness, and recurrence after a corrective release. Layer-ablation tests reveal whether an apparent defense is truly independent or simply redundant under ordinary inputs.

The governing invariant is *bounded failure*: for every high-impact transition, no single plausible model, data, tool, policy, operator, or telemetry failure may both produce the hazardous proposal and remove all independent barriers, detection, and recovery; any escaped effect must remain attributable, containable, and recoverable within the declared boundary. Proportionality then becomes precise rather than rhetorical. A lower-impact, reversible action may justify fewer layers, whereas irreversible, high-scale, or rights-affecting actions require stronger coverage and independence. Autonomy should expand only when measured benefit and current evidence justify the residual risk of the complete stack, and should contract when a control, dependency, or operating assumption no longer holds.

7 Applications and Evidence from Deployed or Experimental Systems

Applications are where an agent architecture becomes a claim about the world. A system does not become dependable merely because its model can reason about a domain: it must encode domain evidence in a representation that survives across steps, constrain what that representation may authorize, and verify the resulting artifact or external effect. The same language model can therefore support a low-risk literature assistant, a consequential account workflow agent, or a laboratory controller, while the required state, authority, and evidence differ sharply. Domain labels alone obscure this distinction. We characterize each application by a *representation–action–verification contract*: what persistent object carries evidence, which bounded effect may be produced from it, and which independent mechanism or accountable person determines whether the transition is valid. AppWorld's executable application state checks and τ-bench's joint treatment of policy and final database state illustrate this contract more directly than answer quality alone [166, 202].

Evidence about an application must also be matched to the claim it can support. A controlled benchmark establishes capability only within its declared initial state, tools, permissions, tasks, and oracle; an experimental closed loop shows that a representation can guide a measurable effect within a particular apparatus; an organizational field study can estimate operational outcomes under the deployed intervention; and a product announcement establishes existence, not effectiveness. Peer review can strengthen confidence in any of these designs, but it is not itself an evidence type. The distinction is visible across WorkArena's enterprise software benchmark [32], A-Lab's physical closed-loop system [159], and a field study of an AI assistant used

by 5,172 customer support workers [10]. The last study reports a 15% average increase in issues resolved per hour, but people remained responsible for the conversation and could ignore or edit suggestions. It is therefore valuable evidence about human-supervised assistance, not evidence that an autonomous agent can safely make account changes.

The literature remains asymmetrical: controlled agent benchmarks and engineered demonstrations are much more common than field evidence that joins agent configuration, decisions, realized effects, human workload, and downstream outcomes. Rather than smoothing over that gap, the following tables separate the application contract from the evidentiary boundary of each reported system. This preserves breadth while preventing a successful benchmark episode, a laboratory demonstration, and an organizational outcome study from being treated as interchangeable proof.

Table 13. Application domains as representation–action–verification contracts.

Domain	Evidence-bearing representation	Bounded effect and principal risk	Verifier and authority gate
Enterprise knowledge work	Claim–evidence ledger with source, version, access scope, and contradiction state	Cited report or business object update; unsupported claims, stale policy, or access leakage	Entailment and version checks, authorization-aware retrieval, artifact review
Customer operations	Case state linked to policy version, customer authorization, and proposed transaction	Order, credit, or service change; incorrect commitment, privacy leakage, or poor handoff	Deterministic eligibility policy, preview, user confirmation, receipt, escalation
Software engineering	Issue contract, repository commit, patch, test record, and unresolved obligation list	Reviewable code change; insecure or incomplete patch and hidden regression	Isolated build and tests, static analysis, diff approval, code owner review
IT and security operations	Incident state, telemetry references, runbook version, change plan, and rollback point	Bounded remediation; disruption, credential misuse, or action on adversarial telemetry	Read-only diagnosis, simulation or canary, approval, postcondition, rollback and audit
Scientific research	Hypothesis, protocol, instrument state, measurement, and analysis provenance	Program, simulation, or physical experiment; invalid inference, untraceable conditions, or physical hazard	Objective score or measurement, protocol checks, safety interlock, domain expert judgment
Healthcare and legal support	Case evidence, authoritative source version, recommendation, uncertainty, and decision record	Draft, recommendation, or routed case; high-consequence factual or procedural error	Source validation, abstention rule, qualified professional authority, outcome monitoring
Personal productivity	User request, recipients, permissions, preference scope, and pending action record	Message, schedule, file, or application change; wrong recipient or overbroad access	Draft-first preview, confirmation, reversible commit, scoped and editable memory

7.1 Enterprise knowledge and research assistants

Enterprise research is not simply RAG with a longer prompt. It becomes agentic when the system must choose where to look, preserve partially resolved questions, compare incompatible sources, invoke business applications, and turn the result into a durable work product. The consequential output may be a cited analysis, a completed case file, a policy recommendation,

Table 14. Representative application evidence and the bounded claims it supports.

System or environment	Setting and design	Observable artifact or effect	Claim supported	Claim not supported
WorkArena [32]	Controlled ServiceNow-based knowledge work benchmark	Browser trajectory and task-specific application state	Enterprise software interaction under the benchmark's tasks and permissions	Reliability across live integrations, organizational policies, and user populations
TheAgentCompany [195]	Self-contained simulated software company and authored work tasks	Files, code, messages, websites, and task completion	Multi-application capability on consequential workplace-like tasks	Autonomous job performance or organizational outcomes in an operating company
SWE-agent [199] and SWE-bench [66]	Repository issue resolution system and executable benchmark	Patch, execution trace, and test outcome	Repository navigation and repair against the available tests	Complete maintainability, security, or requirements coverage beyond the oracle
OSWorld [192] and AppWorld [166]	Controlled GUI and API application environments	Trajectory plus terminal application or database state	Stateful cross-application action with executable end state checks	Full live authentication, interface drift, interruptions, and unobserved side effects
FunSearch [137]	Closed-loop program search with an objective scoring function	Candidate program and measured score	Search improvement where proposals are executable and cheaply ranked	Open-ended scientific validity when no objective evaluator is available
Support assistant field study [10]	Staggered deployment to 5,172 customer support workers	Issues resolved per hour, quality measures, and worker outcomes	Operational effects of editable AI suggestions in one organizational setting	Autonomous policy decisions, account changes, or effects without human responsibility

or a proposed update to a system of record. Each requires more than fluent synthesis: the system must show which admissible evidence supports which claim, which principal is allowed to see and use it, which uncertainty remains, and what downstream action the evidence may authorize. The enterprise contract is therefore evidence-to-artifact first and evidence-to-action only through an explicit authority boundary.

The persistent representation is a *research case* linking the request and audience to subquestions, admissible source versions, claim-level support, contradictions, coverage gaps, intermediate artifacts, tool receipts, the evolving deliverable, and its review state. Model notes remain distinct from authoritative sources, and stable identifiers let a reviewer reject one inference without discarding the entire case.

This representation clarifies when agency is warranted. A deterministic query or fixed RAG pipeline is usually preferable when a known authoritative corpus answers a narrow, read-only question and the output schema is fixed. Agentic research earns its additional cost and risk when the source path is not known in advance, evidence spans repositories or applications, contradictions require conditional follow-up, or the requested artifact has acceptance criteria

Table 15. Representative application evidence, continued: scientific loops and their observable boundaries.

System or environment	Setting and design	Observable artifact or effect	Claim supported	Claim not supported
A-Lab [159]	Seventeen-day closed-loop inorganic materials campaign	Protocol, instrument actions, measurements, and characterized material	Physical loop closure: 353 experiments and 36 of 57 targets realized in the reported apparatus	General laboratory autonomy outside its materials, equipment, and operating constraints
ChemCrow [8]	Tool-augmented chemistry tasks and reported demonstrations	Plan, domain tool calls, and task artifact	Value of chemistry-specific tools and structured execution	Safe authority over arbitrary experiments, equipment, or materials
DiscoveryWorld [63] and ScienceAgentBench [20]	Simulated discovery and curated data analysis tasks	Environment state, generated program, execution, and research artifact	Hypothesis-driven interaction and analysis where effects are observable	Scientific validity or physical safety beyond the authored tasks and evaluators
AI Scientist [91] and SciAgents [48]	Reported open-ended research workflow demonstrations	Ideas, experiments, manuscripts, or knowledge graph traces	Feasibility of joining multiple research stages in one scaffold	Artifact validity or safe open-ended operation without external expert adjudication

that must be checked. The distinction is functional rather than stylistic: multiple retrieval calls do not establish agency unless their results update persistent case state and govern subsequent choices. When the case cannot reach adequate coverage or authority, escalation or an explicitly incomplete artifact is a correct terminal state.

Workplace benchmarks expose progressively harder parts of this contract. WorkArena evaluates browser interaction with enterprise objects, while WorkArena++ expands to 682 compositional workflows designed to require planning, logical or arithmetic reasoning, retrieval, and contextual understanding [7, 32]. TheAgentCompany broadens the substrate to files, code, messages, internal websites, and coworker interaction; its NeurIPS report finds that the strongest evaluated agent completed 30% of its authored tasks [195]. These environments do not establish organizational effectiveness, but their progression is informative: as evidence and effects cross applications, a final answer ceases to be an adequate state representation. Identity, task dependencies, intermediate artifacts, and verified application state must survive tool and context boundaries.

Open web research reveals a complementary bottleneck. AssistantBench contains 214 realistic, time-consuming tasks collected from users and domain experts; no evaluated model exceeded 25 points in accuracy, and the paper reports near-zero scores for contemporary web agents [204]. The result is tied to its 2024 systems and closed-form evaluators, but its failure analysis shows that navigation, planning, and transfer of information across steps cannot be inferred from a model's closed-book knowledge. A research case should therefore retain failed queries, source coverage, extracted fields, and unresolved dependencies, rather than repeatedly reconstructing progress from the conversation transcript.

Evidence collection and report quality must also be separated. ALCE evaluates fluency, correctness, and citation quality and reports that, on ELI5, even its strongest evaluated systems lacked complete citation support half of the time [45]. DeepResearch Bench extends the target to 100

expert-authored research tasks across 22 fields and separately evaluates report quality, effective citation count, and citation accuracy [33]. The benchmarks differ in task, corpus, evaluator, and system generation, so their scores should not be compared directly. Their shared lesson is structural: displaying plausible citations does not prove coverage, and a well-supported collection of excerpts does not prove that the final synthesis follows from them. Claim-level evidence edges, contradiction state, and coverage criteria are required between retrieval and prose.

Enterprise authorization adds a boundary absent from open web benchmarks. Repository access should be resolved against the requesting principal and intended audience at retrieval time and checked again when an artifact is shared or committed. A source label generated inside the prompt cannot grant trust or permission, and a citation must not become a channel for disclosing restricted content. Action should advance through an explicit ladder: read-only discovery, internal draft, reviewed artifact, approved communication, and finally a bounded system-of-record change. Each promotion binds the exact research case version to its reviewer, audience, policy decision, and receipt; a later source or permission change reopens the affected claims and proposed actions.

Evaluation should compare deterministic lookup, fixed RAG, and agentic research under identical repositories, permissions, tasks, and budgets. Measure source admissibility, claim support, conflict and stale source handling, artifact acceptance, reviewer correction, privacy, latency, and cost. The invariant is *bounded research*: committed claims and downstream actions require admissible, version-bound evidence appropriate for the audience, while contradictions and gaps remain explicit.

7.2 Customer support and transactional workflows

Customer support becomes a transactional agent problem when conversation can change customer or business state: an order is cancelled, a credit is issued, a service visit is scheduled, or an account is modified. At that boundary, a fluent resolution is not necessarily a valid transaction. Each effect must follow from what the customer requested, what the system of record verifies, what the current policy permits, and what authority has been delegated. The architectural question is therefore not simply whether an LLM can choose a tool. It is whether conversational evidence can be converted into an authorized, reviewable, and verifiably completed effect without collapsing customer claims, business facts, policy, and permission into one prompt.

The persistent object is a *customer case*, not the transcript. It separates customer statements from authenticated account facts, current policy, eligibility, conflicts, and proposed resolution. A contemplated effect adds a versioned transaction proposal, proposal-bound confirmation, idempotency, receipts, postconditions, privacy scope, and escalation owner. Stable identifiers connect every commitment to the evidence and policy that justified it.

This representation induces a state machine: intake, identity and consent, diagnosis, policy retrieval, eligibility, proposal, preview and confirmation, commit, receipt verification, and closure or escalation. The path may move backward when the customer revises the request or new evidence invalidates an assumption. Confirmation is not a reusable conversational sentiment: it authorizes one displayed proposal against particular policy, price, account, and tool state versions. A material version change expires that authorization and returns the case to proposal or eligibility. Likewise, a successful tool response does not close the case until the system of record exhibits the intended postcondition; timeouts, duplicate callbacks, and partial updates remain explicit transaction states rather than being paraphrased away.

Benchmarks expose different failures along this path. In retail and airline workflows, τ-bench joins multi-turn conversation, policy compliance, tool use, and final database state, and its pass^k analysis shows why one successful episode does not establish repeated reliability [202].

ToolSandbox adds stateful execution, implicit dependencies, and intermediate milestones; its difficult categories include state dependency and insufficient information, directly motivating explicit prerequisites and unresolved field status in the case [92]. τ^2-bench then gives both the user and the agent tools in a shared telecom environment. Moving from no-user operation to this dual control setting reduced pass[1] by 18% for GPT-4.1 and 25% for o4-mini in the reported experiment, showing that coordination over separately observed state is itself a reliability burden [5].

Business process breadth introduces another pressure. CRMArena-Pro evaluates nineteen expert-validated sales, service, and configure–price–quote tasks spanning business-to-business and business-to-customer settings. Its leading evaluated agents achieved about 58% success on single-turn tasks and about 35% on multi-turn tasks; the study also reports near-zero inherent confidentiality awareness and a task performance cost from targeted confidentiality prompting [61]. These results are not directly rankable against τ-bench, ToolSandbox, or τ^2-bench because the tasks, models, interaction protocols, and evaluators differ. Collectively, however, they locate the same systems problem: once evidence and authority are distributed across people, policies, and applications, the conversation alone is too weak a control state.

Field evidence provides a useful but narrower comparator. In a staggered deployment to 5,172 customer support workers, an AI assistant increased issues resolved per hour by 15% on average; workers remained responsible for the interaction and could edit or disregard the generated suggestions [10]. This supports the operational value of suggestion-level assistance, especially as a complement to human judgment. It does not establish that an autonomous system may safely determine eligibility or commit account changes. Evaluation and deployment should therefore compare an assistance baseline, a proposal-only agent, and a transaction-capable agent on the same cases, so that the incremental value and risk of delegated authority are visible rather than attributed to generation quality.

Authority should rise through an explicit effect ladder: read-only explanation, draft recommendation, reversible low-impact change, customer-confirmed transaction, and regulated, high-value, or exceptional action. The model may assemble evidence and propose a transition, but deterministic policy and transaction services outside the model should enforce identity, eligibility, amount limits, separation of duties, confirmation requirements, idempotency, and rollback rules. Each rung has a narrower permitted action set and a stronger verifier. Uncertainty must lower authority: missing identity evidence, contradictory account state, an expired policy, or a mismatched confirmation yields clarification, no action, or escalation rather than a best-effort commitment.

Escalation is consequently a valid terminal transition, not a failure to contain the customer. A useful handoff is a structured transfer rather than a transcript dump: it states the customer's goal, verified facts, applicable policy, proposed or attempted actions, receipts and errors, unresolved questions, urgency, consent, and the decision required from the next owner. The same case can then survive channel changes and team boundaries without forcing the customer or employee to reconstruct the episode. Handoff completeness, customer effort, repeat contact, and time to accountable resolution are therefore part of agent quality alongside automation rate.

Evaluation should measure resolution, policy compliance, unauthorized or duplicate effects, confirmation validity, privacy, rollback, handoff, workload, latency, cost, and repeated consistency under identity, policy, state, and tool faults. The invariant is *authorized resolution*: every effect derives from authenticated case evidence, current policy, delegated authority, proposal-bound consent when required, and a verified receipt. Otherwise the system withholds, rolls back, or escalates.

7.3 Software engineering

Software engineering makes the evidence-to-action problem unusually concrete. The input is rarely a complete specification: an issue, stack trace, feature request, or review comment provides partial evidence about an intended change. The consequential output is not generated code but a new repository state that may be reviewed, merged, deployed, and depended upon. A patch is therefore valid only when it is tied to a pinned codebase, an explicit change contract, observable failure and success conditions, and authority over the branch or release target. Executability makes verification possible, but it does not make correctness automatic; tests observe selected behavior and cannot by themselves supply missing requirements, architectural judgment, or ownership.

The persistent *change case* binds request and acceptance criteria to a pinned repository, smallest failing reproducer, observed code and test evidence, causal hypotheses, affected interfaces, patch versions, checks, unresolved obligations, review decisions, and any deployment or rollback receipt. Agent hypotheses remain distinct from repository facts, and every observation is version-bound.

This object supports a disciplined state machine: clarify, check out a pinned revision, reproduce, localize, form and test a causal hypothesis, propose a patch, run the narrowest decisive check, expand to regression and nonfunctional checks, request review, commit, and only then stage or deploy. A failed transition returns evidence to the case rather than encouraging unrecorded trial and error. The minimal reproducer is especially important: without one, a passing post-change test cannot show that the patch addressed the reported failure. The case should also separate existing tests, agent-authored tests, and independent evaluation tests. Allowing the same model to infer the requirement, generate the patch, and define the only oracle creates circular evidence even when every test is green.

Repository benchmarks isolate successive parts of this contract. SWE-bench pairs real GitHub issues with repository snapshots and executable tests, establishing patch generation against a fixed task and oracle [66]. SWE-agent shows that the agent–computer interface–how the system searches, edits, and observes the environment–materially changes results [199]. Agentless supplies an important simpler baseline: its constrained localization, repair, and patch validation pipeline was competitive with substantially more autonomous systems on SWE-bench Lite and also exposed problems in issue descriptions and tests [189]. The comparison does not show that agency is unnecessary in general. It shows that adaptive tool choice must earn its complexity against a staged, inspectable workflow on the same repository tasks, models, and budgets.

Broader benchmarks change what the case must preserve. SWE-Lancer contains more than 1,400 paid freelance tasks worth approximately $1 million in aggregate, spanning small bug fixes, feature implementations, and engineering management choices; its individual contributor tasks use end-to-end tests independently checked by experienced engineers [107]. SWE-EVO moves from isolated issues to 48 evolution tasks across seven mature projects, averaging 21 changed files and 874 tests per task. In its reported comparison, GPT-5 with OpenHands resolved 21% of these evolution tasks versus 65% on the single-issue SWE-bench Verified setting [164]. The task sets and scores are not directly rankable, but the widening gap is informative: long-horizon work requires the change case to preserve cross-file obligations, intermediate compatibility decisions, and partial progress that a final patch-and-pass label hides.

Executable tests are consequently evidence, not an infallible specification. An OpenAI audit examined 138 SWE-bench Verified tasks that o3 did not solve consistently and judged 59.4% to contain material test design or problem description issues, including tests that were narrower or wider than the stated task [122]. This targeted audit does not invalidate every benchmark result, and its denominator is not the full suite. It demonstrates why a merge decision cannot reduce to a hidden-test bit: reviewers need the issue-to-acceptance mapping, the behavior exercised by

each test, compatibility and security checks, and explicit residual risk. Independent validation should consume the proposed diff and evidence bundle; it should not silently rewrite the patch, relax the acceptance criteria, or treat model confidence as proof.

Field evidence again supports assistance more strongly than autonomous change authority. Across randomized trials at Microsoft, Accenture, and an anonymous Fortune 100 company, access to an AI coding assistant increased completed tasks by 26.08% in the pooled estimate over 4,867 developers, with substantial uncertainty and variation across experiments [23]. Developers and organizational controls still owned review, integration, and release. A matched study of more than 100,000 GitHub developers reports that activity gains associated with autonomous coding agents attenuated markedly from commits to projects and releases [28]. The designs estimate different quantities and should not be compared as a leaderboard. Together they motivate measuring accepted and shipped outcomes, review burden, and downstream defects rather than equating generated code or commit volume with delivered value.

Authority should rise through a software effect ladder: read-only diagnosis, localized recommendation, draft diff, tested pull request, approved merge, staged deployment, and production release. Repository and infrastructure services–not the language model–should enforce branch protection, code ownership, credential scope, required checks, deployment policy, and rollback. Execution belongs in an isolated environment with restricted network access, ephemeral credentials, resource limits, and explicit controls on generated scripts and dependencies. High-risk migrations, authentication changes, security controls, and irreversible data operations require stronger independent review and rehearsal. When the case lacks a reproducer, acceptance criterion, required secret, owner, or safe test environment, clarification or handoff is the correct outcome.

Evaluation should compare an assistant, a fixed repair pipeline, and an adaptive agent under identical issues, revisions, tools, models, and budgets. Measure reproduction, localization, acceptance coverage, regressions, security, unauthorized changes, review burden, merge or rollback, escaped defects, latency, and cost. The invariant is *evidence-complete change*: merge or deployment requires a pinned request and repository, reproduced behavior, explicit criteria, diff, independent checks, accountable approval, and receipt.

7.4 Computer use and personal productivity agents

Computer use becomes agentic when a language model can alter a user's digital environment rather than merely explain what to click. Email, calendars, documents, browsers, and files make this capability immediately useful, but they also expose the distinction that governs the broader survey: fluency at a screen is not evidence that the resulting effect is correct. Intent, identity, application state, recipients, permissions, and consequences are distributed across interfaces and services. A credible productivity agent must therefore transform a request and heterogeneous observations into an explicit state that can support bounded action; a transcript and the pixels currently visible on screen are insufficient substitutes.

The persistent *personal workflow case* binds the user's goal and deadline to principal, account, application, authoritative service state, versioned source objects, recipients, visibility, plan, and unresolved conflicts. Preferences retain source, scope, expiry, and editability: they may guide a draft but cannot authorize a new recipient, disclosure, purchase, or changed fare.

Pending actions carry typed arguments, preconditions, impact, reversibility, idempotency, expiry, and required confirmation. Exact previews bind approval to attempts, receipts, and post-conditions. Cursor position is not identity, focus is not authority, webpage text is not a user instruction, and visual success is not proof of backend state.

The resulting workflow is a state machine: intake and clarification; identity and permission

inspection; state gathering; planning; preparation of a private draft; preview and confirmation; commit; authoritative verification and reconciliation; and closure, rollback, or escalation. Before commit, the agent re-queries material state because a recipient, event, file version, price, login, or sharing policy may have changed while it was reasoning. Confirmation is version-bound: it expires if the content, recipient, time, application state, or consequence shown to the user changes. After commit, the system verifies a receipt and the relevant service state instead of inferring success from a disappearing button or a plausible confirmation page.

Existing benchmarks illuminate different parts of this problem rather than forming a directly comparable leaderboard. WebArena evaluates end-to-end tasks on functional websites and reported a 14.41% success rate for its strongest GPT-4-based agent against 78.24% for humans [219]. OSWorld spans 369 tasks across real desktop operating systems and applications; its initial evaluation reported 12.24% for the best model compared with 72.36% for humans [192]. AndroidWorld adds 116 dynamically instantiated tasks across 20 Android applications and reported 30.6% for its strongest evaluated agent, while showing sensitivity to task variation and weaker transfer across platforms [135]. AppWorld instead exposes 457 APIs across nine simulated everyday applications and evaluates 750 tasks with state-based tests that can recognize multiple valid solutions and detect collateral changes [166]. The substrates, task distributions, evaluators, models, and dates differ, so their percentages should not be ranked. Together they show why both interaction correctness and resulting application state belong in the evidence chain.

Longer workflows make persistent state more consequential. OSWorld 2.0 contains 108 workflows whose median human completion time is approximately 1.6 hours; Claude Opus 4.7 averaged 318 tool calls, roughly an order of magnitude more than agents in the earlier benchmark. Yet the best reported primary binary completion rate was 20.6%, with failures involving lost constraints, missed information that arrived mid-task, ungrounded guesses, skipped verification, and hidden application state [207]. These are not only perception errors. They are failures to preserve and reconcile the workflow case as execution unfolds.

Capability and safety must also be measured separately. ClawsBench evaluates 44 tasks across stateful simulations of email, chat, calendar, documents, and storage. Across six models, four agent harnesses, and 33 conditions, fully scaffolded configurations achieved 39–64% task success while still taking unsafe actions in 7–33% of runs; the leading harnesses did not exhibit a consistent safety ordering [85]. An agent can therefore finish the requested task and still expose the wrong file, contact the wrong person, or make an unauthorized persistent change. Conversely, refusing or failing to complete a task is not evidence of principled safety. Effect correctness, policy compliance, and useful completion require separate denominators.

Field evidence currently supports assistance more strongly than autonomous effect authority. In a six-month randomized experiment across 66 firms and 7,137 knowledge workers, 80% of treated workers who used a generative AI tool spent about two fewer hours on email per week, while individual access did not produce a detectable change in the quantity or composition of their tasks [29]. The result demonstrates practical value from assistance, not that an agent should send mail, alter calendars, or share files without review. Evaluation and deployment should therefore distinguish a fixed shortcut or automation rule, a draft-oriented assistant, and an adaptive agent; time saved by composing a draft cannot be treated as evidence for broader authorization.

Authority should rise through a productivity effect ladder: observe and search, create a private draft, make a reversible local edit, communicate or change a shared object, and finally perform financial, account-level, public, or irreversible actions. Identity providers and application services–not the language model–should enforce account scope, recipient and domain restrictions, sharing visibility, spending and rate limits, confirmation requirements, idempo-

tency, and audit retention. Untrusted content inside a webpage, message, attachment, or document cannot grant permission, widen scope, or override standing user policy. Ambiguous recipients, inaccessible source objects, expired authentication, conflicting instructions, or a materially changed price or schedule require clarification or handoff.

Evaluation should compare macros, fixed workflows, draft assistants, and adaptive agents under identical tasks, permissions, and budgets. Measure end state and collateral effects, recipient and audience errors, authorization, confirmation, preference misuse, duplication, recovery, corrections, workload, latency, and cost under stale, concurrent, adversarial, and interrupted conditions. The invariant is *intent-preserving effect*: persistent effects require a current workflow case, application scope, exact preview-bound consent when required, and a verified receipt; preferences never substitute for authority.

7.5 IT operations and cybersecurity

IT operations makes the paper's central distinction unusually sharp. An agent may summarize an alert, correlate logs, retrieve a runbook, or propose a command, but none of these outputs proves that it has identified the affected system, distinguished cause from symptom, or selected a safe intervention. Operational evidence is heterogeneous, delayed, incomplete, and sometimes adversarial; operational credentials can change production at machine speed. The system must therefore transform telemetry into a bounded remediation case rather than let a plausible narrative flow directly into a shell, cloud, identity, or security control action.

The persistent *operational incident case* binds incident, service, environment, severity, impact, owner, configuration, and policy versions to provenance-bearing telemetry. Observations remain separate from causal hypotheses and falsifying tests; runbooks and threat intelligence remain evidence rather than instructions; credentials are represented only by principal, scope, and expiry.

A proposed remediation specifies target, arguments, preconditions, expected observables, blast radius, privilege, reversibility, idempotency, approval, and rollback before becoming a command. Attempts, diffs, receipts, verification queries, and observation windows append to the case. An exit code proves execution, not recovered service, removed persistence, consistent data, or tenant isolation.

The incident case moves through an explicit state machine: detect, triage, scope, hypothesize, gather and test evidence, propose, simulate or canary, approve, execute, verify, observe, and resolve, rollback, reopen, or escalate. NIST SP 800-61 Rev. 3 similarly places preparation, detection, response, and recovery within continuing cybersecurity risk management rather than treating response as a single reactive step [118]. State must be refreshed immediately before execution because deployments, traffic, credentials, incident ownership, and concurrent operator actions can change. Resolution requires both an authoritative configuration or security state check and stable service postconditions over a declared observation window; a dashboard that briefly turns green is not closure.

Operational benchmarks cover complementary slices of this contract. AIOpsLab deploys microservice environments, injects faults, generates workloads, exports telemetry, and exposes an agent–cloud interface, enabling separate assessment of detection, localization, diagnosis, and mitigation under controlled cloud failures [18]. ITBench broadens the application scope to 94 scenarios derived for site reliability, compliance and security, and financial operations work. In its reported evaluation, agents resolved 13.8% of SRE scenarios, 25.2% of compliance and security scenarios, and none of the FinOps scenarios [64]. These figures are not directly comparable with AIOpsLab because the tasks, environments, agents, and success rules differ. Their shared lesson is narrower: recognizing a fault or producing a technically relevant step does not establish end-to-end resolution.

Open-ended security investigation exposes an even larger evidence coverage problem. The Cyber Defense Benchmark places agents over 75,000–135,000 Windows event records per episode and asks them to hunt across 26 campaigns covering 105 recorded attack procedures. The strongest of five evaluated models flagged only 3.8% of malicious events on average, and no run found every malicious event [22]. SecRespond begins after compromise: 10 ranges span four entry point types, 21 MITRE ATT&CK techniques, and five operating systems, and agents must analyze forensic disk state, reconstruct intrusions, and prepare remediation. Across 23 model configurations, no model completed detection and remediation on any range; agents more readily surfaced alert-exposed problems than silent persistence or a complete verified response [175]. The suites use different artifacts and graders, but both show why an incident case needs explicit coverage obligations and unresolved hypotheses instead of treating the first plausible finding as the incident.

The evidence channel is itself an attack surface. Logs, tickets, webpages, emails, threat reports, package metadata, and even retrieved runbooks can contain attacker-controlled text. Agent Security Bench spans ten scenarios, more than 400 tools, and attacks against prompts, tool use, and memory; its highest reported average attack success rate exceeded 84.3% in the evaluated configurations [212]. AgentDojo separately couples 97 benign tasks to 629 security test cases involving untrusted tool data [25]. Neither benchmark estimates the probability of compromise in a particular production operation. They demonstrate that delimiter prompts and model instructions are not authority boundaries. Provenance and trust labels must survive interpretation, and untrusted content must be unable to widen capabilities, select a privileged target, alter an approved change, or suppress required verification.

The current evidence base remains benchmark-heavy. Fault-injected environments, frozen forensic images, and authored adversarial cases reveal failure modes, but they do not establish lower outage duration, fewer false containments, or acceptable operator workload in deployed organizations. A production claim requires field evidence that binds the exact agent configuration and authority level to realized service and security outcomes. Until such evidence is available, evaluation should distinguish read-only triage assistance, recommendation, supervised remediation, and autonomous execution rather than treating them as one capability. The safest useful system may reduce evidence gathering time while leaving the production decision with an accountable operator.

Authority should therefore rise through an operational effect ladder: read telemetry; correlate and recommend; prepare a change and rollback plan; execute in a sandbox or replay; run a narrow canary; perform a reversible, resource-scoped production change; and only then consider destructive, cross-tenant, identity, network, or data plane actions. Infrastructure and policy services–not the language model–should enforce environment and tenant scope, role-based access, separation of duties, change freezes, concurrency controls, rate and resource limits, break-glass procedure, approval, and credential expiry. High-impact actions require an independent reviewer and a tested rollback path. Missing ownership, ambiguous targets, incomplete telemetry, unavailable verification, scope growth, or conflicting operator changes should halt execution and trigger clarification or escalation.

Evaluation should compare fixed automation, an operations copilot, and an adaptive agent under identical incidents, telemetry, privileges, and budgets. Measure detection, diagnosis, containment errors, recovery time, service loss, security restoration, collateral effects, privilege use, rollback, recurrence, workload, latency, and cost under telemetry, concurrency, credential, command, and rollback faults. The invariant is *verified bounded remediation*: production mutation requires versioned evidence for target, preconditions, authority, effect, blast radius, and recovery, and closes only on authoritative postconditions.

7.6 Scientific discovery and autonomous laboratories

Scientific agency begins when a system can choose the next analysis or experiment, not when it can write a plausible hypothesis. The distinction matters because a fluent explanation, an improved objective value, a completed protocol, and a scientifically supported claim are different artifacts. Scientific evidence carries units, uncertainty, conditions, sample identity, instrument state, and alternative explanations. A credible agent must preserve those relationships while converting prior evidence into a bounded experimental action; otherwise an attractive narrative can outrun both the measurement and the domain in which it is valid.

The persistent *scientific investigation case* links the question, competing hypotheses, predicted observations, and claim scope to versioned literature, data, simulations, prior experiments, samples, custody, instruments, and conditions. Observations remain distinct from derived quantities and interpretations so a generated summary cannot become experimental fact.

Each experiment is a typed protocol binding variables, controls, units, samples, calibration, hazards, falsifying outcomes, stopping, analysis, instrument state, material, authority, and abort conditions. Execution appends commands and receipts; measurement appends raw data, calibration, uncertainty, and quality control; analysis appends code, artifacts, and claim status changes. A transcript cannot safely carry this state.

The investigation proceeds as a state machine: frame the question, assemble and assess evidence, formulate competing hypotheses, design the protocol, review feasibility and hazards, simulate or dry-run where possible, authorize, execute, acquire measurements, perform quality control, analyze, update the hypotheses, and close, revise, or escalate. Failed, null, censored, and ambiguous outcomes remain first-class states. The system rechecks sample, instrument, material inventory, calibration, and safety state immediately before a physical action. It does not start the next experiment merely because the last command returned successfully, and it does not close a claim until the measurements and analysis satisfy the protocol's declared decision conditions.

FunSearch illustrates the unusually favorable case in which candidate actions and evaluation can be tightly specified. It combines an LLM that proposes programs with a systematic evaluator that executes and scores them, retaining high-performing programs to guide search. The reported system discovered improved constructions for the cap set problem and new online bin packing heuristics [137]. This is strong evidence that generative search can produce verifiable advances when the candidate is executable and the property of interest is objective and comparatively cheap to test. It does not imply that every scientific objective can be reduced to the evaluator: the score certifies the encoded property under the tested conditions, not every interpretation, downstream consequence, or unrepresented constraint.

Controlled scientific agent benchmarks expose different gaps. ScienceAgentBench derives 102 data analysis tasks from 44 peer-reviewed papers in four disciplines, validated by nine subject-matter experts, and evaluates executable Python artifacts. With three attempts per task, its best reported agent solved 32.4% independently and 34.3% with expert-provided knowledge [20]. DiscoveryWorld instead requires agents to form hypotheses, manipulate a simulated environment, run experiments, and explain discoveries across 24 tasks, three difficulty levels, and parametric variants; it scores task completion, procedural progress, and explanatory knowledge [63]. Their scores are not directly comparable: one tests data-driven coding against task-specific outputs, whereas the other tests long-horizon discovery in a low-fidelity fictional world. Together they show why end-to-end claims should be decomposed into evidence acquisition, experimental action, analysis, and explanation.

Physical closed loops provide stronger evidence about action but remain bounded by their apparatus and measurement contract. The A-Lab combined literature-derived synthesis recipes,

active learning, robotics, and X-ray diffraction in 353 experiments over 17 days. In the corrected record, manual reanalysis confirmed 36 of 57 target materials while four earlier reported successes remained inconclusive from diffraction alone [159, 160]. The correction also clarified that "new" referred to targets new to the prediction platform rather than necessarily new to science. This history does not erase the demonstrated autonomous synthesis loop. It makes the survey's point concrete: automated classification can guide the next experiment, while the final scientific claim must retain measurement ambiguity, human reanalysis, and its precise novelty scope.

Coscientist connects web and documentation search, code execution, and laboratory APIs, and was demonstrated across six task classes including instrument control and optimization of palladium-catalysed cross-couplings [6]. ChemCrow integrates 18 expert-designed chemistry tools, planned and executed syntheses of an insect repellent and three organocatalysts, and guided a chromophore experiment [8]. In ChemCrow's comparison, expert chemists favored the tool-augmented system on chemical correctness and task completion, whereas a GPT-4 evaluator favored fluent but factually weaker baseline answers. These demonstrations establish that specialized tools and instrumentation can ground parts of chemical work; they also show why another unconstrained language model is not an adequate substitute for domain measurements and expert judgment.

More open-ended systems, including AI Scientist and SciAgents, explore automated experiment cycles, manuscript production, hypothesis generation, and multi-agent reasoning [48, 91]. Their artifacts can be useful proposals, but proposal volume, simulated novelty, or language model review does not establish a scientific result. The evidence base therefore supports a graded claim: agents can search, analyze, optimize, and sometimes operate a tightly bounded experimental loop, while general autonomous discovery across changing instruments, hazards, evidence standards, and scientific communities remains unproven.

Authority should rise through a scientific effect ladder: retrieve and organize literature; analyze an existing dataset; propose a hypothesis; prepare code or a protocol; run a simulation; perform a dry-run or low-risk instrument check; execute a bounded experiment; and only then approach actions involving hazardous chemicals, biological agents, high energy, expensive or scarce samples, or irreversible transformations. Laboratory scheduling, inventory, instrument controllers, and safety systems–not the language model–should enforce material identity, facility access, parameter envelopes, interlocks, waste and exposure limits, resource budgets, and emergency stops. Independent review is required when the hazard, uncertainty, cost, or scientific consequence exceeds the case's approved scope.

Evaluation should compare assistants, fixed experimental pipelines, and adaptive agents under identical questions, evidence, apparatus, materials, and budgets. Measure hypothesis discrimination, protocol and control validity, unit and sample integrity, calibration, safety, measurement quality, uncertainty, claim support, expert correction, time, and cost under conflicting, mislabeled, drifting, missing, and ambiguous conditions. The invariant is *measurement-bounded discovery*: action requires a complete protocol, apparatus state, hazards, authority, controls, and abort path; claims cannot exceed identified measurements and quality checks.

7.7 High-stakes professional support

High-stakes professional support is not a single application category; it is a common transition from evidence to an institutionally consequential decision. The same generated paragraph can be a harmless draft, a clinician's working hypothesis, a legal position, an investment recommendation, or the stated reason for denying a public benefit. A system becomes materially agentic when it selects evidence, advances a case, or invokes a tool that changes another person's care, rights, money, or access to services. The governing question is therefore not whether the out-

put sounds expert, but whether every case transition remains evidentially supported, within delegated authority, reviewable, and contestable.

The shared *professional case record* separates assertions, verified and contested facts, model suggestions, professional judgments, authorized decisions, and effects while preserving provenance, applicability, confidentiality, consent, deadlines, uncertainty, reasons, and review rights. Clinical, legal, financial, and public service cases extend this grammar with domain identities, rules, evidence, constraints, and accountable professionals. Each proposed commitment binds evidence, applicable standard, uncertainty, harms, decision rights, reviewer, executable action, and correction or appeal. Missing identity, consent, authority, current rules, material evidence, or a qualified reviewer is a stop state.

Healthcare evidence illustrates both promise and claim limits. AMIE was compared with 20 primary care physicians in a randomized, double-blind crossover study using 159 simulated, text-based diagnostic consultations. Specialist physicians rated AMIE superior on 30 of 32 evaluation axes and patient actors on 25 of 26 axes [167]. The study required active information gathering rather than diagnosis from a fixed vignette, but its authors explicitly caution that the synchronous text setting was not representative of usual clinical practice. MDAgents dynamically selected solo or group collaboration and achieved the best result on seven of ten medical benchmarks, with gains of up to 4.2% over prior best methods [71]. These studies support narrower claims about simulated dialogue, medical knowledge, and benchmark reasoning. They do not establish patient outcomes, workflow safety, or authority for an agent to diagnose, prescribe, order, or communicate a final care decision without a responsible clinician and a current clinical record.

Legal evidence exposes a similar progression. LegalBench contains 162 expert-constructed tasks across six forms of legal reasoning and evaluated 20 models [54]. LegalAgentBench adds 300 multi-hop and writing tasks, 17 corpora, and 37 retrieval, database, and calculation tools in the Chinese legal domain, while recording progress through intermediate tool use [82]. The latter is closer to agentic research, but a correct benchmark answer is not a valid position in a live matter with changing law, disputed facts, privilege, deadlines, and client duties. In a preregistered evaluation of commercial retrieval-augmented legal research products, Magesh et al. found hallucinations in 17–33% of responses, despite rates below those of a general-purpose model [98]. A linked source is therefore not enough: the case must bind each proposition to the authority's holding, court, jurisdiction, date, subsequent treatment, and applicability to the client's facts.

Financial benchmarks likewise distinguish analytic capability from delegated fiduciary or transactional authority. FinBen spans 42 datasets and 24 tasks across eight areas, including information extraction, question answering, risk management, forecasting, and decision-making. Its evaluation of 21 models found stronger performance on extraction and textual analysis than on advanced reasoning, generation, and forecasting [191]. FinCon organizes analyst agents under a manager, updates investment beliefs through risk-controlled reflection, and evaluates historical stock trading and portfolio management trajectories against LLM, reinforcement learning, buy-and-hold, and classical portfolio baselines using return, Sharpe ratio, and drawdown [205]. Such comparisons can establish performance under a declared dataset and backtest protocol. They do not establish suitability, robustness to regime change, transaction execution, market impact, compliance, or realized client benefit. Those claims require a case record that freezes data vintage, mandate, holdings, exposure, constraints, approvals, orders, fills, and reconciliation.

Public administration supplies stronger field evidence about the consequences of weak case structure, although not from an LLM agent. The Royal Commission into Australia's Robodebt Scheme recommended documenting the sources and bases of assumptions, obtaining legal re-

view of end-to-end data exchanges, retaining vulnerability information on recipient files, providing a clear path to review, disclosing automated decision-making, and making business rules available for independent scrutiny [139]. This is important counterevidence: automation does not become legitimate because a person can inspect its final message. The evidentiary basis, applicable law, individual circumstances, reasons, notice, and route to challenge must travel with each decision, and recovery activity must pause when the underlying case is disputed or under review.

Authority should rise through a professional effect ladder: retrieve and organize sources; identify missing evidence; prepare a chronology, calculation, or draft; generate options and counterarguments; recommend an action; obtain accountable professional approval; and only then perform a narrow, structured, reversible administrative step. Diagnosis and treatment, legal filing or representation, investment and credit decisions, benefit eligibility, debt recovery, and enforcement should not be delegated to an unconstrained language model. Clinical, matter management, trading, case management, and policy systems–not the model–should enforce identity and consent, licensure and privilege, jurisdiction, suitability, position and loss limits, separation of duties, deadlines, notice and appeal, approval scope, and audit retention.

Evaluation should compare existing professional workflows, drafting copilots, and adaptive agents under identical cases, evidence, authority, time, and budgets. Measure evidence and attribution, currency, uncertainty, missed conflicts, professional correction, workload, disparate effects, inappropriate decisions, privacy, notice and appeal, outcomes, recovery, latency, and cost under identity, authority, policy, consent, and adversarial failures. The invariant is *professionally accountable case progression*: consequential action requires a current case, applicable rules, decision rights, a responsible professional, review path, and expected effect; fluency never creates professional authority.

7.8 Cross-domain deployment patterns

Across domains, the nouns change but the control problem does not. A research assistant works with claims and sources; a support agent with customers, policies, and transactions; a coding agent with requirements, diffs, and tests; an operations agent with incidents and infrastructure; a laboratory agent with hypotheses, samples, and measurements; and a professional system with cases, duties, and decisions. In each setting, reliability depends on preserving the relation between what was observed, what was inferred, what was proposed, what was authorized, what was executed, and what happened afterward. Long-horizon agency is therefore not simply a longer conversation or a larger number of tool calls. It is a sequence of durable, inspectable state transitions.

The shared grammar is small: evidence items preserve provenance and applicability; case state preserves facts, disputes, obligations, and authority; proposals bind effects to evidence and validation; commitments freeze target, scope, approval, and recovery; receipts record authoritative postconditions and collateral effects. Summaries may be regenerated, but admitted evidence, approvals, and receipts cannot be silently rewritten.

The shared lifecycle is evidence acquisition, case update, bounded proposal, validation, authority, controlled commitment, and reconciliation. Models interpret ambiguity and select among permitted steps; deterministic services and accountable people control admission, authority, and effects. Missing evidence or authority produces an unresolved or escalated state. Six deductions follow:

1. **Begin with the narrowest architecture.** Fixed retrieval and workflows should handle specifiable transitions. Agentless remained competitive on SWE-bench Lite, while field support evidence showed value from suggestions without transaction authority [10, 189].

Compare fixed and adaptive designs under the same model, tools, and budget.

2. **Center the authoritative artifact.** Cases close against database state, accepted diffs, application state, incident postconditions, measurements, or accountable decisions. AppWorld and the A-Lab correction show why narrative cannot substitute for state or measurement [160, 166].

3. **Separate proposal from commitment.** Proposals are editable; commitments freeze an exact action, target, authority, validity window, and recovery path. ClawsBench and workplace studies show that completion, usefulness, and permission are distinct [10, 29, 85].

4. **Use domain-specific validators.** FunSearch benefits from executable scoring, whereas ChemCrow and legal RAG evidence expose limits of fluent judges and linked citations [8, 98, 137]. Validators judge frozen proposals without rewriting criteria or enlarging authority.

5. **Preserve provenance.** ALCE exposes incomplete citation support, and Agent Security Bench shows that evidence channels can carry attacks [45, 212]. Evidence may inform interpretation but cannot grant capability, change approval, or suppress verification.

6. **Evaluate the human workflow.** Compare manual work, fixed automation, assistants, proposal-only agents, and action-capable agents under common authority. Field productivity evidence and AMIE's simulated results support different claims [10, 23, 167]; measure accepted outcomes, correction, workload, harms, and accountability.

These patterns form three planes: evidence stores the case, decision interprets ambiguity and proposes permitted steps, and effect enforces identity, policy, approval, recovery, and authoritative receipts. Autonomy should therefore be reported as a contract over admitted evidence, selectable transitions, maximum effect, validation, recovery, and accountable authority rather than as a scalar.

The cross-domain invariant is *representation-bounded action*: proposals require typed evidence and criteria; commitments freeze target, preconditions, authority, limits, and recovery; cases close only on authoritative postconditions. Missing or contradictory elements trigger retrieval, revision, withholding, rollback, reopening, or escalation. The evidence supports a stronger conclusion than "more autonomy is better": agentic value comes from turning heterogeneous evidence into bounded, inspectable effects.

8 Future Directions and Open Problems

The next research frontier is not simply a more capable base model, but a system that can preserve evidence, choose a justified transition, obtain bounded authority, act through changing tools, and verify the effect from authoritative state. The open problems below follow that lifecycle: grounding, long-horizon control, memory, delegation, evaluation, adaptive security, human collaboration, and resource use. Progress should be stated against the object or transition improved (claim, case, proposal, commitment, receipt, or recovery) rather than attributed vaguely to "agent intelligence."

8.1 Grounding as a verifiable contract

Grounding should connect four contracts: *evidence admission* records whether a source may enter the case; *claim support* links propositions to supporting, contradicting, or indeterminate evidence; *action justification* binds a proposed effect to intent, policy, authority, and current

preconditions; and *outcome reconciliation* compares authoritative state with the intended postcondition. Retrieval relevance alone satisfies none of them.

Existing factuality work covers complementary parts of claim support. ALCE separates fluency, correctness, and citation quality [45]; RAGTruth annotates hallucinations in nearly 18,000 RAG responses [120]; FActScore checks atomic facts against a designated source [106]; and VeriScore distinguishes verifiable from unverifiable claims and exposes limited cross-task transfer [156]. Because their tasks and evaluators differ, the scores should not be pooled. Together they motivate a typed claim ledger that retains evidence edges, contradiction, indeterminacy, and coverage.

Action grounding adds state, authority, and time. ToolSandbox and AppWorld show that dependencies, missing information, and collateral state changes can invalidate an apparently sensible sequence [92, 166]; ClawsBench separates useful from unsafe completion, while AgentDojo shows that observations may contain adversarial instructions [25, 85]. A valid proposal must therefore bind admitted observations, exact targets, current preconditions, policy, and decision rights, and completion must be established from an authoritative receipt rather than the model's prediction.

Evaluation should compare RAG, claim ledger, proposal-only, and action-capable systems under the same sources, tools, authority, and budget. The decisive measures are evidence and claim coverage, calibrated abstention, invalid proposals, current preconditions, verified postconditions and collateral effects, lineage through retries or delegation, reviewer corrections, latency, and cost. Stress tests should combine stale or adversarial evidence with changing state and partial execution failure.

The open question is how to make these contracts efficient without letting one model manufacture its own evidence, validation, and authority. Missing support, authority, or postconditions should force retrieval, indeterminacy, review, withheld action, or a reopened case. The objective is not merely fewer unsupported sentences, but fewer unsupported interpretations becoming consequential transitions.

8.2 Long-horizon control and learned agent policies

Long-horizon failure is better understood as *state divergence* than as accumulated local mistakes. AgencyBench averages roughly 90 tool calls and one million tokens over 138 tasks [83]; OSWorld 2.0 reports only 20.6% binary completion for its strongest system at a 500-step limit on 108 professional workflows [207]. Although the scores are not comparable, both expose lost constraints, stale observations, missed clarification, and skipped verification. More context or more steps does not ensure that open obligations and environment state remain correct.

A long-running agent therefore needs a persistent *execution case* containing acceptance criteria, verified state, open obligations, evidence conflicts, authority, budgets, checkpoints, and receipts. FS-Researcher demonstrates the utility of a durable workspace beyond the context window [220]. LongHorizon-Harness goes further by storing task state outside the executor and admitting updates after read-only audit; it improves reported pass rates on WeaveBench, Terminal-Bench 2.1, and OSWorld 2.0 [96]. This is comparative, not universal, evidence that independently checked state can matter more than an ever-growing transcript.

Learned policies address distinct parts of the problem: SOAR learns tool consequences, SUPO jointly optimizes tool use and context compression, and AgeMem learns memory operations [84, 93, 206]. A safer separation lets the policy propose *what to try next*, while a controller owns evidence admission, invariants, authority, execution, checkpointing, and recovery. The policy may not rewrite the facts, permissions, or success criteria used to judge its transition.

Comparisons among fixed controllers, prompted agents, and learned policies should hold tools,

authority, budgets, and stopping rules constant, then measure verified progress, obligation retention, invalid transitions, recovery, postcondition closure, intervention, latency, and cost. Delayed observations, schema changes, interrupted episodes, and validator disagreement should test whether learned compression, planning, memory, or tool choices caused the gain.

The governing invariant is *checkpointed bounded control*: a learned policy proposes from current admitted state, consequential effects remain within an explicit risk and authority envelope, and progress is recorded only from authoritative receipts. The research agenda includes transition-level credit assignment, transfer across tool and policy versions, regression detection, weak-evaluator resistance, and calibrated stopping or escalation. Long-horizon autonomy means preserving justified state transitions, not merely acting for more steps.

8.3 Memory that is useful, correct, and governable

The unresolved memory problem is how to govern a stored item's *causal influence* on a later decision. A relevant item may still be false, stale, inferred, cross-user, contradicted, or out of scope. Memory is therefore a lifecycle: admit a typed record, recall it under current scope, judge its applicability, use it in a bounded transition, and confirm, supersede, revoke, or forget it after the outcome.

This lifecycle requires a versioned *memory record* with provenance, time, scope, support, contradiction, and revocation state, plus a *memory use edge* binding that exact version to a claim, parameter, plan step, or proposal. Text2Mem compiles memory instructions into schema-based contracts [178], while Hindsight separates facts, experiences, observations, and opinions [76]. Both improve inspectability, but retrieval performance alone does not establish permissible or causal use in a consequential action.

Benchmarks expose successive gaps. MemBench separates factual from reflective memory and participation from observation [163]; Mem2ActBench shows that seven frameworks remain weak at using memory to choose tools and ground parameters [147]. Experience-following experiments reveal error propagation through similar retrieved episodes [193], while SteeM shows the tension between useful continuity and anchoring [62]. The tasks differ, but jointly show that retention, retrieval, application, and reliance control are distinct capabilities.

Learned policies make causal attribution more urgent. AgeMem learns when to store, retrieve, update, summarize, or discard [206]; Fine-Mem assigns credit to operations that produced downstream evidence and reports gains on Memalpha and MemoryAgentBench [94]. Retrieval lineage is still not causality: a record may be unused, misleading, or correlated with a favorable evaluator. Interventions that remove, replace, rescope, or revoke individual records should measure the resulting change in claims, proposals, actions, and postconditions, including negative utility.

Memory evaluations should hold the task, model, tools, authority, and budget constant while comparing transcripts, similarity retrieval, structured memory, learned policies, and governed ledgers. They should measure applicability, conflict and revocation handling, correct use in claims and tool arguments, calibrated reliance, collateral effects, cross-user leakage, reviewer burden, latency, and cost under stale, contradictory, poisoned, or deleted records.

The governing invariant is *bounded mnemonic influence*: memory affects a transition only through an inspectable edge from a current, scoped, supported version; it cannot grant authority, certify itself, or survive correction through an untracked derivative. Useful memory is persistent evidence whose influence on bounded action can be inspected, challenged, and revised.

8.4 Interoperability with trust boundaries

Dependable interoperability must preserve semantics and responsibility across a trust boundary. MCP standardizes host–tool exchange and A2A agent–agent exchange [1, 110], but a valid tool description is not permission, a task update is not verified progress, and a successful response is not proof of the intended effect. Standardizing the wire lowers integration cost without removing the local obligation to interpret, authorize, validate, and reconcile what crosses it.

Contract Net and FIPA already formalized task allocation and message semantics [42, 155]; modern systems add dynamic discovery, natural language capability claims, black-box services, and consequential tools. MAST identifies 14 failure modes in more than 1,600 multi-agent traces [12], while MCP Security Bench finds that stronger instruction following does not imply safer tool composition [211]. Although the studies differ, both show why conformant messages and successful components can still produce an invalid global transition.

A delegation should therefore cross the boundary as a typed *delegation envelope*: principal, purpose, inputs and lineage, data scope, capability and effect ceiling, policy, expiry, output contract, and recovery requirements. A linked *result envelope* should bind the producer and version to inputs, transformations, claims, validation, effects, receipts, and open obligations. Together they form a responsibility graph across request, evidence, proposal, authorization, execution, and verification.

Trust admission is a gate, not an endpoint. Attested Tool-Server Admission adds signed clearance, deny-by-default allowlists, and tamper-evident logging to MCP [103], but cannot establish that a delegated purpose or returned effect is valid for the current case. Discovery should resolve to a locally reviewed, version-pinned manifest; authority and data scope should attenuate at each hop; returned artifacts should re-enter as untrusted evidence; and revocation or schema change should invalidate dependent permissions.

Evaluation should compare raw messages, schema-valid exchange, reviewed manifests, and full delegation contracts under identical agents, tasks, policies, and budgets. It should measure semantic conformance, lineage, authority attenuation, invalid transitions, duplicate effects, verified postconditions, revocation, recovery, accountability, latency, and cost under version drift, replay, substitution, partial artifacts, and false success reports.

The governing invariant is *responsibility-preserving delegation*: every hop retains purpose, disclosed evidence, delegated authority, and the owners of proposal, authorization, commitment, and verification. A downstream hop may narrow purpose, data, duration, or authority, but not broaden them; a returned message may contribute evidence or a proposal, but not silently become permission or proof.

8.5 Evaluation beyond static leaderboards

A leaderboard result is a historical join among a benchmark case version, a system manifest, episode records, and an evaluator version. If any element is missing or silently updated, an apparent gain may instead reflect a different scaffold, budget, environment, exposure state, or oracle. Evaluation beyond static leaderboards means maintaining the evidence behind the claim while both system and instrument change.

AgencyBench stresses obligation retention over long, tool-intensive episodes [83]; Gaia2 uses asynchronous environments and write action verifiers, revealing trade-offs among accuracy, timeliness, and cost [43]. Their scores are not comparable, but both make a final answer insufficient: the evaluator must know the observed state, attempted transition, and accepted state change.

The instrument itself also fails. Dynamic construction may reduce some contamination pathways but lacks standardized validity criteria [15]; search-time leakage can inflate research agent results [181]; and the continuing SWE-bench Verified audit shows that curated tasks and executable tests still require inspection of descriptions, coverage, and exposure [66, 122]. Recency alone does not establish validity.

A future benchmark should treat each task as a persistent *benchmark case* binding provenance and time, initial state, constraints, permissions, resource policy, acceptance contract, oracle, adjudication, exposure, health checks, and retirement state. Construction, validation, release, challenge, revision, and retirement should be explicit; corrections must create linked versions rather than silently mixing old and new scores.

No renewal strategy serves every claim: audited cores support longitudinal comparison, held-out waves limit exposure, live environments test changing state, and deployment studies test utility. TRACE evolves tasks while retaining construction trajectories and multi-level validation [55], but generated difficulty is not automatically realistic difficulty. Portfolio results should remain partitioned by instrument and claim.

The same configured agent should be compared across versioned task waves, tool revisions, policy changes, asynchronous events, and controlled exposure; benchmarks should be challenged with planted defects, collateral effects, oracle omissions, and environment failures. Measures should cover verified transitions, lineage, oracle error, exposure, environment health, repeated success, recovery, rank stability, adjudication, latency, and cost. Generalization claims must remain bounded to sampled variations.

The governing invariant is *versioned evaluative traceability*: every score resolves to immutable cases, a complete system and resource manifest, observed episodes and effects, and evaluator judgments. A leaderboard is then a temporary view over an evaluation ledger, and each result a scoped claim about a named system, instrument, environment, budget, and time.

8.6 Security for adaptive and connected agents

Section 6 developed security boundaries for evidence, memory, tools, authority, effects, and release. The unresolved problem is whether those boundaries remain valid when the agent changes itself or its composition. A configuration that satisfied an assurance claim at time t may violate it at $t + 1$ after a memory consolidation, skill synthesis, policy update, new connector, delegated agent, altered verifier, or learned stopping rule. Adaptation is therefore not simply more data entering the context. It is a transition that can change which evidence is trusted, which behavior is selected, which capabilities are reachable, and which observations count as success.

These changes occur on four coupled planes. The *evidence plane* contains memories, retrieved artifacts, summaries, and learned case state; the *behavior plane* contains prompts, procedures, skills, routing rules, and model or policy parameters; the *capability plane* contains tools, credentials, remote agents, and delegated decision rights; and the *assurance plane* contains policies, monitors, verifiers, review triggers, and rollback criteria. A low-trust trajectory promoted from evidence into a reusable skill crosses from the first plane into the second. If that skill selects a newly installed tool, it reaches the third; if the same learned process also revises the judge that evaluates its outcomes, it weakens the fourth. Security research must preserve these distinctions because provenance on an observation does not authorize a behavior, a useful behavior does not grant a capability, and apparent success does not validate the monitor that declared it.

Recent evidence shows different ways that influence can persist across these planes. BackdoorAgent instruments planning, memory, and tool use attacks across four agent applications

and reports, for one GPT-based backbone, trigger persistence of 43.58%, 77.97%, and 60.28% at the respective stages [39]. SkillSec-Eval broadens the unit of analysis to repository admission, semantic retrieval, planner selection, execution, and skill evolution across 327 real-world skills [4]. A very recent preprint on PoisonedEvolution targets the promotion boundary directly: with 10% attacker support, it reports target behavior embedding in 546 of 600 SkillClaw trials and 369 of 600 trials in a structurally different pipeline [13]. Reward misspecification creates a separate endogenous failure: experiments in text-adapted AI Safety Gridworlds find that direct optimization can widen the gap between observed and hidden reward across tested 1.5B–14B language model agents [11]. These studies use different threats, systems, and metrics and should not be pooled. Together they show that cross-stage propagation, poisoned experience promotion, vulnerable skill lifecycles, and proxy optimization are distinct routes by which an apparently useful update can become persistent unsafe control.

Every behavior-affecting change should consequently be represented as a typed *adaptation transaction*. The record should identify the target artifact and exact before–after versions; source trajectories, memories, feedback, reward, and contributors; trust, tenant, purpose, and data scope; the adaptation mechanism and claimed benefit; affected capabilities, policies, invariants, and assurance claims; proposed authority or information flow changes; evaluation and red-team evidence; dependency and derivative artifact links; rollout cohort and effect ceiling; approval, validity, expiry, and revocation state; monitoring thresholds; and rollback or containment procedure. Its lifecycle should distinguish *proposed*, *quarantined*, *evaluated*, *approved*, *canary*, *promoted*, *paused*, *revoked*, and *rolled back*. A model may suggest or synthesize an update, but a separate controller must decide whether that exact version may influence production behavior.

Lineage must survive promotion. If a source trajectory is later found to be adversarial, a user correction is withdrawn, or a tool receipt is invalidated, the system should identify every summary, skill, policy update, evaluator rule, and downstream action that depended on it. Deleting the source text is insufficient when its influence has already been compiled into a derivative artifact. Research is needed on dependency-aware invalidation, counterfactual influence tests, quarantine of suspect derivatives, targeted retraining or replacement, and verification that revocation has reached caches, replicas, active episodes, and delegated agents. When precise removal cannot be established, the safe response is to disable the affected pathway or narrow its authority, not to assume that an opaque update has forgotten its source.

Evaluation should compare a fixed configuration, uncontrolled online adaptation, sandboxed adaptation without promotion controls, and governed adaptation transactions under the same tasks, contributors, tools, authority, and resource budget. Measures should include benign utility gain; hazardous effect escape; update admission precision and recall; cross-stage attack transfer; source-to-derivative lineage coverage; use of revoked or stale derivatives; unauthorized capability expansion; verifier co-adaptation and common-mode failure; canary detection; rollback completeness; residual influence; recovery latency; and cost. Stress tests should mix benign and poisoned trajectories, delay outcome feedback, corrupt reward or evaluators, replay previously successful but now invalid experiences, race policy and artifact versions, revoke a source during an active episode, substitute a remote agent, and distribute a suspect skill across multiple tenants. Hidden safety objectives and independent effect checks are necessary because the adapted component cannot be the sole judge of whether its own adaptation is safe.

The governing invariant should be *non-self-authorizing adaptation*: no memory, skill, learned policy, connector, or evaluator may grant itself trust, broaden executable authority, weaken the controls that govern it, or certify its own successful promotion. Every persistent behavioral change creates a new configured system whose affected assurance claims must be reopened; influence remains bounded by recorded purpose, scope, authority, duration, and effect limits until independent evidence supports a separately authorized promotion. Adaptation can

improve how evidence is interpreted and which action is proposed, but it must not silently rewrite the rules that determine which evidence is admissible, which action is permitted, or which receipt closes the case.

8.7 Human–agent collaboration and accountability

Section 6 specified when a consequential proposal should become a review request. The deeper collaboration problem begins earlier and continues later: how should a shared case allocate evidence, expertise, decision rights, commitments, and recovery responsibilities between people and agents as the work evolves? Maximizing autonomous completion is the wrong objective when the agent lacks relevant context, the human lacks time or technical reach, or only a designated role may accept the consequence. A human being present somewhere in the workflow does not establish meaningful control. The system must represent what each participant knows, may decide, has promised to do, and remains responsible for verifying.

Recent evaluations make this distinction empirical. Collaborative Gym gives humans and agents dual control of a shared environment; in real user comparisons, the strongest collaborative agents were preferred over autonomous counterparts in 86% of travel planning cases, 74% of tabular analysis cases, and 66% of related work cases, yet communication and situational awareness failures still appeared in 65% and 40% of real condition trajectories, respectively [146]. HAS-Bench varies agency level, interaction channel, and participant policy across six domains and finds that human participation can improve both task completion and failure recovery, but that the gain depends on when, how, and by whom the input is exercised [186]. A field study of agentic customer service reaches a compatible but domain-specific result: human intervention preserved quality for technical escalations, was less effective for emotional escalations, and supported more post-escalation agent effort when it occurred earlier [180]. These results should not be pooled, but they reject "human-in-the-loop" as a sufficient system description. Timing, role, information access, and action authority are causal parts of the configuration.

Collaboration also has a jagged production frontier. In a randomized experiment with 2,234 participants producing 11,024 advertisements, human–AI teams produced 50% more advertisements per worker and higher-quality text, whereas human–human teams produced higher-quality images; greater delegation to the agent was also associated with more homogeneous outputs [67]. Conversely, 1,482 simulated shared workspace sessions show that adding relevant collaborators can lower performance when their contributions are not integrated; shared group memory and designated approval gates improved mean performance most clearly in three-person teams [72]. One study concerns real creative production and the other simulated data analysis teams, so their effect sizes are not directly comparable. Their common lesson is structural: complementary capability does not become joint performance unless responsibility and evidence are routed to the decision where they matter.

An ongoing collaboration should therefore be governed by a typed *collaboration contract*, not reconstructed from chat history. This record should bind the shared goal and acceptance criteria to participant identities and roles; evidence each participant supplied, can access, or is expected to obtain; delegated tasks and human-reserved decisions; open questions, disagreements, and uncertainty; current artifact and proposal versions; communication, clarification, handoff, takeover, and escalation triggers; deadlines and safe defaults; effect ceilings; and the owners of authorization, execution, verification, appeal, and recovery. Its lifecycle should distinguish *negotiated*, *accepted*, *active*, *waiting*, *handed off*, *proposed*, *authorized*, *committed*, *verified*, *disputed*, and *repaired*. Unlike a review request, which asks whether one exact proposal may cross a boundary, the contract preserves the evolving division of labor that produces, authorizes, and checks that proposal.

Human input must be typed as carefully as agent output. A clarification, fact attestation, pref-

erence, artifact edit, policy interpretation, one-case exception, action approval, and reusable instruction have different scopes and should create different state transitions. A person's correction of one paragraph must not silently become durable memory; approval of one action must not validate the evidence behind it or authorize a later variant; and an agent's incorporation of feedback must remain linked to the exact input and resulting artifact. The case ledger should make it possible to reconstruct who could see which evidence, who held the relevant decision right, which proposal was authorized, what effect was committed, who checked the receipt, and who can contest or repair the outcome. Explanations are then decision-specific views over this record, not substitutes for it.

Evaluation should compare autonomous execution, fixed approval gates, learned deferral, an unstructured shared workspace, and typed collaboration contracts under the same tasks, evidence, authority, models, tools, and budgets. Outcome measures should be joined with missed and unnecessary handoffs; handoff timing; clarification value; evidence and obligation transfer completeness; decision right violations; rework; takeover and recovery success; human workload and situational awareness; calibration; output diversity; appeal accessibility; latency; and cost. Stress tests should withhold expertise from one participant, introduce conflicting evidence and role changes, delay human responses, interrupt a handoff, revise an artifact after approval, make an approver unavailable, distribute knowledge across several people, and require recovery after an agent or human error. Studies should also report the participant population and role competence rather than treating a simulated user, crowd worker, domain expert, and accountable decision owner as interchangeable.

The governing invariant should be *accountable joint control*: no participant may make a commitment beyond recorded role and authority, and every transfer of work must preserve the material evidence, open obligations, decision rights, and recovery owner needed for the next participant to act. Human approval authorizes only the exact proposal reviewed; agent completion neither erases human responsibility nor manufactures it. A human is meaningfully in the loop only when the relevant evidence reaches them early enough, they possess the decision right at issue, and they can change, stop, contest, or repair the resulting transition. The research target is not maximum autonomy or maximum intervention, but a collaboration whose changing division of labor remains explicit, bounded, and auditable from evidence to effect.

8.8 Economics and sustainability

Section 5 treated resources as part of the episode record. The unresolved control problem is whether an agent can decide, before resources are spent, which additional retrieval, model call, tool, verifier, collaborator, or retry is justified by the evidence still missing and the consequence at stake. Cost is therefore not merely a bill attached to a completed trajectory. It is a constraint on which transition may be attempted next. Monetary spend, tokens, compute, wall-clock time, tool quotas, human attention, energy, and emissions belong in the same typed control state as quality, authority, and risk, while remaining distinct quantities that cannot be collapsed into one price.

Current evidence shows why raw capability comparisons are economically incomplete. AgencyBench reports averages of roughly 90 tool calls and one million tokens per task across long, feedback-rich episodes [83]. In a different setting, compute-matched evaluation of reasoning strategies finds that simpler chain-of-thought self-consistency frequently outperforms more elaborate methods and that multi-agent debate or Reflexion can become worse as additional compute is supplied [174]. These results use different tasks, models, and resource definitions and should not be compared numerically. Together they show that more steps or tokens do not constitute progress unless the extra expenditure improves a verified outcome, reduces relevant uncertainty, or lowers downstream risk.

Budget awareness is itself an agent capability, not a deployment wrapper. EcoAgent-Bench assigns priced actions and explicit budgets to 304 real-derived tasks that require agents to decide whether to use local evidence, broaden search, select a stronger model, escalate to a human, or stop on an unsupported premise. Across seven evaluated agents, tool API configurations achieved only 3.9–24.0% micro strict success and at most 7.3% economic consistency, with common failures divided between premature stopping and overspending on inexpensive cases [184]. Conversely, LLM-as-Scheduler shows that a structured gate and dynamic workflow routing can reduce tokens by 43% and end-to-end latency by more than 36%, with at most a 1.4 percentage-point accuracy reduction relative to its strong fixed workflow baseline [190]. The first study tests whether an agent chooses economically; the second tests one architecture for avoiding unnecessary workflow stages. Their combination motivates controllers that can spend selectively without interpreting a large budget as permission or a small budget as justification for unverified completion.

Sustainability requires an equally explicit measurement boundary. An initial empirical and theoretical study of web agents finds that architectural choices materially change energy and carbon estimates, that higher energy consumption does not necessarily produce better results, and that undisclosed model and process details limit credible accounting [73]. A token count alone cannot establish energy use because hardware, batching, model implementation, datacenter region, electricity mix, tool services, and time all matter. Nor does a per-episode efficiency gain establish an aggregate reduction if lower cost induces substantially more episodes. Claims about energy, carbon, or water should therefore identify the measured or estimated components, hardware and provider boundary, workload and concurrency, region and time, allocation method, uncertainty, and whether induced demand is inside the claim.

Each episode should consequently carry a versioned *resource contract*. The contract should state the goal and minimum acceptable evidence, quality, safety, and effect verification; monetary ceiling and currency; token, compute, tool, storage, latency, and human attention limits; pricing and carbon intensity sources with versions and timestamps; per-stage reservations and contingency; reusable artifact and cache policies; permitted model, tool, and human escalation tiers; marginal value and stopping rules; and the principal authorized to amend the budget. A linked *resource ledger* should record, for every transition, predicted, reserved, and realized consumption; evidence gained or uncertainty reduced; artifact and effect identifiers; retries, waste, reuse, and cache provenance; current balance; forecast cost-to-completion; and any price, latency, or emissions estimate revision. Its lifecycle should distinguish *proposed*, *admitted*, *reserved*, *spent*, *reconciled*, *reforecast*, *exhausted*, and *closed*. This makes resource exhaustion a typed state that can trigger a narrower proposal, a justified amendment, an incomplete artifact, or escalation rather than an improvised low-quality answer.

The accounting boundary must extend beyond inference. Total cost of ownership includes evidence acquisition, indexing and storage, tool and connector fees, human clarification and review, evaluation and red teaming, incident response, compensation and rollback, monitoring, benchmark refresh, policy and connector maintenance, and retirement or migration of persistent records. Savings from a cheaper model can be erased by more retries, review corrections, or escaped defects; an additional verifier can increase direct cost while reducing expected incident loss. Economic claims should therefore compare the agentic system with an appropriate deterministic workflow, fixed RAG or automation pipeline, assistive human workflow, and fully manual baseline on the same demand, acceptance criteria, authority, and accounting horizon. The relevant output is net value and a quality–cost–latency–risk frontier, not cost per model call in isolation.

Evaluation should compare fixed budgets, adaptive budgets with explicit amendment, static workflow routing, dynamic routing, and unconstrained execution under the same task distribution and system manifest. Measures should include verified effects per resource envelope;

cost and energy to accepted outcome; marginal evidence or quality gain per transition; budget forecast calibration; premature stopping and unnecessary escalation; overspend and stranded reservations; reuse validity; human time; tail latency; incident and recovery cost; maintenance burden; and the fraction of environmental impact that is measured rather than imputed. Stress tests should change model and tool prices mid-episode, exhaust a quota, delay the inexpensive model, invalidate a cached plan, raise carbon intensity, introduce an apparently cheap tool with hidden review cost, and present a rare high-risk case near budget exhaustion. Hard quality, authority, safety, and verification requirements must remain gates rather than soft terms that an optimizer can trade away.

The governing invariant should be *resource-bounded sufficiency*: every expenditure must be attributable to an authorized objective and a versioned resource contract, and every claim of savings must remain joined to the outcome quality, risk, and accounting boundary under which it was obtained. The system may route, reuse, stop, or escalate to avoid waste, but it may not economize by admitting weaker evidence, bypassing required validation, broadening authority, hiding human work, or declaring an unverified effect complete. Sustainable agency is not minimum computation at any cost; it is enough justified computation, human attention, and external service use to transform admissible evidence into a bounded and verified effect, followed by a defensible stop.

9 Conclusion

This survey has used *LLM-centered agentic systems* as the umbrella for a systems family, not a synonym for autonomous chatbots. Autoregression describes token generation; it does not specify evidence, persistent state, authority, recovery, or proof of completion. Agency emerges at the system boundary where a model adaptively helps choose the next action while tools, policies, state, evaluators, and people constrain the choice. The central transition is from generating a response to transforming evidence into a bounded, reviewable, and verified effect over time.

RAG, workflows, bounded agents, and multi-agent systems are alternative execution forms, not a maturity ladder. RAG interfaces with evidence; workflows specify predictable transformations; bounded agents select among permitted transitions when the useful next step cannot be fixed; and multi-agent systems distribute work across additional boundaries. ReAct and Toolformer show how models can interleave generation and tool interaction, but tool choice alone supplies neither durable state nor permission nor effect verification [144, 201]. A more adaptive form is justified only when it resolves uncertainty a simpler form cannot.

The forms are connected by a persistent case whose typed records survive model calls, tools, participants, and time. Evidence records support claims; scoped memories preserve experience; plans expose dependencies; proposals remain distinct from authorizations; action contracts bind permitted effects; receipts distinguish acknowledgement from verified change; and review, delegation, governance, and resource records preserve responsibility. This chain prevents recurring category errors: retrieved text is not automatically admissible evidence, memory is not permission, a plausible plan is not an authorized action, and a successful API response is not proof of the intended effect.

Comparative evaluation supports this view. GAIA, WebArena, OSWorld, SWE-bench, and AgencyBench test different combinations of information gathering, interface control, software change, state, and tools [66, 83, 104, 192, 219]; AppWorld and τ-bench add state change and repeated run checks [166, 202]. Because tasks, scaffolds, budgets, environments, and evaluators differ, scores cannot be pooled into a universal measure of agency. The common lesson is narrower: fluent behavior and component success do not establish a valid evidence-to-effect trajectory.

Deployment evidence similarly favors bounded augmentation. Assistance improved productivity in customer support and software development studies while people retained review and release responsibility [10, 23]. Human–AI advertising teams produced more output and higher-quality text, whereas human–human teams produced higher-quality images; greater AI delegation also increased homogeneity [67]. These outcomes are not one effect size, but they show why deployment claims must retain the baseline, authority, human work, accepted outcome, and downstream correction.

Dependable composition therefore follows proposal–gate–execute–verify. A model may interpret evidence and propose a claim, plan, tool call, memory update, or adaptation; a governed controller checks schema, provenance, scope, policy, budget, and preconditions before trusted state changes. Execution returns a receipt reconciled with the intended postcondition, and uncertain validation triggers repair, rollback, escalation, or an explicitly incomplete state. Delegation and adaptation may extend capability, but must not silently expand authority.

The credible path forward is disciplined, evidence-bearing composition: use the simplest execution form that can meet the goal; make state, authority, uncertainty, and obligations explicit; treat model outputs as proposals rather than facts or permissions; verify effects against the world; retain enough lineage to repair failure; and stop when evidence, authority, or resources no longer support another action. A dependable agentic system is not the one that acts most often, but the one that can show why an action was justified, who authorized it, what changed, whether the intended effect holds, and when not to act.

References

[1] A2A Project. Agent2agent protocol specification, version 1.0.0. Official specification, 2026. URL https://a2a-protocol.org/latest/specification/.

[2] Reyna Abhyankar, Qi Qi, and Yiying Zhang. OSWorld-Human: Benchmarking the efficiency of computer-use agents. *arXiv preprint arXiv:2506.16042*, 2025.

[3] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*, 2024.

[4] Sanket Badhe and Priyanka Tiwari. Agent skill security: Threat models, attacks, defenses, and evaluation. *arXiv preprint arXiv:2607.13987*, 2026. doi: 10.48550/arXiv.2607.13987. URL https://arxiv.org/abs/2607.13987.

[5] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025. doi: 10.48550/arXiv.2506.07982. URL https://arxiv.org/abs/2506.07982.

[6] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023. doi: 10.1038/s41586-023-06792-0. URL https://www.nature.com/articles/s41586-023-06792-0.

[7] Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier De Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. WorkArena++: Towards compositional planning and reasoning-based common knowledge work tasks. In *Advances in Neural Information Processing Systems: Datasets and Benchmarks Track*, volume 37, 2024. doi: 10.52202/079017-0195. URL https://papers.nips.cc/paper/2024/

hash/0b82662b6c32e887bb252a74d8cb2d5e-Abstract-Datasets_and_Benchmarks_Track.html.

[8] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6:525–535, 2024. doi: 10.1038/s42256-024-00832-8.

[9] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[10] Erik Brynjolfsson, Danielle Li, and Lindsey R. Raymond. Generative AI at work. *The Quarterly Journal of Economics*, 140(2):889–942, 2025. doi: 10.1093/qje/qjae044.

[11] Ömer Veysel Çağatan and Xuandong Zhao. Reward hacking in language model agents: Revisiting AI safety gridworlds. *arXiv preprint arXiv:2606.15385*, 2026. doi: 10.48550/arXiv.2606.15385. URL https://arxiv.org/abs/2606.15385.

[12] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi: 10.52202/085713-4082. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/b1041e52d3be19f0a9bc491657488e4a-Abstract-Datasets_and_Benchmarks_Track.html.

[13] Jialuo Chen, Lingqi Jiang, Xinhao Deng, Xiaohu Du, Jianan Ma, Yunhao Feng, Yuqi Qing, Zhihao Yuan, Linkang Du, and Jingyi Wang. When experience becomes instruction: Trajectory poisoning in self-evolving agent skill systems. *arXiv preprint arXiv:2608.05563*, 2026. doi: 10.48550/arXiv.2608.05563. URL https://arxiv.org/abs/2608.05563.

[14] Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. Seeing the whole elephant: A benchmark for failure attribution in LLM-based multi-agent systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 19888–19905. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.912. URL https://aclanthology.org/2026.acl-long.912/.

[15] Simin Chen, Yiming Chen, Zexin Li, Yifan Jiang, Zhongwei Wan, Yixin He, Dezhi Ran, Tianle Gu, Haizhou Li, Tao Xie, and Baishakhi Ray. Benchmarking large language models under data contamination: A survey from static to dynamic evaluation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 10080–10098. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.511. URL https://aclanthology.org/2025.emnlp-main.511/.

[16] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium*, pages 2383–2400, 2025. URL https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe.

[17] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.

[18] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds. In *Proceedings of the Eighth Conference on Machine Learning and Systems*, 2025. URL https://openreview.net/forum?id=3EXBLwGxtq.

[19] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-4136.

[20] Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/f12b4df26344f3be803c06b555252efe-Abstract-Conference.html.

[21] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[22] Alankrit Chona, Igor Kozlov, and Ambuj Kumar. Cyber defense benchmark: Agentic threat hunting evaluation for LLMs in SecOps. *arXiv preprint arXiv:2604.19533*, 2026. doi: 10.48550/arXiv.2604.19533. URL https://arxiv.org/abs/2604.19533.

[23] Kevin Zheyuan Cui, Mert Demirer, Sonia Jaffe, Leon Musolff, Sida Peng, and Tobias Salz. The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers. *Management Science*, 2026. doi: 10.1287/mnsc.2025.00535. URL https://doi.org/10.1287/mnsc.2025.00535.

[24] Pritam Dash, Tongyu Ge, Aditi Jain, Tanmay Shah, and Zhiwei Shang. From untrusted input to trusted memory: A systematic study of memory poisoning attacks in LLM agents. *arXiv preprint arXiv:2606.04329*, 2026. URL https://arxiv.org/abs/2606.04329.

[25] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-2636.

[26] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.

[27] deepset. Haystack agent state. Official documentation, 2026. URL https://docs.haystack.deepset.ai/docs/state.

[28] Mert Demirer, Leon Musolff, and Liyuan Yang. Writing code vs. shipping code: Productivity effects across generations of AI coding tools. Working Paper 35275, National Bureau of Economic Research, 2026. URL https://www.nber.org/papers/w35275.

[29] Eleanor W. Dillon, Sonia Jaffe, Nicole Immorlica, and Christopher T. Stanton. Shifting work patterns with generative AI. Working Paper 33795, National Bureau of Economic Research, 2025. URL https://www.nber.org/papers/w33795.

[30] Liming Dong, Qinghua Lu, and Liming Zhu. AgentOps: Enabling observability of LLM agents. *arXiv preprint arXiv:2411.05285*, 2024. URL https://arxiv.org/abs/2411.05285.

[31] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen J. Xiang. Memory injection attacks on LLM agents via query-only interaction. In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi: 10.52202/085713-1554.

[32] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks? In *International Conference on Machine Learning*, 2024.

[33] Mingxuan Du, Benfeng Xu, Chiwei Zhu, Xiaorui Wang, and Zhendong Mao. DeepResearch Bench: A comprehensive benchmark for deep research agents. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=hQ0K2Hhq7H.

[34] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning*, 2024.

[35] Mica R. Endsley and Esin O. Kiris. The out-of-the-loop performance problem and level of control in automation. *Human Factors*, 37(2):381–394, 1995. doi: 10.1518/001872095779064555.

[36] Lutfi Eren Erdogan, Hiroki Furuta, Sehoon Kim, Nicholas Lee, Suhong Moon, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Plan-and-act: Improving planning of agents for long-horizon tasks. In *International Conference on Machine Learning*, 2025.

[37] Kutluhan Erol, James Hendler, and Dana S. Nau. HTN planning: Complexity and expressivity. In *Proceedings of the National Conference on Artificial Intelligence*, pages 1123–1128, 1994.

[38] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024.

[39] Yunhao Feng, Yige Li, Yutao Wu, Yingshui Tan, Yanming Guo, Yifan Ding, Kun Zhai, Xingjun Ma, and Yu-Gang Jiang. BackdoorAgent: A unified framework for backdoor attacks on LLM-based agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 16115–16127. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.791. URL https://aclanthology.org/2026.findings-acl.791/.

[40] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP semantics. Technical Report RFC 9110, RFC Editor, 2022. URL https://www.rfc-editor.org/rfc/rfc9110.html.

[41] Richard E. Fikes and Nils J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.

[42] Foundation for Intelligent Physical Agents. FIPA communicative act library specification. Technical Report SC00037J, Foundation for Intelligent Physical Agents, 2002. URL https://www.fipa.org/specs/fipa00037/SC00037J.html.

[43] Romain Froger, Pierre Andrews, Matteo Bettini, Amar Budhiraja, Ricardo Silveira Cabral, Virginie Do, Emilien Garreau, Jean-Baptiste Gaya, Hugo Laurençon, Maxime Lecanu, Kunal Malkan, Dheeraj Mekala, Pierre Ménard, Gerard Moreno-Torres Bertran, Ulyana Piterbarg, Mikhail Plekhanov, Mathieu Rita, Andrey Rusakov, Vladislav Vorotilov, Mengjue Wang, Ian Yu, Amine Benhalloum, Grégoire Mialon, and Thomas Scialom. Gaia2: Benchmarking LLM agents on dynamic and asynchronous environments. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2602.11964.

[44] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, pages 10764–10799, 2023.

[45] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6465–6488. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.398. URL https://aclanthology.org/2023.emnlp-main.398/.

[46] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

[47] Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pages 249–259, 1987. doi: 10.1145/38713.38742.

[48] Alireza Ghafarollahi and Markus J. Buehler. SciAgents: Automating scientific discovery through multi-agent intelligent graph reasoning. *arXiv preprint arXiv:2409.05556*, 2024.

[49] Samira Ghodratnama and Mehrdad Zakershahrak. Adapting LLMs for efficient, personalized information retrieval: Methods and implications. In *Service-Oriented Computing – ICSOC 2023 Workshops*, volume 14518 of *Lecture Notes in Computer Science*, pages 17–26. Springer, 2024. doi: 10.1007/978-981-97-0989-2_2.

[50] Samira Ghodratnama and Mehrdad Zakershahrak. SumRecom: A personalized summarization approach by learning from users' feedback. *arXiv preprint arXiv:2408.07294*, 2024. doi: 10.48550/arXiv.2408.07294.

[51] Samira Ghodratnama, Amin Beheshti, and Mehrdad Zakershahrak. A personalized reinforcement learning summarization service for learning structure from unstructured data. In *2023 IEEE International Conference on Web Services (ICWS)*, pages 206–213. IEEE, 2023. doi: 10.1109/ICWS60048.2023.00040.

[52] Google. Agent development kit documentation. Official documentation, 2026. URL https://google.github.io/adk-docs/.

[53] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

[54] Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, Adam Chilton, et al. LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. In *Advances in Neural Information Processing Systems: Datasets and Benchmarks Track*, volume 36, 2023. doi: 10.52202/075280-1915.

URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/89e44582fd28ddfea1ea4dcb0ebbf4b0-Abstract-Datasets_and_Benchmarks.html.

[55] Dadi Guo, Tianyi Zhou, Dongrui Liu, Chen Qian, Qihan Ren, Shuai Shao, Zhiyuan Fan, Yi R. Fung, Kun Wang, Linfeng Zhang, and Jing Shao. Towards self-evolving agent benchmarks: Validatable agent trajectory via test-time exploration. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=2H03gm4Rq6.

[56] Norman Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988. doi: 10.1145/54289.871709.

[57] Jeff Hawkins. *A Thousand Brains: A New Theory of Intelligence*. Basic Books, 2021.

[58] Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the Third International Joint Conference on Artificial Intelligence*, pages 235–245, 1973.

[59] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.

[60] Yuan-Tung Hsieh et al. AgentBoard: An analytical evaluation board of multi-turn LLM agents. In *International Conference on Learning Representations*, 2024.

[61] Kung-Hsiang Huang, Akshara Prabhakar, Onkar Thorat, Divyansh Agarwal, Prafulla Kumar Choubey, Yixin Mao, Silvio Savarese, Caiming Xiong, and Chien-Sheng Wu. CRMArena-Pro: Holistic assessment of LLM agents across diverse business scenarios and interactions. *arXiv preprint arXiv:2505.18878*, 2025. doi: 10.48550/arXiv.2505.18878. URL https://arxiv.org/abs/2505.18878.

[62] Zisu Huang, Muzhao Tian, Xiaohua Wang, Jingwen Xu, Zhengkang Guo, Qi Qian, Kaitao Song, Jiakang Yuan, Changze Lv, and Xiaoqing Zheng. Controllable memory usage: Balancing anchoring and innovation in long-term human–agent interaction. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14699–14719. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.670.

[63] Peter Jansen, Marc-Alexandre Côté, Tushar Khot, Erin Bransom, Bhavana Dalvi Mishra, Bodhisattwa Prasad Majumder, Øyvind Tafjord, and Peter Clark. Discoveryworld: A virtual environment for developing and evaluating automated scientific discovery agents. In *NeurIPS Datasets and Benchmarks Track*, volume 37, 2024. doi: 10.52202/079017-0324. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/13836f251823945316ae067350a5c366-Abstract-Datasets_and_Benchmarks_Track.html.

[64] Saurabh Jha, Rohan Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, Jackson Clark, Bhavya Bhavya, Mudit Verma, Harshit Kumar, Hirokuni Kitahara, et al. IT-Bench: Evaluating AI agents across diverse real-world IT automation tasks. *arXiv preprint arXiv:2502.05352*, 2025. doi: 10.48550/arXiv.2502.05352. URL https://arxiv.org/abs/2502.05352.

[65] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[66] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[67] Harang Ju and Sinan Aral. Collaborating with AI agents: Field experiments on teamwork, productivity, and performance. *arXiv preprint arXiv:2503.18238*, 2026. doi: 10.48550/arXiv.2503.18238. URL https://arxiv.org/abs/2503.18238.

[68] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

[69] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *Transactions on Machine Learning Research*, 2025. URL https://openreview.net/forum?id=zB3irY2PkH.

[70] Eli M. Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Hofit Bata, Yoav Levine, Kevin Leyton-Brown, et al. MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. *arXiv preprint arXiv:2205.00445*, 2022.

[71] Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik Siu Chan, Xuhai Xu, Daniel McDuff, Hyeonhoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, and Hae Won Park. MDAgents: An adaptive collaboration of LLMs for medical decision-making. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-2522. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/90d1fc07f46e31387978b88e7e057a31-Abstract-Conference.html.

[72] Nachiket Kotalwar, Rohini Das, and Carolyn Rosé. Searching for synergy in shared-workspace human–AI collaboration. *arXiv preprint arXiv:2606.18413*, 2026. doi: 10.48550/arXiv.2606.18413. URL https://arxiv.org/abs/2606.18413.

[73] Lars Krupp, Daniel Geißler, Vishal Banwari, Paul Lukowicz, and Jakob Karolus. Promoting sustainable web agents: Benchmarking and estimating energy consumption through empirical and theoretical analysis. *arXiv preprint arXiv:2511.04481*, 2025. doi: 10.48550/arXiv.2511.04481. URL https://arxiv.org/abs/2511.04481.

[74] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978. doi: 10.1145/359545.359563.

[75] LangChain. LangGraph persistence and interrupts. Official documentation, 2026. URL https://docs.langchain.com/oss/python/langgraph/persistence.

[76] Christopher Latimer, Nicolò Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight: Structured agent memory that retains, recalls, and reflects. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 275–285. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-demo.27.

[77] Sung Une Lee, Liming Zhu, Md Shamsujjoha, Liming Dong, Qinghua Lu, Jieshan Chen, and Lionel Briand. A structured approach to safety case construction for AI systems. *arXiv preprint arXiv:2601.22773*, 2026. URL https://arxiv.org/abs/2601.22773.

[78] Nancy G. Leveson. *Engineering a Safer World: Systems Thinking Applied to Safety*. The MIT Press, 2012. ISBN 9780262016629. doi: 10.7551/mitpress/8179.001.0001.

[79] Ido Levy, Ben Wiesel, Sami Marreed, Alon Oved, Avi Yaeli, and Segev Shlomov. ST-webagentbench: A benchmark for evaluating safety and trustworthiness in web agents. In *International Conference on Learning Representations*, 2026.

[80] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[81] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for mind exploration of large scale language model society. *arXiv preprint arXiv:2303.17760*, 2023.

[82] Haitao Li, Junjie Chen, Jingli Yang, Qingyao Ai, Wei Jia, Youfeng Liu, Kai Lin, Yueyue Wu, Guozhi Yuan, Yiran Hu, Wuyue Wang, Yiqun Liu, and Minlie Huang. LegalAgentBench: Evaluating LLM agents in legal domain. *arXiv preprint arXiv:2412.17259*, 2024. doi: 10.48550/arXiv.2412.17259. URL https://arxiv.org/abs/2412.17259.

[83] Keyu Li, Junhao Shi, Yang Xiao, Mohan Jiang, Jie Sun, Yunze Wu, Dayuan Fu, Shijie Xia, Xiaojie Cai, Tianze Xu, Weiye Si, Wenjie Li, Dequan Wang, and Pengfei Liu. AgencyBench: Benchmarking the frontiers of autonomous agents in 1M-token real-world contexts. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 7422–7440, 2026. doi: 10.18653/v1/2026.acl-long.337.

[84] Meng Li, Lei Li, Xiting Wang, Yi Yuan, Zheng Wei, Brucebian, and Zang Li. SOAR: Supervision from observation for agentic reinforcement learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 35175–35197, 2026. doi: 10.18653/v1/2026.acl-long.1624.

[85] Xiangyi Li, Kyoung Whan Choe, Yimin Liu, Xiaokun Chen, Chujun Tao, Bingran You, Wenbo Chen, Zonglin Di, Jiankai Sun, Shenghan Zheng, Jiajun Bao, Yuanli Wang, Weixiang Yan, Yiyuan Li, and Han chung Lee. ClawsBench: Evaluating capability and safety of LLM productivity agents in simulated workspaces. *arXiv preprint arXiv:2604.05172*, 2026. doi: 10.48550/arXiv.2604.05172. URL https://arxiv.org/abs/2604.05172.

[86] Weixin Liang, Nazneen Rajani, Xinyu Yang, Ezinwanne Ozoani, Eric Wu, Yiqun Chen, Daniel Scott Smith, and James Zou. What's documented in AI? systematic analysis of 32k AI model cards. *arXiv preprint arXiv:2402.05160*, 2024. URL https://arxiv.org/abs/2402.05160.

[87] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[88] Xiao Liu et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.

[89] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522, 2023. doi: 10.18653/v1/2023.emnlp-main.153. URL https://aclanthology.org/2023.emnlp-main.153/.

[90] Yanming Liu, Xinyue Peng, Jiannan Cao, Xinyi Wang, Songhang Deng, Jintao Chen, Jianwei Yin, and Xuhong Zhang. ToolGate: Contract-grounded and verified tool execution for LLMs. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 9653–9684. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.470. URL https://aclanthology.org/2026.findings-acl.470/.

[91] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

[92] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-naacl.65. URL https://aclanthology.org/2025.findings-naacl.65/.

[93] Miao Lu, Weiwei Sun, Weihua Du, Zhan Ling, Xuesong Yao, Kang Liu, and Jiecao Chen. Beyond the context window: Scaling agentic RL via end-to-end optimized context compression. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 21074–21125, 2026. doi: 10.18653/v1/2026.acl-long.966.

[94] Weitao Ma, Xiaocheng Feng, Lei Huang, Xiachong Feng, Zhanyu Ma, Jun Xu, Jiuchong Gao, Jinghua Hao, Renqing He, and Bing Qin. Fine-Mem: Fine-grained feedback alignment for long-horizon memory management. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 19666–19684. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.900.

[95] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. Query rewriting for retrieval-augmented large language models. *arXiv preprint arXiv:2305.14283*, 2023.

[96] Ziyu Ma, Hailang Huang, Shun Zou, Yong Wang, Shidong Yang, Yiming Hu, Fei Wei, and XiangXiang Chu. LongHorizon-Harness: Advancing long-horizon agents for real-world tasks. *arXiv preprint arXiv:2608.01964*, 2026.

[97] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.

[98] Varun Magesh, Faiz Surani, Matthew Dahl, Mirac Suzgun, Christopher D. Manning, and Daniel E. Ho. Hallucination-free? assessing the reliability of leading AI legal research tools. *Journal of Empirical Legal Studies*, 22(2):216–242, 2025. doi: 10.1111/jels.12413. URL https://onlinelibrary.wiley.com/doi/10.1111/jels.12413.

[99] Isham Kalappurackal Mansoor, Abhishek Phadke, and Pratip Rana. Verified tool calls improve LLM agent reliability under non-atomic failures. *arXiv preprint arXiv:2608.02645*, 2026. URL https://arxiv.org/abs/2608.02645.

[100] David Marr. *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. W. H. Freeman, San Francisco, 1982.

[101] Adam Mazzocchetti. Runtime governance for agentic AI: Action-boundary control with trusted provenance and fail-closed execution. *arXiv preprint arXiv:2608.16891*, 2026. doi: 10.5281/zenodo.20262303. URL https://arxiv.org/abs/2608.16891.

[102] Drew McDermott et al. PDDL: The planning domain definition language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, 1998.

[103] Alfredo Metere. Attested tool-server admission: A security extension to the model context protocol. *arXiv preprint arXiv:2605.24248*, 2026. doi: 10.48550/arXiv.2605.24248. URL https://arxiv.org/abs/2605.24248.

[104] Gregoire Mialon, Clementine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. In *International Conference on Learning Representations*, 2024.

[105] Microsoft. Semantic kernel agent architecture and orchestration. Microsoft Learn documentation, 2025. URL https://learn.microsoft.com/en-us/semantic-kernel/frameworks/agent/agent-architecture.

[106] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long-form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12076–12100. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.741. URL https://aclanthology.org/2023.emnlp-main.741/.

[107] Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. SWE-Lancer: Can frontier LLMs earn $1 million from real-world freelance software engineering? *arXiv preprint arXiv:2502.12115*, 2025. doi: 10.48550/arXiv.2502.12115. URL https://arxiv.org/abs/2502.12115.

[108] Abhinav Mishra and Kumar Sharad. Observability for delegated execution in agentic AI systems. *arXiv preprint arXiv:2606.09692*, 2026. URL https://arxiv.org/abs/2606.09692.

[109] MITRE. ATLAS: Adversarial threat landscape for artificial-intelligence systems. https://atlas.mitre.org/, 2026. Accessed July 2026.

[110] Model Context Protocol Contributors. Model context protocol specification, revision 2025-11-25. Official specification, 2025. URL https://modelcontextprotocol.io/specification/2025-11-25.

[111] Bardia Mohammadi, Nearchos Potamitis, Lars Klein, Akhil Arora, and Laurent Bindschaedler. Atomix: Timely, transactional tool use for reliable agentic workflows. *arXiv preprint arXiv:2602.14849*, 2026. URL https://arxiv.org/abs/2602.14849.

[112] John Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander Rush. Text embeddings reveal (almost) as much as text. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12448–12460, 2023. doi: 10.18653/v1/2023.emnlp-main.765.

[113] Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. ToolSafe: Enhancing tool invocation safety of LLM-based agents via proactive step-level guardrail and feedback. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 37125–37153, 2026. doi: 10.18653/v1/2026.findings-acl.1850.

[114] Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 7076–7087. PMLR, 2020. URL https://proceedings.mlr.press/v119/mozannar20b.html.

[115] Hussein Mozannar, Gagan Bansal, Cheng Tan, Adam Fourney, Victor Dibia, Jingya Chen, Jack Gerrits, Tyler Payne, Matheus Kunzler Maldaner, Madeleine Grunde-McLaughlin, Eric Zhu, Griffin Bassman, Jacob Alber, Peter Chang, Ricky Loynd, Friederike Niedtner, Ece Kamar, Maya Murad, Rafah Hosn, and Saleema Amershi. Magentic-UI: Towards human-in-the-loop agentic systems. *arXiv preprint arXiv:2507.22358*, 2025. URL https://arxiv.org/abs/2507.22358.

[116] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.

[117] National Institute of Standards and Technology. Artificial intelligence risk management framework: Generative artificial intelligence profile. Technical Report NIST AI 600-1, National Institute of Standards and Technology, 2024.

[118] Alexander Nelson, Sanjay Rekhi, Murugiah Souppaya, and Karen Scarfone. Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile. Special Publication 800-61 Rev. 3, National Institute of Standards and Technology, 2025. URL https://csrc.nist.gov/pubs/sp/800/61/r3/final.

[119] H. Penny Nii. Blackboard systems: The blackboard model of problem solving and the evolution of blackboard architectures. *AI Magazine*, 7(2):38–53, 1986. doi: 10.1609/aimag.v7i2.537.

[120] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, KaShun Shum, Randy Zhong, Juntong Song, and Tong Zhang. RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10862–10878. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.585. URL https://aclanthology.org/2024.acl-long.585/.

[121] OpenAI. New tools for building agents. Technical announcement, 2025. URL https://openai.com/index/new-tools-for-building-agents/.

[122] OpenAI. Why SWE-bench Verified no longer measures frontier coding capabilities. Technical report, 2026. URL https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/.

[123] OpenTelemetry. Opentelemetry GenAI semantic conventions. OpenTelemetry specification repository, 2026. URL https://github.com/open-telemetry/semantic-conventions-genai.

[124] OWASP Foundation. OWASP top 10 for large language model applications 2025. https://genai.owasp.org/llm-top-10/, 2025.

[125] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2024.

[126] Raja Parasuraman and Dietrich H. Manzey. Complacency and bias in human use of automation: An attentional integration. *Human Factors*, 52(3):381–410, 2010. doi: 10.1177/0018720810376055.

[127] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, 30(3):286–297, 2000. doi: 10.1109/3468.844354.

[128] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. doi: 10.1145/3586183.3606763.

[129] Shishir G. Patil, Tianjun Zhang, Vivian Fang, Noppapon C., Roy Huang, Aaron Hao, Martin Casado, Joseph E. Gonzalez, Raluca Ada Popa, and Ion Stoica. GoEX: Perspectives and designs towards a runtime for autonomous LLM applications. *arXiv preprint arXiv:2404.06921*, 2024. URL https://arxiv.org/abs/2404.06921.

[130] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *International Conference on Machine Learning*, 2025.

[131] Aske Plaat, Max van Duijn, Niki van Stein, Mike Preuss, Peter van der Putten, and Kees Joost Batenburg. Agentic large language models, a survey. *Journal of Artificial Intelligence Research*, 84, 2025. doi: 10.1613/jair.1.18675.

[132] Yujia Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*, 2024.

[133] Anand S. Rao and Michael P. Georgeff. BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multiagent Systems*, pages 312–319, 1995.

[134] Anita Rao, Andrew Keller, Neha Kalra, Ryan Steed, Kweku Kwegyir-Aggrey, Kevin Klyman, Diane Staheli, and Amanda Bergman. Challenges to the monitoring of deployed AI systems. NIST Trustworthy and Responsible AI 800-4, National Institute of Standards and Technology, 2026. URL https://doi.org/10.6028/NIST.AI.800-4.

[135] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. AndroidWorld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024. doi: 10.48550/arXiv.2405.14573. URL https://arxiv.org/abs/2405.14573.

[136] Zhe Ren, Yimeng Chen, Dandan Guo, Guowei Rong, Tonghui Li, R. B. Xiong, Qingfeng Lan, Wenyi Wang, Nanbo Li, Yibo Yang, Mingchen Zhuge, and Jürgen Schmidhuber. Self-improvements in modern agentic systems: A survey. *arXiv preprint arXiv:2607.13104*, 2026. doi: 10.48550/arXiv.2607.13104.

[137] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.

[138] Ron Ross, Mark Winstead, and Michael McEvilley. Engineering trustworthy secure systems. Special Publication 800-160 Vol. 1 Rev. 1, National Institute of Standards and Technology, 2022. URL https://csrc.nist.gov/pubs/sp/800/160/v1/r1/final.

[139] Royal Commission into the Robodebt Scheme. Report of the royal commission into the robodebt scheme. Technical report, Commonwealth of Australia, 2023. URL https://robodebt.royalcommission.gov.au/publications/report.

[140] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=GEcwtMk1uA.

[141] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2021.

[142] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.

[143] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. *arXiv preprint arXiv:2401.18059*, 2024.

[144] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[145] Chris Schmitz, Jonathan Rystrøm, and Jan Batzner. Oversight structures for agentic AI in public-sector organizations. In *Proceedings of the 1st Workshop for Research on Agent Language Models (REALM 2025)*, pages 298–308. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.realm-1.21. URL https://aclanthology.org/2025.realm-1.21/.

[146] Yijia Shao, Vinay Samuel, Yucheng Jiang, John Yang, and Diyi Yang. Collaborative gym: A framework for enabling and evaluating human–agent collaboration. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=GDYueXtKXT.

[147] Yiting Shen, Kun Li, Wei Zhou, and Songlin Hu. Mem2ActBench: A benchmark for evaluating long-term memory utilization in task-oriented autonomous agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 8173–8190, 2026. doi: 10.18653/v1/2026.acl-long.370.

[148] Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Rich James, Mike Lewis, Luke Zettlemoyer, and Wen tau Yih. REPLUG: Retrieval-augmented black-box language models. *arXiv preprint arXiv:2301.12652*, 2023.

[149] Zhengliang Shi, Shen Gao, Lingyong Yan, Yue Feng, Xiuyi Chen, Zhumin Chen, Dawei Yin, Suzan Verberne, and Zhaochun Ren. Tool learning in the wild: Empowering language models as automatic tool agents. In *Proceedings of the ACM Web Conference*, pages 2222–2237, 2025.

[150] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[151] Ben Shneiderman. Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human–Computer Interaction*, 36(6):495–504, 2020. doi: 10.1080/10447318.2020.1741118.

[152] Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. Agentic retrieval-augmented generation: A survey on agentic RAG. *arXiv preprint arXiv:2501.09136*, 2025. doi: 10.48550/arXiv.2501.09136.

[153] SLSA Community. SLSA specification, version 1.2: Provenance. Official specification, 2026. URL https://slsa.dev/spec/v1.2/provenance.

[154] Andries Petrus Smit, Nathan Grinsztajn, Paul Duckworth, Thomas D. Barrett, and Arnu Pretorius. Should we be going MAD? a look at multi-agent debate strategies for LLMs. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 45883–45905, 2024. URL https://proceedings.mlr.press/v235/smit24a.html.

[155] Reid G. Smith. The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980. doi: 10.1109/TC.1980.1675516.

[156] Yixiao Song, Yekyung Kim, and Mohit Iyyer. VeriScore: Evaluating the factuality of verifiable claims in long-form text generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 9447–9474. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-emnlp.552. URL https://aclanthology.org/2024.findings-emnlp.552/.

[157] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=1i6ZCvflQJ.

[158] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112 (1–2):181–211, 1999.

[159] Nathan J. Szymanski, Bernard Rendy, Yuxing Fei, Rishi E. Kumar, Tanjin He, David Milsted, Matthew J. McDermott, Max Gallant, Ekin Dogus Cubuk, Amil Merchant, et al. An autonomous laboratory for the accelerated synthesis of inorganic materials. *Nature*, 624: 86–91, 2023. doi: 10.1038/s41586-023-06734-w.

[160] Nathan J. Szymanski, Bernardus Rendy, Yuxing Fei, Rishi E. Kumar, Tanjin He, David Milsted, Matthew J. McDermott, Max Gallant, Ekin Dogus Cubuk, Amil Merchant, Haegyeom Kim, Anubhav Jain, Christopher J. Bartel, Kristin Persson, Yan Zeng, and Gerbrand Ceder. Author correction: An autonomous laboratory for the accelerated synthesis of inorganic materials. *Nature*, 650:E1, 2026. doi: 10.1038/s41586-025-09992-y. URL https://www.nature.com/articles/s41586-025-09992-y.

[161] Elham Tabassi. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, 2023.

[162] Elham Tabassi. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, 2023.

[163] Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. MemBench: Towards more comprehensive evaluation on the memory of LLM-based agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19336–19352, 2025. doi: 10.18653/v1/2025.findings-acl.989.

[164] Minh V. T. Thai, Tue Le, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*, 2025. doi: 10.48550/arXiv.2512.18470. URL https://arxiv.org/abs/2512.18470.

[165] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1393–1410. USENIX Association, 2019. URL https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.

[166] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 16022–16076, 2024. doi: 10.18653/v1/2024.acl-long.850.

[167] Tao Tu, Mike Schaekermann, Anil Palepu, Khaled Saab, Jan Freyberg, Ryutaro Tanno, Amy Wang, Brenna Li, Mohamed Amin, Yong Cheng, Elahe Vedadi, Nenad Tomasev, Shekoofeh Azizi, Karan Singhal, Le Hou, Albert Webson, Kavita Kulkarni, S. Sara Mahdavi, Christopher Semturs, Juraj Gottweis, Joelle Barral, Katherine Chou, Greg S. Corrado, Yossi Matias, Alan Karthikesalingam, and Vivek Natarajan. Towards conversational diagnostic artificial intelligence. *Nature*, 642:442–450, 2025. doi: 10.1038/s41586-025-08866-7. URL https://www.nature.com/articles/s41586-025-08866-7.

[168] Wil M. P. van der Aalst, Arthur H. M. ter Hofstede, Bartek Kiepuszewski, and Alistair P. Barros. Workflow patterns. *Distributed and Parallel Databases*, 14(1):5–51, 2003. doi: 10.1023/A:1022883727209.

[169] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[170] Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, Jerry Wei, Jason Wei, Chris Tar, Yun-Hsuan Sung, Denny Zhou, Quoc V. Le, and Thang Luong. FreshLLMs: Refreshing large language models with search engine augmentation. *arXiv preprint arXiv:2310.03214*, 2023. doi: 10.48550/arXiv.2310.03214.

[171] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024.

[172] Bo Wang, Weiyi He, Shenglai Zeng, Zhen Xiang, Yue Xing, Jiliang Tang, and Pengfei He. Unveiling privacy risks in LLM agent memory. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 25241–25260. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1227. URL https://aclanthology.org/2025.acl-long.1227/.

[173] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[174] Junlin Wang, Siddhartha Jain, Dejiao Zhang, Baishakhi Ray, Varun Kumar, and Ben Athiwaratkun. Reasoning in token economies: Budget-aware evaluation of LLM reasoning strategies. In *Proceedings of the 2024 Conference on Empirical Methods in Natural*

Language Processing, pages 19916–19939. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.1112. URL https://aclanthology.org/2024.emnlp-main.1112/.

[175] Lehan Wang, Boli Chen, Ruixue Ding, Pengjun Xie, Jinwei Huang, Zhendong Liu, Shuo Wang, Tao Lei, Xin Ouyang, and Xiaomeng Li. SecRespond: Benchmarking AI agents for real-world post-compromise incident response. *arXiv preprint arXiv:2607.26791*, 2026. doi: 10.48550/arXiv.2607.26791. URL https://arxiv.org/abs/2607.26791.

[176] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 2609–2634, 2023.

[177] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024. doi: 10.1007/s11704-024-40231-1.

[178] Leo Wang, Lihai Yang, Boyu Chen, Kerun Xu, Gongyi Zou, Bo Tang, Feiyu Xiong, Siheng Chen, and Zhiyu Li. Text2Mem: A unified memory operation language for memory operating system. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 2105–2119. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.100.

[179] Liang Wang, Nan Yang, and Furu Wei. Query2doc: Query expansion with large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[180] Yiwei Wang, Chuan Zhu, Tianjun Feng, Lauren Xiaoyuan Lu, and Bingxin Jia. Agentic AI and human-in-the-loop interventions: Field experimental evidence from alibaba's customer service operations. *arXiv preprint arXiv:2605.14830*, 2026. URL https://arxiv.org/abs/2605.14830.

[181] Yongjie Wang, Xinyue Zhang, Kunhong Yao, Zhiwei Zeng, Kaisong Song, Jun Lin, and Zhiqi Shen. Search-time contamination in deep research agents: Measuring performance inflation in public benchmark evaluation. *arXiv preprint arXiv:2606.05241*, 2026. doi: 10.48550/arXiv.2606.05241. URL https://arxiv.org/abs/2606.05241.

[182] Amy A. Winecoff and Miranda Bogen. Improving governance outcomes through AI documentation: Bridging theory and practice. *arXiv preprint arXiv:2409.08960*, 2024. URL https://arxiv.org/abs/2409.08960.

[183] Michael Wooldridge. *An Introduction to MultiAgent Systems*. Wiley, 2 edition, 2009.

[184] Jie Wu, Ming Gong, Feixiang Cheng, and Qinqin Zhao. EcoAgent-Bench: Evaluating economic decision-making in budget-constrained LLM agents. *arXiv preprint arXiv:2608.05519*, 2026. doi: 10.48550/arXiv.2608.05519. URL https://arxiv.org/abs/2608.05519.

[185] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[186] Yaozu Wu, Wei-Chieh Huang, Jizhou Guo, Dongyuan Li, Renhe Jiang, Henry Peng Zou, Chunyu Miao, Shanghao Li, Weizhi Zhang, WeiWei Ye, Yankai Chen, Meng Zhang, Xue Liu, and Philip S. Yu. HAS-Bench: Evaluating LLM-based human–agent systems under configurable human participation. *arXiv preprint arXiv:2607.04329*, 2026. doi: 10.48550/arXiv.2607.04329. URL https://arxiv.org/abs/2607.04329.

[187] Yiran Wu, Tianwei Yue, Shaokun Zhang, Chi Wang, and Qingyun Wu. StateFlow: Enhancing LLM task-solving through state-driven workflows. In *Proceedings of the First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=3nTbuygoop.

[188] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2):121101, 2025.

[189] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024. doi: 10.48550/arXiv.2407.01489. URL https://arxiv.org/abs/2407.01489.

[190] Dawei Xiang, Kexin Chu, Wenyan Xu, Wenhui Zhang, and Wei Zhang. LLM-as-scheduler: Agentic workflow dynamic scheduling. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12752–12763. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.581. URL https://aclanthology.org/2026.acl-long.581/.

[191] Qianqian Xie, Weiguang Han, Zhengyu Chen, Ruoyu Xiang, Xiao Zhang, et al. FinBen: A holistic financial benchmark for large language models. In *Advances in Neural Information Processing Systems: Datasets and Benchmarks Track*, volume 37, 2024. doi: 10.52202/079017-3033. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/adb1d9fa8be4576d28703b396b82ba1b-Abstract-Datasets_and_Benchmarks_Track.html.

[192] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.

[193] Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts LLM agents: An empirical study of experience-following behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 623–645. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.27. URL https://aclanthology.org/2026.acl-long.27/.

[194] Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2310.04408.

[195] Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, et al. Theagentcompany: Benchmarking LLM agents on consequential real-world tasks. In *Advances in Neural Information Processing Systems: Datasets and Benchmarks Track*, 2025. URL https://papers.nips.cc/paper_files/paper/2025/hash/0d744742f6fac4d1134c019b7cef3c8a-Abstract-Datasets_and_Benchmarks_Track.html.

[196] Peng Xu, Wei Ping, Xianchao Wu, Lawrence McAfee, Chen Zhu, Zihan Liu, Sandeep Subramanian, Evelina Bakhturina, Mohammad Shoeybi, and Bryan Catanzaro. Retrieval meets long-context large language models. In *International Conference on Learning Representations*, 2024.

[197] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

[198] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024.

[199] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

[200] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[201] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[202] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025. First released as arXiv:2406.12045 in 2024.

[203] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. A survey on evaluation of LLM-based agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 26690–26714, 2026. doi: 10.18653/v1/2026.findings-acl.1330.

[204] Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. AssistantBench: Can web agents solve realistic and time-consuming tasks? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8938–8968. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.505. URL https://aclanthology.org/2024.emnlp-main.505/.

[205] Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yuechen Jiang, Yupeng Cao, Zhi Chen, Jordan W. Suchow, Zhenyu Cui, Rong Liu, Zhaozhuo Xu, Denghui Zhang, Koduvayur Subbalakshmi, Guojun Xiong, Yueru He, Jimin Huang, Dong Li, and Qianqian Xie. FinCon: A synthesized LLM multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-4354. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/f7ae4fe91d96f50abc2211f09b6a7e49-Abstract-Conference.html.

[206] Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 21457–21483, 2026. doi: 10.18653/v1/2026.acl-long.981.

[207] Mengqi Yuan, Zilong Zhou, Xinzhuang Xiong, Weiming Wu, Jiayang Sun, Jiamin Song, Kaiqian Cui, Bowen Wang, Haoyuan Wu, Yitong Li, et al. OSWorld 2.0: Benchmarking computer-use agents on long-horizon real-world tasks. *arXiv preprint arXiv:2606.29537*, 2026.

[208] Mehrdad Zakershahrak and Samira Ghodratnama. Explanation, debate, align: A weak-to-strong framework for language model generalization. *arXiv preprint arXiv:2409.07335*, 2024. doi: 10.48550/arXiv.2409.07335.

[209] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, 2024. doi: 10.18653/v1/2024.findings-acl.624.

[210] Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 7116–7132, 2025. doi: 10.18653/v1/2025.findings-naacl.395.

[211] Dongsen Zhang, Zekun Li, Xu Luo, Xuannan Liu, Peipei Li, and Wenjun Xu. MCP Security Bench (MSB): Benchmarking attacks against model context protocol in LLM agents. *arXiv preprint arXiv:2510.15994*, 2025. URL https://arxiv.org/abs/2510.15994.

[212] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *International Conference on Learning Representations*, 2025.

[213] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025.

[214] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. *arXiv preprint arXiv:2308.10144*, 2024.

[215] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.

[216] Yusheng Zheng, Yanpeng Hu, Tong Yu, and Andi Quinn. AgentSight: System-level observability for AI agents using eBPF. *arXiv preprint arXiv:2508.02736*, 2025. doi: 10.1145/3766882.3767169. URL https://arxiv.org/abs/2508.02736.

[217] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731, 2024.

[218] Denny Zhou, Nathanael Scharli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations*, 2023.

[219] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

[220] Chiwei Zhu, Benfeng Xu, Mingxuan Du, Shaohan Wang, Xiaorui Wang, Zhendong Mao, and Yongdong Zhang. FS-researcher: Test-time scaling for long-horizon research tasks with file-system-based agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 6353–6373, 2026. doi: 10.18653/v1/2026.acl-long.288.

[221] Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. MultiAgentBench: Evaluating the collaboration and competition of LLM agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025. doi: 10.18653/v1/2025.acl-long.421.

[222] Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*, 2026. URL https://arxiv.org/abs/2607.18754.